

République Algérienne Démocratique et Populaire
الجمهورية الجزائرية الديمقراطية الشعبية
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
وزارة التعليم العالي و البحث العلمي



المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الإعلام الآلي سابقا)
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Mémoire de Master

Pour l'obtention du diplôme de Master en Informatique

Option : Systèmes d'information et technologies

Une étude des approches d'IA pour l'apprentissage adaptatif et continu dans la reconnaissance d'activités humaines

Réalisée par :

Mme. Nedjam Walaa

Encadrée par :

Pr. Attal Ferhat (LISSI, UPEC)

Dr. Meziani Lila (ESI)

Organisme d'accueil :

Laboratoire LISSI — Université Paris-Est Créteil (UPEC), France

École Nationale Supérieure d'Informatique (ESI), Alger

Promotion : **2025/2026**

Dédicace

À mes parents, pour leur soutien et leurs encouragements tout au long de mon parcours universitaire.

À mes enseignants et encadrants, pour la qualité de leur accompagnement.

À tous ceux qui m'ont aidée, de près ou de loin, à mener ce travail à bien.

— *Walaa Nedjam*

Remerciements

Tout d'abord, nous remercions Allah, le Tout-Puissant, de nous avoir accordé la force, la patience et la persévérance nécessaires à l'accomplissement de ce travail.

Nous exprimons notre gratitude la plus sincère à notre encadrante Dr. Meziani Lila de l'École Nationale Supérieure d'Informatique (ESI) d'Alger, et à notre promoteur Pr. Attal Ferhat du Laboratoire LISSI de l'Université Paris-Est Créteil Val de Marne (UPEC), pour la qualité de leur encadrement, la pertinence de leurs orientations et leur accompagnement constant tout au long de ce projet.

Nous adressons également nos remerciements à l'ensemble du corps enseignant de l'École Nationale Supérieure d'Informatique d'Alger pour la qualité de l'enseignement dispensé durant notre formation.

À toutes celles et ceux qui ont, de près ou de loin, contribué à l'aboutissement de ce projet, nous adressons notre plus sincère reconnaissance.

Résumé

Les capteurs inertiels des smartphones et montres permettent de reconnaître les activités humaines (HAR), avec des applications en santé et assistance. Un modèle entraîné une seule fois sur des données de laboratoire peine pourtant à suivre de nouveaux utilisateurs ou contextes : c'est l'oubli catastrophique. Par ailleurs, peu de systèmes anticipent une transition avant qu'elle ne se produise.

Ce mémoire propose une architecture HAR adaptative et continue, alignée sur la fiche PFE 26/2755. Après un état de l'art, nous décrivons un backbone Transformer partagé, la reconnaissance continue avec Uncertainty-Weighted Replay (UWR), l'anticipation multi-classe (tête LSTM sur fenêtres partielles), plus des modules complémentaires : SSL SimCLR-style, pipeline foundation-style, calibration de confiance en ligne, bandit de seuil (RL léger) et fall-risk binaire. L'évaluation sur HAPT et WISDM montre des gains nets par rapport à EWC, iCaRL et finetuning naïf (+16,8 points macro-F1 en user-incrémental). L'anticipation multi-classe atteint un macro-F1 de validation d'environ 0,60–0,64 (baseline majorité $\approx 0,04$) ; le fall-risk binaire atteint environ 0,88.

Le système `har_project` comprend plus de 531 000 paramètres pour le backbone, un tampon de rejeu de 2 000 fenêtres, et des modules CORAL, SSL, calibration, fall-risk et bandit de seuil. Les expériences couvrent le pré-entraînement (91,3 % accuracy HAPT), l'adaptation 44 sujets (F1 0,543), le class-incrémental six tâches, l'anticipation LSTM (macro-F1 $\approx 0,60$ –0,64), le fall-risk binaire ($\approx 0,88$), et l'adaptation cross-dataset CORAL (F1 0,629). Le scénario « préparation de repas » reste hors portée faute de corpus cuisine. Les annexes documentent datasets, hyperparamètres, code et fiches bibliographiques pour faciliter la reproduction.

Mots clés : Reconnaissance d'activités humaines, apprentissage continu, capteurs inertiels, Transformer, Uncertainty-Weighted Replay, anticipation d'activités, auto-supervision, calibration en ligne.

Abstract

Human activity recognition (HAR) from IMUs on phones and wearables supports mobile health and assisted living. Models trained once on lab data often fail when new users or contexts appear — catastrophic forgetting. Few systems anticipate activity changes before they occur.

This thesis presents an adaptive continual HAR architecture : a shared Transformer backbone, continual recognition with Uncertainty-Weighted Replay (UWR), and multi-class next-activity anticipation from partial windows (LSTM head, frozen backbone). Experiments on HAPT and WISDM show clear gains over EWC, iCaRL and naive fine-tuning (+16.8 macro-F1 points in user-incremental learning). Multi-class anticipation reaches validation macro-F1 ≈ 0.60 – 0.64 (majority baseline ≈ 0.04); binary fall-risk reaches ≈ 0.88 . Complementary modules cover SSL, foundation-style pretraining, online calibration and a light threshold bandit (RL).

Keywords : Human activity recognition, continual learning, inertial sensors, Transformer, Uncertainty-Weighted Replay, activity anticipation, self-supervised learning, online calibration.

ملخص

يعتمد التعرف على الأنشطة البشرية (HAR) من حساسات القصور الذاتي (IMU) في الهواتف والساعات على تطبيقات الصحة والمساعدة. النماذج المدربة مرة واحدة على بيانات مخبرية تفشل أمام مستخدمين أو سياقات جديدة (النسيان الكارثي)، وقليل منها يتوقع الانتقالات مسبقاً.

يقترح هذا المذكر بنية HAR تكيفية ومستمرة: عمود فقري Transformer، تعرف مستمر بـ UWR، وتوقع متعدد الأصناف عبر رأس LSTM على نوافذ جزئية. التجارب على HAPT و WISDM تظهر مكاسب واضحة مقارنة بـ EWC و iCaRL والضبط البسيط.

الكلمات المفتاحية: التعرف على الأنشطة البشرية، التعلم المستمر، حساسات القصور الذاتي، Transformer، UWR توقع النشاط.

Table des matières

Dédicace	I
Remerciements	II
Résumé	III
Abstract	IV
ملخص	V
Liste des sigles et acronymes	XVII
Introduction générale	1
0.1 Problématique détaillée	1
0.1.1 Limites du paradigme batch	1
0.1.2 Oubli catastrophique en pratique	2
0.1.3 Anticipation vs classification instantanée	2
0.2 Approche proposée — vue d’ensemble	2
0.3 Organisation du document	2
I État de l’art	4
1 Reconnaissance d’activités humaines, capteurs, pipelines et limites	5
1.1 Définition et enjeux	5
1.1.1 Définition de la HAR	5
1.1.2 Applications	5
1.1.2.1 Santé et bien-être	6
1.1.2.2 Aide à domicile et sécurité	6
1.1.2.3 Sport et coaching	6
1.1.2.4 Industrie et ergonomie	6
1.1.2.5 Synthèse des exigences transverses	6
1.2 Capteurs, modalités et déploiement	7
1.2.1 Les capteurs inertiels (IMU)	7

1.2.2	Wearables versus smartphones	7
1.2.3	Fréquence d'échantillonnage et filtrage	7
1.2.4	Qualité des annotations	8
1.2.5	Embarqué, cloud, et contraintes réelles	8
1.2.6	Multi-modalité	8
1.3	Pipeline classique de la HAR	8
1.3.1	Acquisition	9
1.3.2	Segmentation	9
1.3.3	Features et classification	9
1.3.4	Chaîne ARC	9
1.4	Méthodes profondes modernes	10
1.4.1	Du feature engineering à la représentation apprise	10
1.4.2	Attention et Transformers	11
1.4.3	SSL, intégration de données, personnalisation	12
1.5	Méthodes traditionnelles (hors deep learning)	12
1.5.1	Machines à vecteurs de support (SVM)	12
1.5.2	Forêts aléatoires	13
1.5.3	Modèles de Markov cachés (HMM)	13
1.6	Protocoles d'évaluation en HAR	13
1.6.1	Split train / test	13
1.6.2	Métriques	13
1.7	Jeux de données publics fréquents en HAR	14
1.8	Limites et passage à l'apprentissage continu	14
1.8.1	Données et généralisation	14
1.8.2	Variabilité inter-sujets et personnalisation	15
1.8.3	Oubli catastrophique	15
1.8.4	Anticipation	15
1.9	Conclusion	16
2	Apprentissage continu et adaptation dynamique	17
2.0.1	Organisation du chapitre	17
2.1	Apprentissage continu : concepts	17
2.1.1	Online learning	17
2.1.2	Incremental learning	17
2.1.3	Lifelong learning	18
2.1.4	Continual learning	18
2.2	Problème clé : l'oubli catastrophique	18
2.3	Taxonomie des approches	19
2.3.1	Régularisation (<i>regularization-based</i>)	20

2.3.2	Replay et mémoires	20
2.3.2.1	Replay uniforme vs priorisé	20
2.3.2.2	Prototypes vs exemplaires bruts	20
2.3.3	Approches prototypiques	21
2.3.4	Pré-entraînement et fine-tuning	21
2.4	Adaptation au contexte HAR	21
2.4.1	Adaptation de domaine	21
2.4.2	Personnalisation	21
2.4.3	Few-shot learning	21
2.4.4	Calibration en ligne	22
2.5	Anticipation d’activités	22
2.5.1	Incertitude et Monte Carlo Dropout	22
2.6	Métriques d’évaluation	22
2.6.1	Accuracy	22
2.6.2	F1-score	23
2.6.3	Mesure d’oubli (<i>forgetting</i>)	23
2.6.4	Backward transfer (BWT) et forward transfer (FWT)	23
2.6.5	Exemple numérique (3 tâches)	23
2.6.6	Empreinte mémoire et latence embarquée	24
2.7	Taxonomie des scénarios CL appliqués au HAR	24
2.7.1	Replay vs régularisation en HAR	25
2.7.2	Lien avec l’anticipation	25
2.8	Conclusion	25
3	Travaux antérieurs sur la HAR Continue et l’Adaptation Dynamique	26
3.1	Méthodologie de sélection des articles	26
3.1.1	Bases documentaires	26
3.1.2	Requêtes et mots-clés	26
3.1.3	Critères d’inclusion et d’exclusion	26
3.2	Catégorisation des travaux	27
3.2.1	Continual learning et HAR (cœur du corpus)	27
3.2.1.1	Amrani et al. (2025) — intégration multi-bases	27
3.2.1.2	Schiemer et al. (2023) — OCL-HAR	28
3.2.1.3	Adaimi & Thomaz (2022) — LAPNet-HAR	28
3.2.2	Incrémental de classes et représentation	29
3.2.2.1	iKAN (Liu et al., 2024)	29
3.2.2.2	iCaRL (Rebuffi et al., 2017)	29
3.2.3	Self-supervised HAR	30
3.2.3.1	Apprentissage contrastif	30

3.2.3.2	Pré-entraînement à grande échelle	30
3.2.4	Personnalisation et adaptation de domaine	30
3.2.4.1	Batch norm adaptive	30
3.2.4.2	Personnalisation de classifieurs	30
3.2.5	Anticipation d'activité	31
3.3	Analyse détaillée — synthèse transversale	32
3.4	Tableau comparatif (synthèse obligatoire)	33
3.4.1	Résultats quantitatifs rapportés (extraits des articles)	35
3.5	Analyse critique	37
3.5.1	Positionnement de notre contribution	37
3.5.2	Limites de la revue	38
3.5.3	Scénarios types et couverture bibliographique	38
3.5.4	Feuille de route implicite du mémoire	39
3.6	Conclusion	39
II	Contribution et Évaluation	40
4	Conception de la solution	41
4.1	Introduction	41
4.2	Définition et formulation du problème	41
4.2.1	Cadre formel et scénarios retenus	41
4.2.1.1	Formulation générale	41
4.2.1.2	Scénario 1 — Apprentissage incrémental par utilisateurs	42
4.2.1.3	Scénario 2 — Apprentissage incrémental par classes	42
4.2.2	Contraintes de déploiement	42
4.2.3	Positionnement par rapport à l'état de l'art	43
4.3	Objectifs du projet et contributions principales	43
4.4	Données : homogénéisation et pipeline	44
4.4.1	Datasets retenus et leurs caractéristiques	44
4.4.2	Pipeline de prétraitement	45
4.4.3	Unification des classes	46
4.5	Architecture générale du système	46
4.5.1	Vue d'ensemble	46
4.5.2	Backbone Transformer	47
4.5.3	Module 1 — Reconnaissance continue	48
4.5.4	Module 2 — Anticipation d'activité	49
4.6	Conception du module de reconnaissance continue	49
4.6.1	Mémoire de prototypes	49
4.6.2	Tampon de replay standard	49

4.6.3	Uncertainty-Weighted Replay — contribution originale	50
4.6.4	Calibration adaptative en ligne	51
4.6.5	Fonction de perte totale	51
4.7	Conception du module d’anticipation	52
4.7.1	Architecture LSTM sur fenêtres partielles	52
4.7.2	Limites de la formulation multi-classe	52
4.8	Modules complémentaires alignés sur la fiche PFE	52
4.8.1	Pré-entraînement auto-supervisé (SSL)	53
4.8.2	Pipeline foundation-style	53
4.8.3	Calibration de confiance en ligne	53
4.8.4	Bandit de seuil (RL léger)	53
4.8.5	Anticipation du risque de chute / équilibre	53
4.8.6	Scénario «préparation de repas»	53
4.9	Scénarios d’utilisation et flux de données	53
4.9.1	Cas 1 — Personnalisation progressive (user-incrémental)	53
4.9.2	Cas 2 — Extension du répertoire d’activités (class-incrémental)	54
4.9.3	Cas 3 — Déploiement cross-appareil (CORAL + anticipation)	54
4.9.4	Contraintes temps réel rappelées	54
4.10	Protocole d’évaluation	55
4.10.1	Baselines comparées	55
4.10.2	Métriques retenues	55
4.11	Module auxiliaire — détection de chutes	56
4.11.1	Motivation	56
4.11.2	Approche hybride	56
4.11.3	Métriques cibles	56
4.12	Conclusion	56
4.12.1	Synthèse des choix de conception	57
5	Réalisation de la solution	58
5.1	Introduction	58
5.2	Environnement de développement	58
5.2.1	Plateformes utilisées	58
5.2.2	Langages et frameworks	58
5.2.3	Bibliothèques et outils	59
5.2.4	Structure du projet et suivi de version	59
5.3	Préparation des données	60
5.3.1	Chargement et homogénéisation des datasets	60
5.3.2	Implémentation du pipeline de prétraitement	60
5.3.3	Construction du flux incrémental simulé	61

5.4	Implémentation du backbone Transformer	62
5.4.1	Architecture détaillée	62
5.4.2	Pré-entraînement initial	63
5.4.3	Visualisation des représentations apprises	64
5.5	Implémentation du module de reconnaissance continue	64
5.5.1	Gestion des prototypes	64
5.5.2	Tampon de rejeu et Uncertainty-Weighted Replay	65
5.5.3	Intégration du Monte Carlo Dropout	66
5.5.4	Entraînement incrémental complet	66
5.5.5	Orchestration des scripts d'expérience	67
5.5.6	Validation unitaire et tests de non-régression	68
5.6	Implémentation du module d'anticipation	68
5.6.1	Architecture LSTM multi-classe	68
5.6.2	Dataset et entraînement	68
5.7	Contributions supplémentaires	69
5.7.1	Détection de chute — approche hybride	69
5.7.2	SSL, foundation-style, calibration et RL léger	69
5.7.3	Adaptateur de domaine CORAL	69
5.8	Conclusion	71
5.8.1	Retour d'expérience implémentation	71
6	Évaluation de la solution	72
6.1	Introduction	72
6.2	Résultats du pré-entraînement	72
6.2.1	Courbes d'apprentissage	72
6.2.2	Performances par classe	73
6.2.3	Visualisation des embeddings (t-SNE)	74
6.3	Comparaison avec l'état de l'art (classification standard)	74
6.4	Évaluation user-incrémental	75
6.4.1	Protocole	75
6.4.2	Résultats quantitatifs	77
6.4.3	Analyse qualitative du protocole 44 tâches	78
6.5	Évaluation class-incrémentale	78
6.5.1	Analyse du déséquilibre inter-tâches	80
6.6	Ablation study	81
6.7	Évaluation cross-dataset (CORAL)	81
6.8	Évaluation du module d'anticipation	82
6.9	Contributions supplémentaires	83
6.9.1	Fall-risk, SSL et calibration	83

6.10	Guide de reproductibilité	83
6.11	Discussion générale	84
6.11.1	Synthèse	84
6.11.2	Limites	84
6.11.3	Perspectives d'amélioration	84
6.12	Conclusion	85
	Conclusion générale et Perspectives	86
6.13	Rappel du contexte	86
6.14	Ce que nous avons apporté	86
6.15	Bilan chiffré	87
6.16	Limites	87
6.17	Perspectives	87
6.17.1	Publication et ouverture	87
6.18	Retour sur les objectifs du mémoire	88
6.19	Apports méthodologiques	88
6.20	Message pour le déploiement	88
A	Jeux de données utilisés	91
B	Protocoles expérimentaux et hyperparamètres	96
C	Résultats expérimentaux détaillés	100
D	Extraits de code et algorithmes	110
E	Fiches détaillées des articles de référence	115
F	Tableaux de résultats par classe et par tâche	119
G	Éléments mathématiques complémentaires	123
H	Synthèse comparative des expériences	126
I	Jalons du projet et évolution du dépôt	129

Table des figures

1	Capteur IMU triaxial	7
2	Pipeline classique HAR — chaîne ARC (Bulling et al., 2014)	8
3	Fenêtrage glissant sur signaux IMU	9
4	Architecture Transformer IMU (Dirgová Luptáková et al., 2022)	11
5	Panorama des approches deep learning HAR (Gu et al., 2021)	12
6	Placements du capteur (Bulling et al., 2014)	14
7	Homogénéisation multi-datasets (Amrani et al., 2025)	15
1	Oubli catastrophique et EWC (Kirkpatrick et al., 2017)	19
2	Taxonomie de l'apprentissage continu (De Lange et al., 2021)	19
3	Taxonomie continual learning (De Lange et al., 2021)	24
4	Matrice d'évaluation CL	25
1	Pipeline Amrani et al. (Amrani et al., 2025)	27
2	F1 incrémental Amrani et al. (Amrani et al., 2025)	28
3	Architecture CNN-LSTM de référence	29
4	Matrice d'évaluation CL (De Lange et al., 2021)	31
5	Transformer pour HAR (Dirgová Luptáková et al., 2022)	32
6	Positionnement des travaux recensés	36
7	Comparaison quantitative des approches	37
1	Architecture globale du système proposé	46
2	Architecture du backbone Transformer	48
1	Courbes de pré-entraînement	63
2	Projection t-SNE des embeddings	64
1	Courbes d'apprentissage du backbone	72
2	Projection t-SNE des embeddings CLS	74
3	Matrice user-incrémental	75
4	Courbes d'oubli user-incrémental	76
5	Comparaison avec baseline sans rejeu	76
6	Comparaison SOTA user-incrémental	77
7	Métriques récapitulatives user-incrémental	77

8	Matrice class-incrémental	79
9	Impact taille du tampon	79
10	Oubli class-incrémental par classe	80
11	Contexte cross-domain	81
12	Résultats anticipation multi-classe	82
1	Courbes pré-entraînement fusion	100
2	Matrice détaillée user-incrémental	101
3	Courbes d'oubli détaillées	102
4	Comparaison baseline détaillée	103
5	Métriques agrégées user-incrémental	104
6	Matrice class-incrémental détaillée	104
7	Analyse class-incrémental	105
8	Oubli par classe	105
9	Poids d'attention Transformer	106
10	t-SNE détaillé	106
11	Anticipation détaillée	107
12	Comparaison méthodes CL	107
13	Référence littérature CL-HAR	108

Liste des tableaux

I	Jeux HAR publics cités dans le corpus (chapitre 3)	14
I	Synthèse comparative des travaux recensés (Chapitre 3)	34
II	Indicateurs quantitatifs extraits des articles de référence	35
III	Positionnement qualitatif de notre contribution	38
I	Comparaison des caractéristiques HAPT et WISDM	45
II	Baselines — apprentissage continu	55
III	Baseline anticipation — formulation implémentée dans <code>har_project</code>	55
IV	Baselines — adaptation cross-dataset HAPT → WISDM	55
I	Principales dépendances Python (<code>requirements.txt</code>)	59
II	Scripts d’expérience et artefacts produits	67
III	Résultats du module d’anticipation	69
IV	Résultats adaptation HAPT → WISDM	70
I	Impact de l’augmentation et de la profondeur du backbone	73
II	Performances par classe — validation HAPT	73
III	Comparaison classification standard — HAPT	74
IV	Comparaison SOTA — protocole 10 sujets HAPT (doc) / 44 sujets fusionnés (projet)	77
V	Résultats class-incrémental — effet de la capacité mémoire	79
VI	Ablation — contribution cumulative (user-incrémental, 10 sujets HAPT)	81
VII	Adaptation de domaine HAPT → WISDM	82
VIII	Anticipation — résultats	82
IX	Synthèse des modules auxiliaires	83
X	Bilan quantitatif global	84
XI	Bilan quantitatif des contributions	87
I	Distribution des classes HAPT après fenêtrage (150 échantillons, 50 % chevauchement)	92
II	Protocole des 12 classes HAPT	93
III	Distribution WISDM après homogénéisation (36 sujets retenus)	94
IV	PAMAP2 homogénéisé vers le schéma 12 classes	94

V	Types de chutes annotés dans MobiAct	95
VI	Comparaison des jeux de données	95
I	Stack logiciel	96
II	Backbone Transformer IMU	97
III	Pré-entraînement HAPT / fusion	97
IV	Hyperparamètres continual learning	98
V	Organisation du code	99
VI	Hyperparamètres anticipation, chutes et CORAL	99
I	Performance finale pré-entraînement fusion	100
II	User-incrémental 44 tâches — synthèse	103
III	Bilan quantitatif complet (annexe)	109
I	Validation HAPT — 4 blocs, augmentation, 30 époques	119
II	Décomposition user-incrémental par domaine capteur	120
III	Baselines CL — 10 sujets HAPT, même backbone	120
IV	Séquence class-incrémental — déséquilibre cumulatif	121
V	Anticipation LSTM multi-classe — observation partielle	121
VI	F1 par classe cross-dataset après CORAL supervisé	122
VII	Ablation par retrait — protocole 10 sujets	122
I	Leçons transverses des expériences	127
II	Guide de lecture des figures du mémoire	128
I	Traçabilité document source → version finale	130

Liste des sigles et acronymes

Sigle	Signification en anglais	Signification en français
HAR	Human Activity Recognition	Reconnaissance d'activités humaines
IMU	Inertial Measurement Unit	Unité de mesure inertielle
CL	Continual Learning	Apprentissage continu
AI	Artificial Intelligence (AI)	Intelligence artificielle (IA)
ML	Machine Learning	Apprentissage automatique
DL	Deep Learning	Apprentissage profond
CNN	Convolutional Neural Network	Réseau de neurones convolutionnel
LSTM	Long Short-Term Memory	Mémoire à long et court terme
GRU	Gated Recurrent Unit	Unité récurrente à portes
MLP	Multilayer Perceptron	Perceptron multicouche
SVM	Support Vector Machine	Machine à vecteurs de support
RF	Random Forest	Forêt aléatoire
HMM	Hidden Markov Model	Modèle de Markov caché
EWC	Elastic Weight Consolidation	Consolidation élastique des poids
LwF	Learning without Forgetting	Apprentissage sans oubli
ER	Experience Replay	Rejeu d'expériences

Sigle	Signification en anglais	Signification en français
iCaRL	Incremental Classifier and Representation Learning	Classifieur et représentation incrémentaux
UWR	Uncertainty-Weighted Replay	Rejeu pondéré par l'incertitude
MC	Monte Carlo	Monte Carlo
SSL	Self-Supervised Learning	Apprentissage auto-supervisé
CORAL	CORrelation ALignment	Alignement de covariance
DA-BN	Domain Adaptive Batch Normalization	Normalisation adaptative par lots
BWT	Backward Transfer	Transfert rétroactif
FWT	Forward Transfer	Transfert prospectif
AUC	Area Under the ROC Curve	Aire sous la courbe ROC
ROC	Receiver Operating Characteristic	Courbe caractéristique de fonctionnement du récepteur
KAN	Kolmogorov-Arnold Network	Réseau de Kolmogorov-Arnold
OCL	Online Continual Learning	Apprentissage continu en ligne
CLS	Classification Token	Token de classification
FFN	Feed-Forward Network	Réseau à propagation directe
IIR	Infinite Impulse Response	Réponse impulsionnelle infinie
ADL	Activities of Daily Living	Activités de la vie quotidienne
GAN	Generative Adversarial Network	Réseau antagoniste génératif

Introduction générale

Reconnaître une activité à partir d'un accéléromètre ou d'un gyroscope dans un téléphone ou une montre, c'est devenu courant — pour le sport, la santé, l'aide à domicile. Le modèle classique reste pourtant le même : on collecte des données en laboratoire, on entraîne un réseau une fois, on déploie. Dès qu'un nouvel utilisateur arrive, que le capteur change de place ou qu'une activité inédite apparaît, les performances chutent. Réentraîner tout depuis zéro coûte cher ; finetuner naïvement fait oublier ce qui marchait avant. Et dans la majorité des systèmes, on ne fait que classifier l'instant présent, sans prévenir la transition qui arrive.

Ce mémoire part de ce constat. Nous proposons un système de HAR qui (1) apprend de façon continue quand de nouveaux utilisateurs ou classes arrivent, et (2) détecte tôt les changements d'activité. L'implémentation s'appuie sur le projet `har_project` : backbone Transformer, tampon de rejeu guidé par l'incertitude (UWR), anticipation multi-classe de l'activité suivante via une tête LSTM sur fenêtres partielles, plus des modules CORAL, SSL, foundation-style, calibration, bandit de seuil et fall-risk.

Partie I (chapitres 1 à 3) pose le contexte : fondements HAR, concepts d'apprentissage continu, revue des travaux proches et lacunes identifiées.

Partie II (chapitres 4 à 6) décrit la conception, l'implémentation et l'évaluation de notre solution, avec comparaisons aux baselines EWC, iCaRL et finetuning séquentiel.

Les chapitres suivants détaillent chaque brique ; la conclusion générale en synthétise les résultats et les perspectives.

0.1 Problématique détaillée

0.1.1 Limites du paradigme batch

Le paradigme dominant en HAR — collecter un corpus, entraîner offline, figer les poids — suppose une distribution stationnaire. Or en déploiement, trois sources de non-stationnarité apparaissent presque toujours : (1) *variabilité inter-sujets* (morphologie, habitudes, placement du capteur) ; (2) *évolution du contexte* (nouvelles activités, changement d'appareil) ; (3) *dérive temporelle* (fatigue, pathologie, saison). Réentraîner périodiquement sur tout l'historique est coûteux en calcul, en stockage et en privacy : les données brutes ne peuvent pas rester indéfiniment sur le serveur.

0.1.2 Oubli catastrophique en pratique

Quand on fine-tune séquentiellement sujet par sujet — exactement ce que simule notre protocole 44 tâches — la accuracy chute de 91 % (joint training) à 62 % sans mécanisme anti-oubli (chapitre 6). EWC et iCaRL améliorent la situation mais ne priorisent pas les exemples «à risque» près des frontières de décision. UWR comble ce gap en guidant le tampon de 2 000 exemples vers les fenêtres incertaines.

0.1.3 Anticipation vs classification instantanée

La plupart des publications HAR optimisent l’accuracy sur la fenêtre courante. Or les cas d’usage sécurité (chute, sortie de zone sûre) demandent d’agir *avant* le changement complet d’activité. La littérature vidéo (Ryoo, 2011 ; Gammulle et al., 2019) traite l’early recognition ; peu d’articles IMU combinent anticipation et CL. Dans har_project, nous implémentons une anticipation multi-classe (prédire la classe suivante depuis $S = 5$ fenêtres tronquées à $p \in \{25, 50, 75\}\%$), filtrée sur les transitions (transitions_only). Le macro-F1 de validation atteint environ 0,60–0,64 selon p (baseline majorité $\approx 0,04$).

0.2 Approche proposée — vue d’ensemble

Le système HARContinualModel s’organise en trois couches :

1. **Backbone partagé** : IMUTransformerEncoder pré-entraîné sur HAPT ou fusion multi-jeux ; embeddings 128-D via token CLS.
2. **Module CL (UWR)** : tampon pondéré par entropie MC, prototypes par classe, perte contrastive ; scénarios user- et class-incremental.
3. **Modules auxiliaires** : tête anticipation LSTM multi-classe, adaptateur CORAL, fall-risk, calibration online, bandit de seuil, SSL / foundation-style.

Cette modularité permet d’évaluer chaque brique indépendamment (ablations chapitre 6) tout en partageant les représentations apprises.

0.3 Organisation du document

Le **chapitre 1** pose le vocabulaire HAR, capteurs, pipelines classiques et deep learning. Le **chapitre 2** formalise l’apprentissage continu (oubli, EWC, replay, métriques). Le **chapitre 3** compare les travaux récents HAR+CL et identifie les lacunes (anticipation, embarqué, protocoles).

Le **chapitre 4** décrit la conception : formalisation mathématique, UWR, anticipation LSTM, SSL/foundation-style, calibration, RL léger et fall-risk. Le **chapitre 5** détaille l'implémentation Python (har_project/). Le **chapitre 6** présente l'évaluation complète avec figures, tableaux et analyse des limites.

Les **annexes** documentent les datasets, hyperparamètres, résultats étendus, extraits de code et fiches bibliographiques pour la reproductibilité. L'annexe A détaille HAPT/WISDM/PAMAP2 ; les annexes B à H couvrent protocoles, figures commentées, code UWR, fiches articles, tableaux par classe, mathématiques, synthèse expérimentale et jalons du projet.

Première partie

État de l'art

Chapitre 1

Reconnaissance d'activités humaines, capteurs, pipelines et limites

La reconnaissance d'activités humaines (HAR) à partir de capteurs portés accompagne aujourd'hui de nombreuses applications : santé mobile, aide à domicile, sport, sécurité. Ce chapitre pose le vocabulaire de base pour la suite du mémoire : définition de la HAR, capteurs utilisés, pipeline de traitement classique, passage aux méthodes profondes, puis les limites qui nous poussent vers l'apprentissage continu (chapitres 2 et 3) et notre contribution (partie II).

1.1 Définition et enjeux

1.1.1 Définition de la HAR

En HAR « capteur », on cherche à inférer une activité physique (marche, assis, escaliers, etc.) à partir de signaux mesurés sur le corps ou dans la poche (**Bulling et al., 2014**). Les surveys récents rappellent que cette information sert souvent de couche intermédiaire : une application en aval déclenche une alerte, adapte une interface ou estime une charge d'effort (**Gu et al., 2021**).

On distingue classiquement la HAR vidéo de la HAR capteur. La seconde est privilégiée quand la confidentialité et le déploiement embarqué comptent (**Wang et al., 2019**). Un voisin proche est l'inférence précoce : reconnaître une activité avant la fin de la séquence (**Ryoo, 2011**), ou anticiper la transition suivante en vision (**Gammulle et al., 2019**). Ces travaux ne remplacent pas la HAR IMU, mais expliquent pourquoi notre mémoire traite aussi l'anticipation (chapitre 4).

1.1.2 Applications

Les activités couvrent la locomotion, les gestes du quotidien, la rééducation, parfois la détection de chutes (**Gu et al., 2021 ; Wang et al., 2019**). Selon l'usage, les contraintes changent : latence, robustesse au bruit, respect de la vie privée. Côté données, étiqueter des protocoles longs reste coûteux ; d'où l'intérêt du pré-entraînement auto-supervisé ou contrastif (**Yuan et al., 2024 ; Tang et al., 2021**). Notre PFE vise explicitement l'adaptation continue et l'anticipation, au-delà d'un classifieur statique entraîné une fois for all.

1.1.2.1 Santé et bien-être

En santé mobile, la HAR sert à estimer le niveau d'activité physique (comptage de pas, détection sédentarité), à suivre la rééducation post-opératoire ou à détecter des chutes chez les personnes âgées (**Gu et al., 2021**). Les applications cliniques exigent une faible latence et une robustesse aux variations de port du capteur : un modèle entraîné sur des volontaires jeunes en laboratoire peut dégrader fortement sur un patient âgé à domicile. D'où l'intérêt de l'apprentissage continu utilisateur par utilisateur, thème central de notre mémoire.

1.1.2.2 Aide à domicile et sécurité

Les systèmes d'aide à la vie autonome combinent souvent HAR et détection d'événements critiques (chute, immobilité prolongée). La HAR fournit le contexte (assis, debout, marche); un module spécialisé déclenche l'alerte. Notre architecture prévoit explicitement un détecteur de chutes hybride (règles + classifieur) alimenté par le même backbone IMU, même si l'évaluation actuelle repose sur un proxy HAPT faute de MobiAct.

1.1.2.3 Sport et coaching

En sport, la HAR distingue types d'exercices, compte répétitions ou estime l'intensité (**Wang et al., 2019**). Les contraintes diffèrent de la santé : signaux plus intenses, mouvements non standardisés, besoin de personnalisation rapide (nouvel athlète, nouvel exercice). Le class-incremental (nouvelles classes d'activité) modélise partiellement ce cas — scénario évalué au chapitre 6.

1.1.2.4 Industrie et ergonomie

En environnement professionnel, la HAR peut repérer des postures à risque (torsion, port de charge) ou valider le respect de gestes de sécurité (**Gu et al., 2021**). Les capteurs portés remplacent parfois des caméras en zone restreinte. L'adaptation à de nouveaux opérateurs ou postes de travail reprend les mêmes enjeux d'oubli et de domain shift que la santé connectée.

1.1.2.5 Synthèse des exigences transverses

Quel que soit le domaine, trois exigences reviennent : (i) généraliser à de nouveaux utilisateurs sans réentraînement complet; (ii) intégrer de nouvelles activités ou capteurs au fil du temps; (iii) anticiper les changements d'état pour agir avant qu'un incident ne se produise. Les chapitres 2 à 6 structurent notre réponse à ces trois points.

1.2 Capteurs, modalités et déploiement

1.2.1 Les capteurs inertiels (IMU)

Une IMU combine au minimum un accéléromètre et un gyroscope; le magnétomètre est souvent présent sur smartphone (**Bulling et al., 2014**; **Gu et al., 2021**). L'accéléromètre mesure l'accélération (souvent en g ou m/s^2); le gyroscope apporte la dynamique de rotation, utile pour discriminer des activités proches (**Ordóñez et al., 2016**). En pratique, deux datasets HAPT et WISDM n'utilisent pas les mêmes fréquences, unités ou conventions de gravité : toute chaîne multi-sources doit homogénéiser ces détails (**Amrani et al., 2025**).

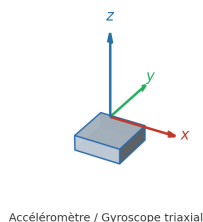


Fig. 1 Capteur IMU triaxial — axes accéléromètre et gyroscope.

1.2.2 Wearables versus smartphones

Smartphones et montres restent les plateformes les plus répandues pour collecter des IMU «au contact» de l'utilisateur (**Bulling et al., 2014**; **Wang et al., 2019**). La HAR capteur est en général mieux acceptée que la vidéo dans des contextes sensibles (**Wang et al., 2019**). Sur téléphone, l'accéléromètre seul suffit parfois pour des activités grossières; le gyroscope améliore les cas ambigus (**Ferrari et al., 2020**).

Les montres connectées offrent un placement poignet stable mais des amplitudes de signal différentes de la ceinture (HAPT) ou de la poche (WISDM). Un modèle entraîné sur ceinture peut perdre 20–30 points de F1 sur poignet sans adaptation — phénomène documenté dans les comparaisons multi-datasets (**Amrani et al., 2025**). C'est une motivation directe de nos expériences CORAL et du protocole 44 sujets mélangeant placements.

1.2.3 Fréquence d'échantillonnage et filtrage

La fréquence typique varie de 20 Hz (WISDM) à 100 Hz (PAMAP2, MobiAct). Notre pipeline unifie à 50 Hz par rééchantillonnage Fourier (**Amrani et al., 2025**). La composante gravitationnelle ($\approx 9,81 \text{ m/s}^2$) biaise l'accéléromètre selon l'orientation; un filtre passe-bas 0,5 Hz estime et soustrait cette composante avant fenêtrage (chapitre 5). Sans cette étape, «debout» et

«assis» peuvent se confondre sur la seule magnitude accélérométrique.

1.2.4 Qualité des annotations

En laboratoire (HAPT), les activités sont scriptées et étiquetées échantillon-par-échantillon. En conditions libres (WISDM), les labels sont déclaratifs et bruités : marche vs jogging flou, transitions non marquées. Cette asymétrie explique pourquoi nous réservons WISDM surtout au test cross-domain et pas au pré-train principal initial, avant la fusion étendue du projet `har_project`.

1.2.5 Embarqué, cloud, et contraintes réelles

Bulling et al. (Bulling et al., 2014) opposent traitement hors ligne (acquisition puis analyse) et traitement en ligne (décision pendant l'enregistrement). En déploiement, on sépare souvent un entraînement lourd (serveur ou poste de travail) d'une inférence ou d'un fine-tuning léger sur appareil (Amrani et al., 2025 ; Mazankiewicz et al., 2020). Les gros pré-entraînements SSL (Yuan et al., 2024) renforcent cette logique hybride : ressources importantes en amont, adaptation ciblée ensuite.

1.2.6 Multi-modalité

Fusionner accéléromètre et gyroscope est courant lorsqu'une seule modalité prête à confusion (Ordóñez et al., 2016). Les surveys élargissent parfois le spectre (WiFi, RFID, etc.) (Gu et al., 2021). Côté modèle, les Transformers sur séries (Dirgová Luptáková et al., 2022) offrent une alternative aux chaînes CNN/LSTM — piste retenue dans notre backbone.

1.3 Pipeline classique de la HAR

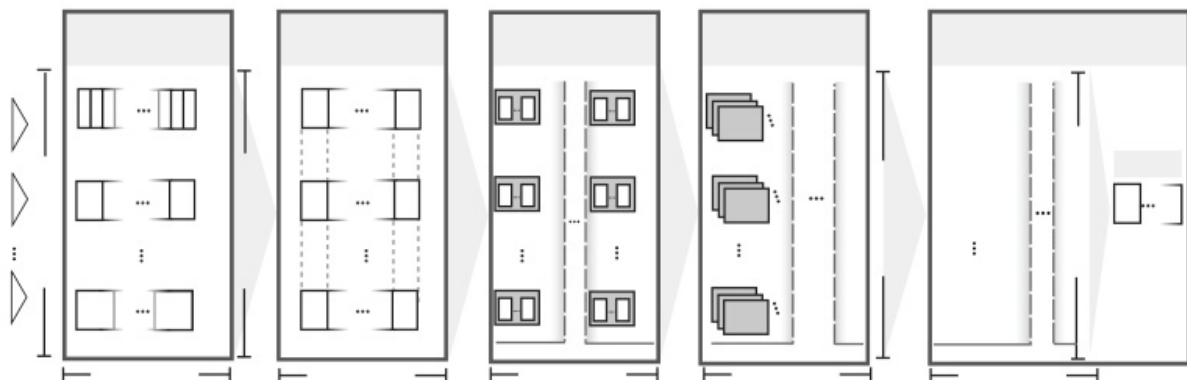


Fig. 2 Pipeline classique de la HAR — chaîne ARC (Bulling et al., 2014 ; Wang et al., 2019 ; Gu et al., 2021).

1.3.1 Acquisition

On enregistre des séries brutes : fréquence d'échantillonnage, placement du capteur, protocole d'activités. En conditions réelles, la variabilité inter-sujets et la dérive du capteur sont la norme, pas l'exception (**Bulling et al., 2014**). Fusionner plusieurs bases publiques impose en plus d'aligner labels, unités et documentation (**Amrani et al., 2025**).

1.3.2 Segmentation

Les activités quasi périodiques (marche, course) se découpent en fenêtres glissantes, souvent avec recouvrement (**Bulling et al., 2014**). Les modèles profonds multimodaux utilisent le même principe sur signaux bruts (**Ordóñez et al., 2016**). Dans notre projet, une fenêtre de 3 s à 50 Hz (150 échantillons, stride 50 %) suit cette convention (**Amrani et al., 2025**).

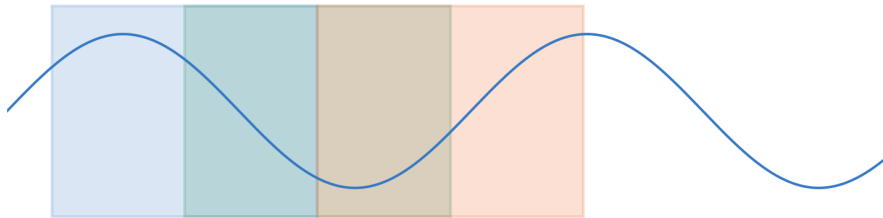


Fig. 3 Principe de fenêtrage glissant avec chevauchement (**Bulling et al., 2014**).

1.3.3 Features et classification

Historiquement, chaque fenêtre était décrite par des statistiques ou des descripteurs fréquentiels, puis classée (SVM, forêts aléatoires, k-NN, HMM) (**Bulling et al., 2014** ; **Gu et al., 2021** ; **Wang et al., 2019**). Ces pipelines restent utiles comme baseline légère ou quand les données sont rares. Leur limite commune en HAR : la qualité des descripteurs fixe un plafond, surtout quand l'utilisateur ou le capteur change.

1.3.4 Chaîne ARC

Le tutorat de Bulling formalise la chaîne *Acquisition – Segmentation – Features – Classification – Fusion – Évaluation* (**Bulling et al., 2014**). Les surveys deep learning (**Gu et al., 2021** ; **Wang et al., 2019**) la retrouvent avec des blocs neuronaux à la place de l'extraction manuelle.

1.4 Méthodes profondes modernes

1.4.1 Du feature engineering à la représentation apprise

Les réseaux profonds apprennent directement des représentations à partir des fenêtres, ce qui réduit la dépendance aux descripteurs artisanaux (**Gu et al., 2021 ; Wang et al., 2019**). Deep-ConvLSTM (**Ordóñez et al., 2016**) illustre le couple CNN+LSTM sur signaux portés multimodaux.

1.4.2 Attention et Transformers

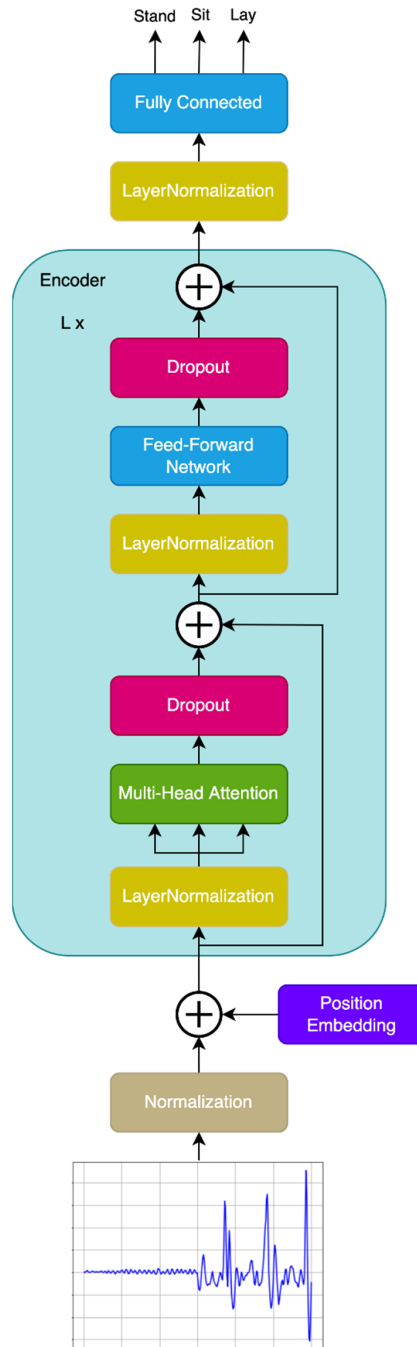


Fig. 4 Architecture IMUTransformerEncoder (Dirgová Luptáková et al., 2022; Vaswani et al., 2017).

Un Transformer calcule, pour chaque pas de temps, une combinaison pondérée de toute la fenêtre via l'attention (Vaswani et al., 2017; Dirgová Luptáková et al., 2022). Avec Q, K, V les projections requête, clé et valeur :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V.$$

En HAR, Q_t encode «ce que cherche» l’instant t , K_s mesure la compatibilité avec l’instant s , et V_s porte l’information agrégée. Les encodages positionnels réintroduisent l’ordre temporel. Le coût quadratique en longueur de séquence limite l’embarqué ; notre backbone reste volontairement compact (4 blocs, $d = 128$).

1.4.3 SSL, intégration de données, personnalisation

Yuan et al. (Yuan et al., 2024) montrent l’intérêt d’un très grand pré-entraînement non supervisé sur wearables, puis d’un fine-tuning aval. Amrani et al. (2025) insistent plutôt sur l’homogénéisation multi-bases (fréquence, unités, labels) avant fusion. Côté utilisateur, la personnalisation par sélection de données (Ferrari et al., 2020) ou par batch norm adaptive en ligne (Mazankiewicz et al., 2020) complète le tableau sans résoudre seul l’oubli séquentiel.



Fig. 5 Panorama des familles d’architectures profondes pour la HAR (Gu et al., 2021 ; Wang et al., 2019).

1.5 Méthodes traditionnelles (hors deep learning)

Avant les réseaux profonds, la HAR capteur suivait presque toujours le même schéma : fenêtre glissante, extraction de descripteurs, classifieur (Bulling et al., 2014). Ces pipelines restent utiles comme baseline légère ou sur microcontrôleur quand la mémoire est très contrainte (Wang et al., 2019).

1.5.1 Machines à vecteurs de support (SVM)

Une SVM cherche une frontière (éventuellement via noyau RBF) qui sépare les vecteurs de features par classe. En multi-classe, on combine des classifieurs binaires one-vs-one ou one-vs-rest. La performance dépend entièrement des descripteurs : moyenne, écart-type, énergie, FFT, etc. Dès qu’un nouvel utilisateur ou un capteur décalé modifie la distribution des features, la marge s’effondre sans réentraînement complet.

1.5.2 Forêts aléatoires

Une forêt aléatoire vote entre des arbres entraînés sur des sous-échantillons bootstrap. Elle tolère mieux le bruit et les variables hétérogènes qu’une SVM mal calibrée, mais ne voit la dynamique fine à l’intérieur d’une fenêtre que si les features la rendent explicite. En pratique, elle sert souvent de référence rapide avant de passer au deep learning (**Ferrari et al., 2020**).

1.5.3 Modèles de Markov cachés (HMM)

Un HMM modélise une séquence comme une chaîne d’états latents (activités ou sous-phases) avec des lois d’émission gaussiennes. Viterbi et Forward-Backward permettent l’inférence. L’hypothèse markovienne limite la modélisation des durées d’activité; les HMM restent pédagogiques pour comprendre la dimension temporelle, mais rarement compétitifs face aux LSTM ou Transformers sur IMU brut (**Gu et al., 2021**).

1.6 Protocoles d’évaluation en HAR

1.6.1 Split train / test

Deux protocoles dominant. Le *random split* mélange fenêtres de tous les sujets : scores optimistes, peu réalistes en déploiement (**Wang et al., 2019**). Le *leave-one-subject-out* (LOSO) retient un sujet entier pour le test : c’est le standard pour mesurer la généralisation inter-individus. Notre pré-entraînement HAPT utilise 80/20 par sujet (24 train, 6 val).

1.6.2 Métriques

L’accuracy seule trompe sur jeux déséquilibrés (transitions rares en HAPT). Le macro-F1 moyenne les F1 par classe; le micro-F1 agrège les comptages (**Schiemer et al., 2023**). En CL, on ajoute forgetting et backward transfer (chapitre 2). Pour l’anticipation, précision et rappel sur la classe «transition» comptent plus que l’accuracy globale.

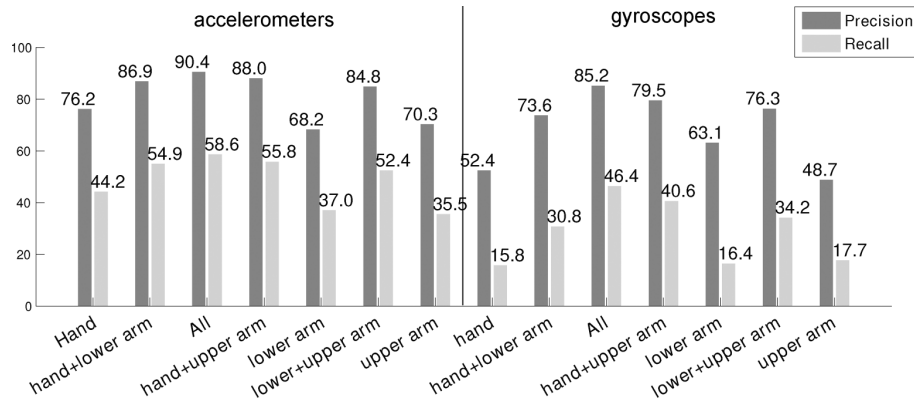


Fig. 6 Variations de placement du smartphone — ceinture, poignet, poche (**Bulling et al., 2014** ; **Ferrari et al., 2020**).

1.7 Jeux de données publics fréquents en HAR

Au-delà de HAPT et WISDM utilisés dans ce mémoire, la littérature mobilise régulièrement les jeux suivants (**Gu et al., 2021** ; **Wang et al., 2019** ; **Amrani et al., 2025**) :

Dataset	Sujets	Capteurs	Usage typique
UCI HAR (sans transitions)	30	Smartphone ceinture	Baseline smartphone
Opportunity	4	Corps + objets	HAR ambient, multimodal
PAMAP2	9	3 IMU corps	Activités + sport
DSADS	8	5 IMU corps	Daily + sport
MobiAct	61	Smartphone	Chutes réelles

Tab. I Jeux HAR publics cités dans le corpus (chapitre 3)

OCL-HAR (**Schiemer et al., 2023**) et LAPNet (**Adaimi et al., 2022**) évaluent sur PAMAP2 et Opportunity ; Amrani (**Amrani et al., 2025**) fusionne huit bases. Notre choix HAPT+WISDM+PAMAP2 couvre ceinture, poche et capteurs corps — triangle représentatif des domain shifts rencontrés en déploiement consumer vs clinique.

1.8 Limites et passage à l'apprentissage continu

1.8.1 Données et généralisation

Les jeux publics restent des protocoles contrôlés ; généraliser à un nouvel utilisateur ou un nouveau capteur est difficile (**Wang et al., 2019** ; **Gu et al., 2021**). L'hétérogénéité entre datasets n'est pas un détail technique : elle reflète des différences d'appareil et de protocole (**Amrani**

et al., 2025). Amrani et al. montrent que fusionner huit bases sans homogénéisation préalable creuse l'écart de F1 entre modèle global et modèles locaux ; d'où leur pipeline d'alignement (Amrani et al., 2025).

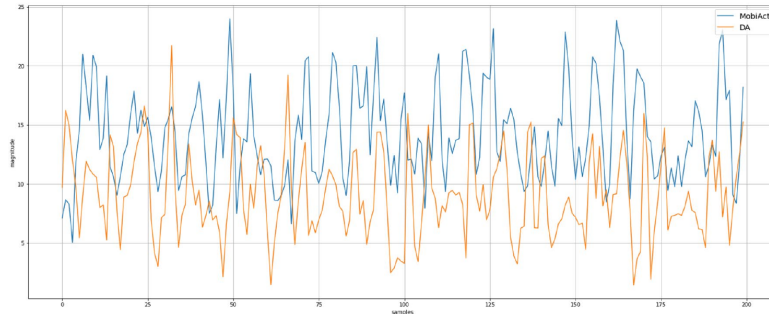


Fig. 2 Comparison between an excerpt of the jogging activity (magnitude) and an excerpt of the running activity (magnitude) as recorded in the MobiAct (jogging) and DA (running) datasets

Fig. 7 Ambiguïté et homogénéisation des labels entre jeux HAR (Amrani et al., 2025).

1.8.2 Variabilité inter-sujets et personnalisation

Deux utilisateurs exécutant « marche » produisent des signaux différents : morphologie, cadence, placement du téléphone (Ferrari et al., 2020). La personnalisation par fine-tuning ou batch norm adaptative aide, mais ne traite pas l'arrivée séquentielle de dizaines d'utilisateurs avec mémoire bornée (Mazankiewicz et al., 2020 ; Amrani et al., 2025).

1.8.3 Oubli catastrophique

Quand les tâches ou les utilisateurs arrivent en séquence, réentraîner naïvement efface souvent les acquis précédents (Kirkpatrick et al., 2017 ; Parisi et al., 2019). Le cadre de Ven et al. (2022) (task-, domain- et class-incremental) aide à préciser *ce qui change* dans le flux. EWC (Kirkpatrick et al., 2017) et iCaRL (Rebuffi et al., 2017) sont des réponses fondateurs, reprises ensuite en HAR (OCL-HAR (Schiemer et al., 2023), LAPNet (Adaimi et al., 2022), iKAN (Liu et al., 2024)).

1.8.4 Anticipation

La reconnaissance « instantanée » ne suffit pas si l'on veut alerter *avant* une chute ou une transition. Les références vidéo (Ryoo, 2011 ; Gammulle et al., 2019) posent le problème ; notre mémoire l'implémente en anticipation multi-classe (LSTM, fenêtres partielles). sur IMU (partie II).

1.9 Conclusion

Ce chapitre a posé la HAR capteur, ses pipelines et le passage aux modèles profonds. Les verrous récurrents — données limitées, décalage de domaine, oubli séquentiel, absence d’anticipation — motivent le chapitre 2 (concepts CL) et le chapitre 3 (revue ciblée). La partie II propose une réponse concrète avec `har_project`.

Chapitre 2

Apprentissage continu et adaptation dynamique

Dans ce chapitre, nous fixons le vocabulaire et les outils d'évaluation de l'apprentissage continu (CL), puis nous les relient au HAR sur capteurs portés. L'anticipation d'activité est introduite comme extension naturelle de la modélisation temporelle. Un fil rouge ressort de la littérature : le CL est bien documenté en vision, mais beaucoup moins standardisé pour la HAR en conditions réelles (nouveaux utilisateurs, nouvelles classes, flux continu). C'est précisément ce décalage qui motive la revue du chapitre 3 et notre contribution aux chapitres 4 à 6.

2.0.1 Organisation du chapitre

Nous commençons par les définitions (§2.1), l'oubli catastrophique et EWC (§2.2), puis la taxonomie des méthodes (§2.3) et des scénarios (§2.4). Les métriques CL (§2.5) sont essentielles pour lire le chapitre 6. La section 2.6 instancie cette taxonomie pour la HAR ; la conclusion relie au chapitre 3.

2.1 Apprentissage continu : concepts

2.1.1 Online learning

En apprentissage en ligne, le modèle reçoit des observations au fil du temps (mini-lots ou fenêtres) et met à jour ses paramètres sans repasser sur tout l'historique. Le défi principal est double : suivre une distribution qui dérive (*concept drift*) et garder un coût calcul acceptable. Sur smartphone ou montre, cela correspond à un utilisateur qui produit des signaux IMU en continu : on préfère alors des mises à jour légères plutôt qu'un réentraînement complet à chaque changement de contexte (Mazankiewicz et al., 2020 ; Amrani et al., 2025).

2.1.2 Incremental learning

L'apprentissage incrémental décrit les situations où l'information arrive par *blocs* : une nouvelle tâche, un nouvel utilisateur, une nouvelle classe ou un nouveau capteur. Ven et al. (2022) classent ces scénarios selon ce qui change dans le flux et selon que la frontière de tâche est connue ou non au test — ce qui conditionne à la fois le protocole d'évaluation et le choix de méthode (replay, régularisation, isolation de paramètres, etc.). En HAR, l'arrivée séquentielle

d'utilisateurs ou de conditions de port est modélisée ainsi dans les travaux d'online continual learning, par exemple (Schiemer et al., 2023).

2.1.3 Lifelong learning

Le *lifelong learning* insiste sur la durée : le système doit accumuler des compétences utiles pour la suite, sans effacer les acquis précédents. Parisi et al. (2019) résument le problème comme un équilibre plasticité / stabilité, avec plusieurs familles de réponses (régularisation, replay, architectures dynamiques).

2.1.4 Continual learning

Le terme *continual learning* est souvent utilisé comme fourre-tout couvrant l'incrémental et le lifelong, avec un focus explicite sur l'oubli catastrophique et la rétention des performances passées (De Lange et al., 2021 ; Parisi et al., 2019). Les surveys rappellent aussi que le CL en classification n'est pas une seule boîte : comparer deux articles sans préciser le scénario (task-incremental, class-incremental, domain-incremental) peut conduire à des conclusions trompeuses (De Lange et al., 2021 ; Ven et al., 2022).

2.2 Problème clé : l'oubli catastrophique

Quand on entraîne un réseau profond sur une nouvelle distribution, les poids s'ajustent fortement : la performance sur les anciennes données chute souvent brutalement. C'est l'oubli catastrophique (Kirkpatrick et al., 2017 ; De Lange et al., 2021) — une interférence entre objectifs successifs dans le même espace de paramètres, distincte d'une simple perte de généralisation.

Kirkpatrick et al. (2017) proposent l'Elastic Weight Consolidation (EWC) : pénaliser le déplacement des poids jugés importants pour les tâches passées, via une approximation diagonale de la matrice de Fisher. La figure 1 schématise l'idée : un finetuning naïf « efface » la tâche A ; une régularisation trop forte bloque la tâche B ; EWC cherche un compromis.

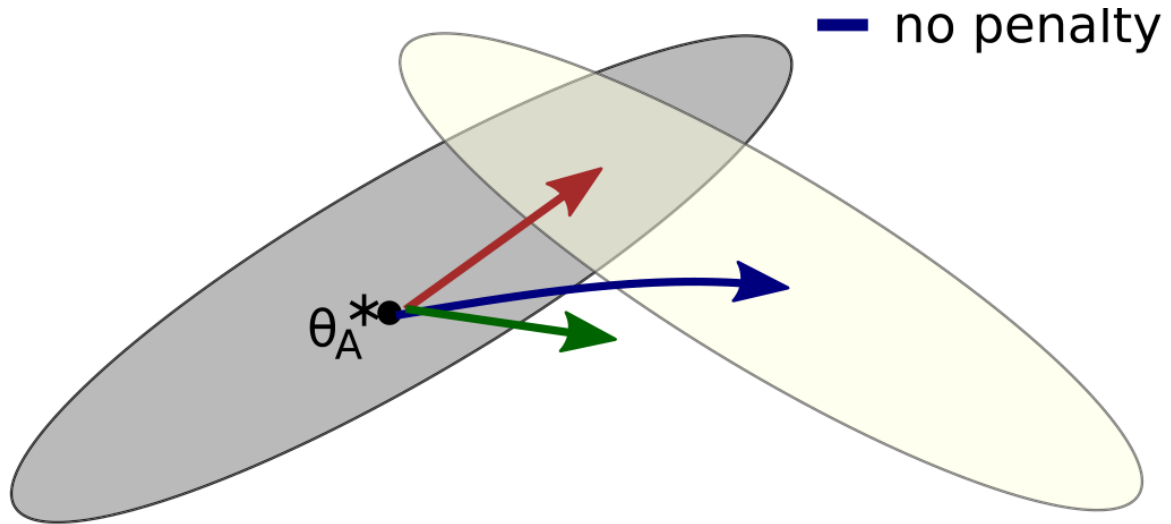


Fig. 1 Oubli catastrophique et régularisation EWC dans l'espace des paramètres (Kirkpatrick et al., 2017 ; De Lange et al., 2021).

Du côté class-incremental, iCaRL (Rebuffi et al., 2017) combine exemplaires mémorisés, distillation et classificateur incrémental. Cette ligne reste une référence fréquente, y compris dans les protocoles HAR continus (De Lange et al., 2021 ; Schiemer et al., 2023).

2.3 Taxonomie des approches

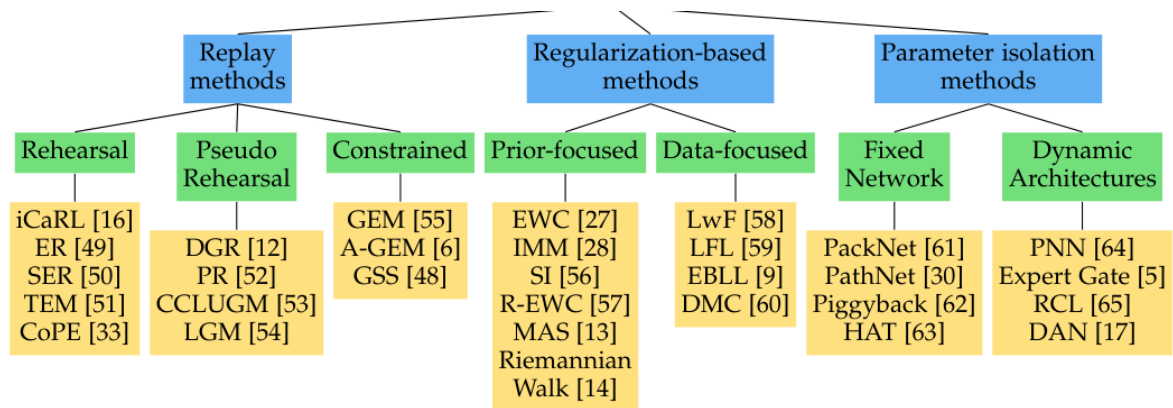


Fig. 2 Taxonomie de l'apprentissage continu (De Lange et al., 2021 ; Parisi et al., 2019 ; Ven et al., 2022).

La figure 2 et le découpage ci-dessous reprennent les grandes familles des surveys (De Lange et al., 2021 ; Parisi et al., 2019 ; Ven et al., 2022), en les reliant aux travaux HAR retenus dans notre corpus.

2.3.1 Régularisation (*regularization-based*)

L'idée est simple : empêcher le réseau de s'éloigner trop de la solution apprise avant. EWC (Kirkpatrick et al., 2017) ajoute une pénalité quadratique sur les poids, pondérée par l'importance Fisher. LwF (*Learning without Forgetting*) stabilise les sorties anciennes par distillation lors de l'apprentissage d'un nouvel objectif — famille discutée dans (De Lange et al., 2021) et reprise sous forme de distillation dans OCL-HAR (Schiemer et al., 2023).

2.3.2 Replay et mémoires

Deux variantes dominant. Le *rehearsal* garde un petit tampon d'exemples passés et les mélange aux nouvelles données : c'est le cœur d'iCaRL (Rebuffi et al., 2017) et de nombreuses variantes d'expérience replay. Le *generative replay* synthétise des pseudo-échantillons ; les surveys (De Lange et al., 2021) rappellent le compromis fidélité / coût.

En HAR, Schiemer et al. (2023) combinent distillation et rehearsal dans un cadre online avec décalage lié à l'arrivée d'utilisateurs. Adaimi et al. (2022) proposent LAPNet-HAR : prototypes, replay et contraste dans un cadre *task-free* plus proche d'un déploiement long terme.

2.3.2.1 Replay uniforme vs priorisé

Le replay uniforme ($P(x) = 1/|M|$) est la baseline la plus simple : chaque exemple mémorisé a la même chance d'être rejoué. *Prioritized Experience Replay* (Schaul et al., 2016) pondère par la TD-error en RL ; en CL supervisé, notre UWR transpose l'idée en pondérant par l'entropie de prédiction (Gal et al., 2016). Intuition : les exemples près des frontières de décision deviennent incertains avant d'être mal classifiés ; les rejouer en priorité retarde l'oubli. Le tampon de 2 000 fenêtres ($\approx 3,5$ % du corpus fusionné) force cette sélection : on ne peut pas tout garder, il faut optimiser l'information mémorisée.

2.3.2.2 Prototypes vs exemplaires bruts

iCaRL stocke des exemplaires bruts par classe ; LAPNet et notre système maintiennent des prototypes (moyennes d'embeddings). Avantage : empreinte réduite (un vecteur 128-D par classe vs fenêtres complètes 150×6). Inconvénient : perte d'information fine sur les transitions courtes — d'où le maintien du tampon UWR *en plus* des prototypes, pas à la place.

Plutôt que de tout partager, on isole : sous-réseaux par tâche, masques qui figent certaines connexions, compression de poids. De Lange et al. (2021) mentionnent GEM, A-GEM, PackNet ou HAT comme réponses structurelles à l'interférence. Pour des jeux HAR hétérogènes, iKAN (Liu et al., 2024) illustre une autre piste : branches par tâche et redistribution de caractéristiques

pour limiter à la fois l’oubli et l’incompatibilité entre datasets.

2.3.3 Approches prototypiques

Les *prototypical networks* représentent chaque classe par un centroïde dans l’espace d’embeddings ; la décision se fait par distance au prototype le plus proche. LAPNet-HAR (**Adaimi et al., 2022**) s’inscrit dans cette logique, avec mise à jour continue des prototypes et replay pour éviter qu’ils ne « dérivent » quand le backbone évolue.

2.3.4 Pré-entraînement et fine-tuning

Une part croissante du corpus HAR passe par un gros pré-entraînement (souvent SSL), puis une adaptation légère : Yuan et al. (**Yuan et al., 2024**) sur 700 000 person-days de wearables, contrastif santé (**Tang et al., 2021**), fine-tuning après homogénéisation multi-bases (**Amrani et al., 2025**), personnalisation par batch norm adaptive (**Mazankiewicz et al., 2020**). Les surveys HAR (**Wang et al., 2019** ; **Gu et al., 2021**) rappellent en parallèle que l’hétérogénéité des capteurs reste un frein même avec de bonnes représentations.

2.4 Adaptation au contexte HAR

2.4.1 Adaptation de domaine

Un même algorithme de marche ne se comporte pas pareil selon le capteur, sa position (ceinture, poignet, poche) ou l’utilisateur. Gu et al. (2021) et Wang et al. (2019) passent en revue les approches profondes et ces sources de décalage. Mazankiewicz et al. (2020) proposent une personnalisation incrémentielle via batch normalization *domain-adaptive*, pensée pour un suivi en temps réel sans réentraînement complet.

2.4.2 Personnalisation

La personnalisation vise à coller au geste individuel, au prix d’un risque de sur-apprentissage et d’un coût de mise à jour. Dans notre corpus, elle se croise souvent avec l’homogénéisation de données et le fine-tuning ciblé (**Amrani et al., 2025** ; **Mazankiewicz et al., 2020**).

2.4.3 Few-shot learning

Quand les labels par utilisateur ou par classe sont rares, le few-shot ou le métapprentissage devient pertinent. Les travaux retenus y arrivent surtout via des représentations génériques (SSL, contrastif) plutôt que via un meta-learner classique (**Yuan et al., 2024** ; **Tang et al., 2021**).

2.4.4 Calibration en ligne

On regroupe ici la mise à jour des statistiques de normalisation (**Mazankiewicz et al., 2020**), le recalibrage des scores de confiance (température, Platt scaling — détaillé au chapitre 5 si utilisé) et la stabilisation par distillation (**Schiemer et al., 2023**).

2.5 Anticipation d’activités

Au-delà de «quelle activité maintenant?», l’anticipation demande «que va-t-il se passer ensuite?» ou «peut-on reconnaître tôt une activité à partir d’une fenêtre incomplète?». Ryoo (**Ryoo, 2011**) pose des bases en vidéo; Gammulle et al. (**Gammulle et al., 2019**) montrent des architectures jointes en vision. Côté capteurs, Ordóñez et Roggen (**Ordóñez et al., 2016**) ancrent le séquentiel CNN+LSTM; les transformeurs sur séries (**Dirgová Luptáková et al., 2022**) ouvrent la même question avec d’autres blocs. Dans notre projet, nous implémentons l’anticipation en prédiction multi-classe de la classe suivante (tête LSTM, fenêtres partielles); les résultats du chapitre 6 en précisent les performances.

Le lien avec l’apprentissage par renforcement reste surtout conceptuel pour le HAR batch sur IMU; les modèles séquentiels supervisés restent la voie la plus fréquente dans la littérature capteur.

2.5.1 Incertitude et Monte Carlo Dropout

Pour prioriser les exemples difficiles en replay (UWR, chapitre 4), nous estimons l’incertitude épistémique par Monte Carlo Dropout (**Gal et al., 2016**): plusieurs passes forward avec dropout actif, puis entropie de la distribution moyenne des probabilités. Gal et Ghahramani distinguent incertitude épistémique (modèle) et aléatoire (données); seule la première nous intéresse pour guider la mémoire sous contrainte embarquée.

2.6 Métriques d’évaluation

2.6.1 Accuracy

Pour C classes et N exemples de test :

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\hat{y}_i = y_i\}.$$

En CL, on trace souvent l’accuracy par tâche au fil des sessions (**De Lange et al., 2021**).

2.6.2 F1-score

Pour une classe c , avec TP_c, FP_c, FN_c :

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_cR_c}{P_c + R_c}.$$

Le **macro-F1** moyenne les $F1_c$ sur les classes ; le **micro-F1** agrège d'abord les comptages globaux. Sur jeux déséquilibrés — fréquent en HAR — Schiemer et al. (2023) argumentent pour les deux ; Amrani et al. (2025) rapportent des F1 après fusion multi-datasets.

2.6.3 Mesure d'oubli (*forgetting*)

Soit $a_{i,j}$ l'accuracy sur la tâche j après apprentissage jusqu'à la tâche i ($i \geq j$). L'oubli de la tâche j après T tâches :

$$f_j = \max_{i \in \{j, \dots, T-1\}} a_{i,j} - a_{T,j}, \quad \bar{f} = \frac{1}{T-1} \sum_{j=1}^{T-1} f_j.$$

De Lange et al. (2021) s'appuient sur des métriques de ce type. En online HAR, Schiemer et al. (2023) proposent aussi un score d'oubli par classe à l'étape t :

$$F_t = \frac{1}{|C|} \sum_{c \in C} \left(\max_{l \in \{1, \dots, t-1\}} a_l^{(c)} - a_t^{(c)} \right).$$

2.6.4 Backward transfer (BWT) et forward transfer (FWT)

Ven et al. (2022) distinguent le transfert vers le passé (BWT) et vers l'avenir (FWT). Convention fréquente, avec $\bar{b}_j$ baseline isolée sur la tâche j :

$$\text{BWT} = \frac{1}{T-1} \sum_{j=1}^{T-1} (a_{T,j} - a_{j,j}), \quad \text{FWT} = \frac{1}{T-1} \sum_{j=2}^T (a_{j-1,j} - \bar{b}_j).$$

Dans nos expériences (chapitre 6), nous reportons BWT, forgetting et F1 macro final avec la définition de $\bar{b}_j$ explicitée dans le protocole.

2.6.5 Exemple numérique (3 tâches)

Supposons trois sujets A, B, C et accuracy sur A après chaque phase : $R[1, A] = 0,80$, $R[2, A] = 0,75$, $R[3, A] = 0,70$. L'oubli de A est $0,80 - 0,70 = 0,10$. Si B et C suivent le même schéma, le forgetting moyen $\bar{f}$ agrège ces décroissances. Un algorithme CL efficace maintient

$R[3, A]$ proche de $R[1, A]$; le finetuning naïf typiquement fait chuter $R[3, A]$ sous 0,50 — ce que nous observons sur 44 tâches sans jeu (F1 0,483).

2.6.6 Empreinte mémoire et latence embarquée

L’empreinte agrège paramètres entraînaables, taille du tampon de replay, prototypes (**Adaimi et al., 2022**) et structures auxiliaires (ex. termes Fisher pour EWC). Sur mobile, on distingue latence d’inférence (une fenêtre de 3 s) et latence d’adaptation (un pas de gradient). Amrani et al. (2025) montrent que le fine-tuning peut rester léger pour de nouveaux utilisateurs — critère retenu aussi dans notre cahier des charges (chapitre 4).

2.7 Taxonomie des scénarios CL appliqués au HAR

De Lange et al. (2021) et Ven et al. (2022) proposent des grilles de lecture communes. En HAR capteur, on peut les instancier ainsi :

- **Domain-incremental** : nouveau capteur ou placement (HAPT ceinture → WISDM poche → PAMAP2 corps). C’est le scénario le plus proche de nos tâches 37–44.
- **User-incremental** : même jeu, nouveaux sujets séquentiellement — protocole OCL-HAR (**Schiemer et al., 2023**) et notre séquence 44 tâches.
- **Class-incremental** : nouvelles activités annotées au fil du temps (6 tâches $\times$ 2 classes dans nos expériences).
- **Task-incremental** : frontière de tâche connue au test — rare en déploiement réel; LAPNet (**Adaimi et al., 2022**) cherche justement à s’en affranchir.

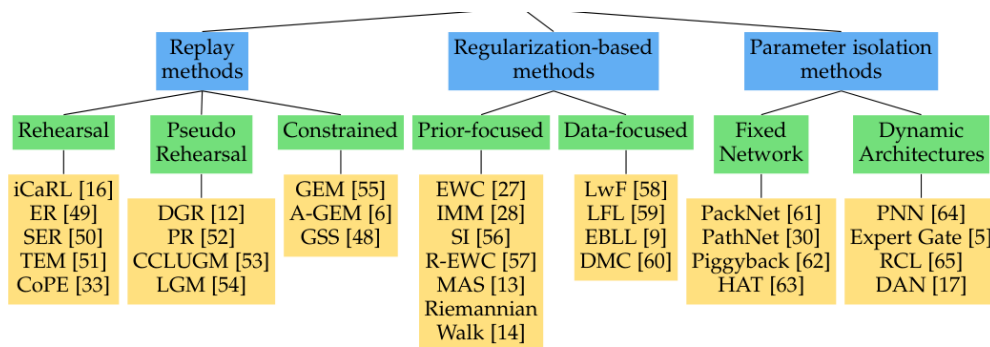


Fig. 3 Taxonomie des scénarios et méthodes CL (**De Lange et al., 2021**) — repère pour situer nos protocoles.

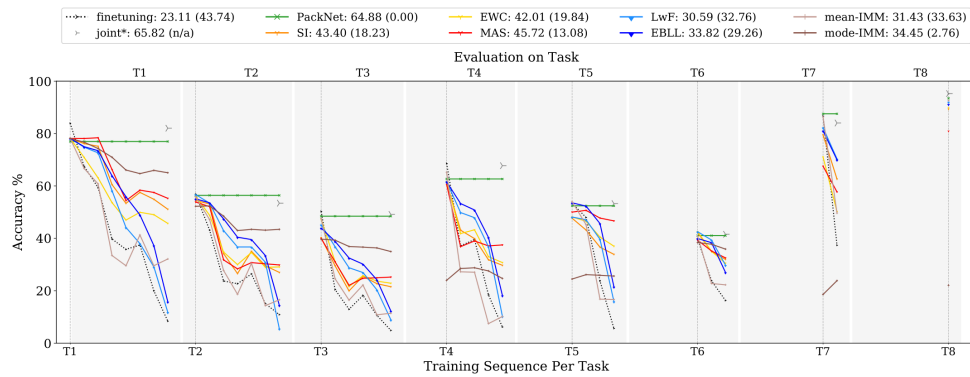


Fig. 4 Quelles métriques pour quel scénario (De Lange et al., 2021) : accuracy, forgetting, BWT, FWT.

2.7.1 Replay vs régularisation en HAR

Le replay stocke un sous-ensemble de fenêtres passées et les remélange aux nouvelles données — approche la plus robuste quand la mémoire le permet (Rebuffi et al., 2017; Schiemer et al., 2023). EWC (Kirkpatrick et al., 2017) évite le stockage explicite mais sous-estime les corrélations entre poids (Fisher diagonale); en HAR multi-sujets, nous l’avons gardé comme baseline intermédiaire (71,3 % accuracy, chapitre 6). Les prototypes (Adaimi et al., 2022) comprennent le replay en représentants par classe : moins coûteux en RAM, mais sensibles au dés-équilibre quand une classe domine (class-incremental).

2.7.2 Lien avec l’anticipation

Anticiper une transition, c’est prédire une *future* classe avant la fin de la fenêtre courante. Le CL, lui, met à jour les poids pour ne pas oublier le passé. Les deux problèmes partagent l’axe temporel mais optimisent des objectifs différents : BCE/CE sur labels futurs vs distillation/replay sur labels passés. Notre architecture (chapitre 4) les sépare en modules couplés seulement au niveau du backbone gelé ou partagé — choix motivé par la lacune bibliographique (chapitre 3).

2.8 Conclusion

Régularisation, replay, isolation et prototypes donnent un vocabulaire commun pour lire la littérature CL. Appliqué au HAR, ce vocabulaire se heurte encore à des protocoles hétérogènes : peu d’articles combinent flux continu, nouveaux utilisateurs, nouvelles classes *et* métriques de rétention dans un même benchmark (Schiemer et al., 2023; Adaimi et al., 2022; Liu et al., 2024; Amrani et al., 2025). Le chapitre 3 détaille les travaux retenus; les chapitres 4 à 6 montrent comment nous comblons une partie de ce vide avec UWR, anticipation LSTM multi-classe et CORAL sur notre pipeline har_project.

Chapitre 3

Travaux antérieurs sur la HAR Continue et l'Adaptation Dynamique

Ce chapitre complète le cadre théorique du chapitre 2 par une revue ciblée des articles HAR + apprentissage continu (CL), adaptation et anticipation. Nous expliquons d'abord comment le corpus a été construit (§3.1), puis nous regroupons les travaux par familles (§3.2), nous les comparons dans des tableaux (§3.3) et nous discutons ce qui manque encore (§3.4).

3.1 Méthodologie de sélection des articles

3.1.1 Bases documentaires

La recherche a surtout porté sur IEEE Xplore, ACM Digital Library (IMWUT, ISWC, Ubicomp) et Springer (Sensors, IJMLC, etc.). Quelques prépublications arXiv ont été gardées lorsqu'elles correspondent à la version citée dans un survey retenu.

3.1.2 Requêtes et mots-clés

Les requêtes combinaient HAR et continual learning, par exemple : *human activity recognition AND (continual learning OR incremental learning OR catastrophic forgetting)*; variantes avec *online, streaming, personalization, self-supervised, activity anticipation*. Les chaînes de citations à partir de (De Lange et al., 2021 ; Gu et al., 2021 ; Wang et al., 2019 ; Parisi et al., 2019) ont permis d'élargir le corpus au-delà du premier résultat de recherche.

3.1.3 Critères d'inclusion et d'exclusion

Nous avons retenu des articles revus par les pairs, explicitement liés à la HAR capteur ou indispensables pour cadrer le CL (surveys, iCaRL, EWC). Sont exclus les travaux purement vidéo sans transfert méthodologique clair (sauf pour l'anticipation), les textes inaccessibles et les doublons arXiv/journal. Le corpus final compte une vingtaine de références directement utiles au raisonnement du mémoire.

3.2 Catégorisation des travaux

Le CL appliqué à la HAR progresse, mais reste «under-explored» comparé à la vision : plusieurs articles récents le disent explicitement. Nous organisons le corpus en cinq blocs.

3.2.1 Continual learning et HAR (cœur du corpus)

Amrani et al. (2025) homogénéisent huit bases HAR, comparent S-CNN/LSTM et relient la démarche à une plateforme CLP. Le message principal : réduire l'écart de F1 entre modèle fusionné et modèle entraîné jeu par jeu, puis fine-tuner pour de nouveaux utilisateurs.

Schiemer et al. (2023) (OCL-HAR) formalisent un apprentissage en ligne avec arrivée d'utilisateurs et de classes, distillation + rehearsal, métriques micro/macro F1 et score d'oubli adapté au déséquilibre. Sur PAMAP2, DSADS, HAPT et WISDM, OCL-HAR reste au-dessus des baselines tandis qu'un finetuning naïf s'effondre rapidement — illustration concrète de l'oubli en HAR.

Adaimi et al. (2022) (LAPNet-HAR) visent un cadre *task-free* : prototypes, replay, perte contrastive. Les auteurs critiquent les hypothèses « frontière de tâche connue » empruntées à la vision.

Ces trois articles forment le noyau CL+capteurs : intégration de données, protocole OCL explicite, prototypes sans oracle de tâche.

3.2.1.1 Amrani et al. (2025) — intégration multi-bases

Le travail d'Amrani et al. (Amrani et al., 2025) part d'un constat simple : entraîner un modèle unique sur huit jeux HAR sans harmonisation produit un écart de F1 de 24,3 % par rapport à des modèles entraînés jeu par jeu. Leur pipeline aligne d'abord les labels ambigus (marche vs jogging, assis vs debout), puis entraîne des S-CNN/LSTM avec fine-tuning utilisateur. La plateforme CLP relie l'expérience à un cadre continual explicite, même si le rejou n'est pas le cœur de l'article.

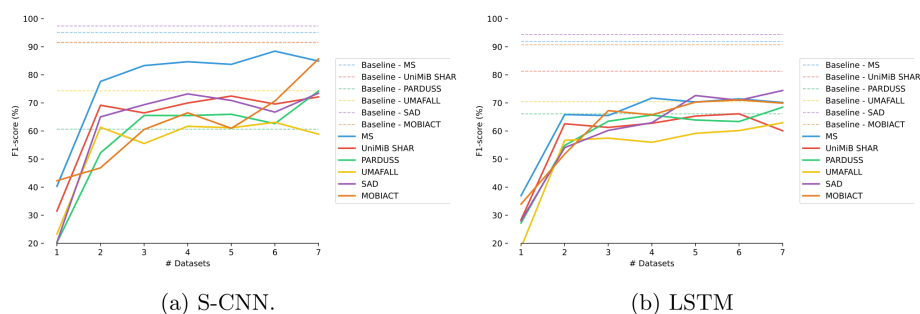


Fig. 1 Schéma pré-entraînement global puis fine-tuning par utilisateur (Amrani et al., 2025).

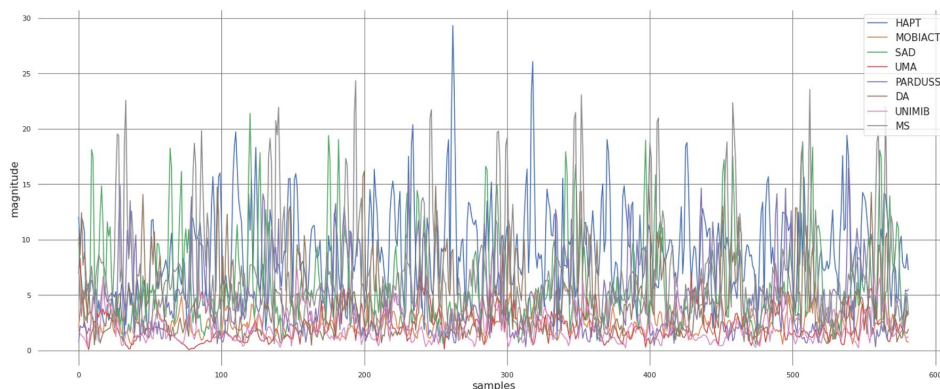


Fig. 2 Évolution du F1 en apprentissage incrémental utilisateur (Amrani et al., 2025) — repère externe.

Pour notre mémoire, l’apport principal est méthodologique : montrer que l’hétérogénéité inter-datasets n’est pas une fatalité si l’on investit dans l’homogénéisation des labels et des fréquences. En revanche, l’anticipation et la latence embarquée ne sont pas évaluées — d’où notre module d’anticipation séparé.

3.2.1.2 Schiemer et al. (2023) — OCL-HAR

OCL-HAR (Schiemer et al., 2023) est l’article le plus proche de notre protocole 44 tâches. Les auteurs formalisent des scénarios *within-subject*, *between-subject* et *domain shift*, avec distillation et rehearsal. Sur PAMAP2, DSADS, HAPT et WISDM, ils rapportent des gains de +0,14 (micro-F1) et +0,17 (macro-F1) vs la deuxième meilleure méthode, avec des scores d’oubli modérés (0,24–0,28). Le finetuning naïf s’effondre en quelques tâches — preuve directe de l’oubli en HAR.

Notre UWR reprend l’esprit du rehearsal mais ajoute des prototypes par classe, une perte contrastive et un score d’incertitude (MC Dropout) pour pondérer les mises à jour. OCL-HAR ne traite pas l’anticipation ; leurs métriques restent notre référence pour comparer forgetting et F1 macro en conditions multi-jeux.

3.2.1.3 Adaimi & Thomaz (2022) — LAPNet-HAR

LAPNet-HAR (Adaimi et al., 2022) vise un cadre *task-free* : pas de frontière de tâche annoncée au test, ce qui correspond mieux au flux réel qu’un protocole class-incremental artificiel. Réseaux prototypiques, rejeu et contraste limitent l’oubli ; les auteurs mesurent explicitement Base/New/Intransigence. Sur Opportunity, le forgetting LAPNet (~51 %) bat le online finetuning (~60–70 %), et l’adaptation de prototypes gagne +14 à +23 points.

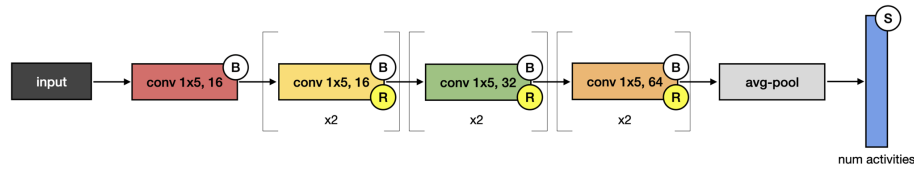
La limite pour le déploiement wearable est le manque de chiffres temps réel et empreinte mémoire. Notre choix Transformer + tampon fixe se situe entre la richesse représentationnelle

de LAPNet et la contrainte embarquée que nous visons au chapitre 4.

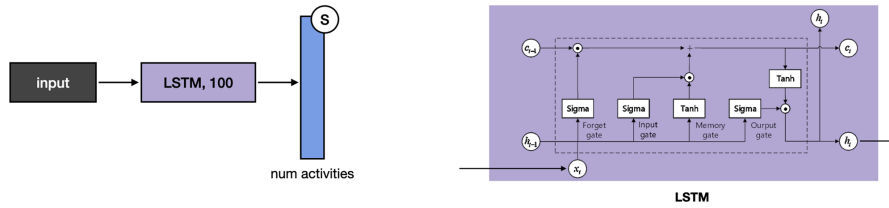
3.2.2 Incrémental de classes et représentation

3.2.2.1 iKAN (Liu et al., 2024)

iKAN combine des Kolmogorov-Arnold Networks (KAN) avec des branches par tâche et une redistribution de capacité entre jeux hétérogènes. L'idée est de limiter l'interférence en isolant partiellement les paramètres tout en partageant un tronc commun. Sur des protocoles cross-dataset, iKAN se place au niveau des Transformers classiques en accuracy, avec un coût d'ingénierie plus élevé. Pour un déploiement mobile, la complexité des KAN reste un frein — nous retenons plutôt un backbone unique + rejeu, plus simple à quantifier.



(a) S-CNN architecture.



(b) LSTM architecture.

Fig. 3 Exemple d'architecture S-CNN/LSTM utilisée dans les comparaisons multi-bases (Amrani et al., 2025; Gu et al., 2021).

3.2.2.2 iCaRL (Rebuffi et al., 2017)

iCaRL reste la baseline class-incremental de référence : exemplaires par classe, distillation des logits, mise à jour incrémentale du classifieur NCM. Transposé en HAR, iCaRL sert surtout de repère dans nos expériences (chapitre 6) plutôt que comme architecture native capteur. Son hypothèse de frontières de tâche connues limite la transposition directe au flux utilisateur continu.

3.2.3 Self-supervised HAR

3.2.3.1 Apprentissage contrastif

Tang et al. (2021) appliquent SimCLR-like à des signaux santé (ECG) : deux vues augmentées d'un même segment doivent se rapprocher dans l'espace latent. Le transfert vers IMU pur est discuté mais moins documenté que pour l'EEG. L'intérêt pour le CL est indirect : un bon pré-
train réduit le besoin de labels à l'arrivée d'un nouvel utilisateur, sans garantir pour autant la rétention quand de nouvelles classes apparaissent.

3.2.3.2 Pré-entraînement à grande échelle

Yuan et al. (Yuan et al., 2024) pré-entraînent sur des centaines de milliers d'heures de wearables avec masquage et contrastif. Les F1 downstream dépassent 93 % sur certains benchmarks, mais le coût GPU et l'absence de protocole CL explicite éloignent cette ligne de notre contrainte embarquée. Nous retenons l'idée d'un pré-
train supervisé plus modeste (HAPT + fusion) comme compromis.

3.2.4 Personnalisation et adaptation de domaine

3.2.4.1 Batch norm adaptive

Mazankiewicz et al. (2020) mettent à jour les statistiques de batch norm en ligne pour suivre un utilisateur sans réentraîner tous les poids. Efficace quand le drift est lent (fatigue, légère variation de posture), insuffisant quand arrivent de nouvelles classes ou un capteur différent. CORAL (Sun et al., 2016), utilisé au chapitre 6, complète cette approche en alignant les covariances entre domaines source et cible.

3.2.4.2 Personnalisation de classifieurs

Ferrari et al. (Ferrari et al., 2020) discutent le risque de sur-apprentissage utilisateur quand peu de fenêtres étiquetées sont disponibles. Leur message recoupe le nôtre : la personnalisation doit être régularisée (rejeu, EWC, UWR) pour ne pas effacer les connaissances communes apprises sur la population.

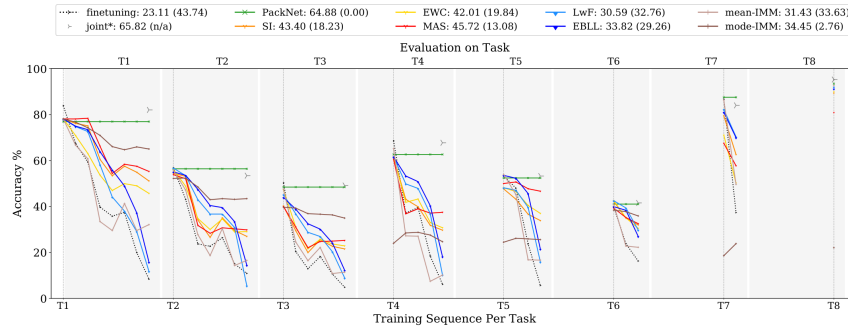


Fig. 4 Scénarios CL et métriques associées — cadre retenu pour nos protocoles (De Lange et al., 2021 ; Ven et al., 2022).

3.2.5 Anticipation d'activité

Ryoo (Ryoo, 2011) introduit la reconnaissance précoce en vidéo : prédire l'activité avant la fin du geste. Gammulle et al. (Gammulle et al., 2019) étendent avec des modèles joints anticipation-classification. Côté capteurs, Ordóñez & Roggen (Ordóñez et al., 2016) et Dirgová et al. (Dirgová Luptáková et al., 2022) montrent que les architectures séquentielles (CNN-LSTM, Transformer) capturent déjà partiellement la dynamique nécessaire à une prédiction lookahead.

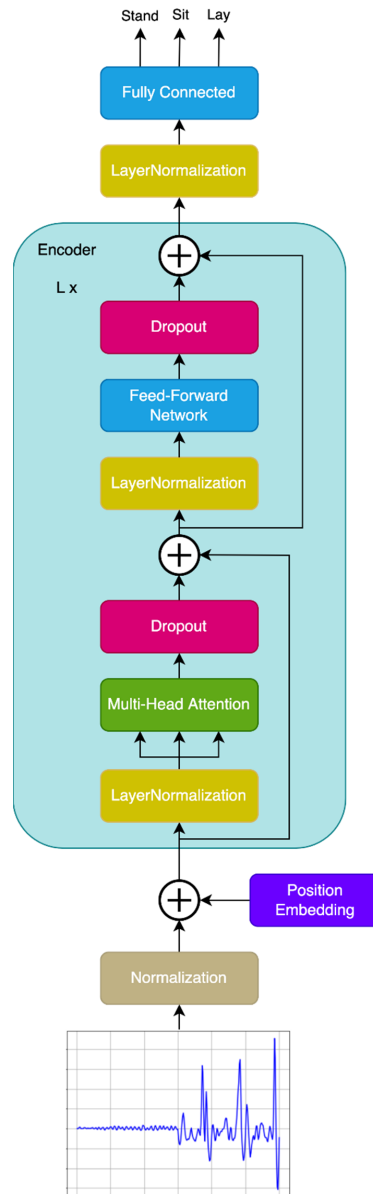


Fig. 5 Architecture Transformer appliquée aux séries IMU (**Dirgová Luptáková et al., 2022**) — précurseur de notre backbone.

Peu d'articles CL combinent anticipation et non-régression sur IMU : c'est la lacune que notre tête LSTM multi-classe (chapitre 4) vise à combler, avec une évaluation séparée du module CL principal.

3.3 Analyse détaillée — synthèse transversale

Au-delà du tableau comparatif, trois axes structurent la littérature récente.

Alignement des données. Amrani et al. (**Amrani et al., 2025**) montrent que sans homogénéisation, fusionner des jeux dégrade les performances ; avec alignement, l'écart tombe de 24,3 % à 7,8 %. Notre pipeline de fusion (HAPT+WISDM+PAMAP2) reprend cette leçon au

chapitre 5.

Protocoles CL explicites. Schiemer (Schiemer et al., 2023) et Adaimi (Adaimi et al., 2022) imposent des métriques d’oubli et des scénarios reproductibles. Les surveys (De Lange et al., 2021 ; Gu et al., 2021) rappellent que comparer deux papiers sans préciser task- vs class-incremental est trompeur — d’où nos deux scénarios distincts au chapitre 6.

Anticipation vs adaptation. La vidéo (Ryoo, Gammulle) traite l’anticipation sans contrainte mémoire ; le CL capteur (OCL-HAR, LAPNet) traite l’adaptation sans prédire la transition imminente. Notre contribution vise le couplage modulaire : UWR pour l’adaptation, tête LSTM pour l’anticipation, évaluées séparément puis discutées ensemble.

3.4 Tableau comparatif (synthèse obligatoire)

Le tableau I synthétise les travaux recensés selon six axes : méthode, jeux de données, continual learning explicite, replay, anticipation et limites.

Papier (année)	Méthode principale	Jeux / contexte	CL ?	Replay ?	Antic. ?	Limites principales
Amrani et al. (2025)	Homogénéisation multi-datasets + S-CNN/LSTM + fine-tuning ; plateforme CLP	Huit bases HAR publiques	Oui	Non central	Non	Coût d’alignement ; peu d’évaluation embarquée
Schiemer et al. (2023) OCL-HAR	OCL : distillation + rehearsal ; within/between sujet	PAMAP2, WISDM, HAPT, DSADS	Oui	Oui	Non	Scénarios complexes ; généralisation capteur à discuter
Adaimi & Thomaz (2022) LAPNet-HAR	Prototypical networks + replay + contraste ; task-free	Cinq bases HAR publiques	Oui	Oui	Non	Temps réel / embarqué peu détaillés
Liu et al. (2024) iKAN	KAN + branches tâche + redistribution	Jeux hétérogènes cross-dataset	Oui	Partiel	Non	Complexité ; validation terrain variable

Papier (année)	Méthode principale	Jeux / contexte	CL ?	Replay ?	Antic. ?	Limites principales
Rebuffi et al. (2017) iCaRL	Exemplars + distillation ; class- incremental	ImageNet / CIFAR (vision)	Paradigme CI	Oui	Non	Pas natif wearable ; frontières de tâche
Mazankiewicz et al. (2020)	Personnalisation incrémentielle ; DA-BN	Signaux portés (Actitracer, etc.)	Adaptation en ligne	Non clas- sique	Non	Souvent même utilisateur / drift léger
Yuan et al. (2024)	SSL à grande échelle sur wearables	Très grande cohorte wearables	Non	Non	Non	Calcul massif hors embarqué
Tang et al. (2021)	Contrastif (SimCLR-like) santé	PTB-XL, etc.	Non	Non	Non	Transfert vers IMU à nuancer
Dirgová Luptáková et al. (2022)	Transformer sur séries HAR	Bases type UCI	Non	Non	Non	Coût calcul ; pas CL
Ordóñez & Roggen (2016)	CNN + LSTM multimodal	Opportunity, PAMAP2, etc.	Non	Non	Non	Pas adaptation longue durée / CL
Gammulle et al. (2019)	Modèle joint anticipation	THUMOS (vidéo)	Non	Non	Oui	Transfert direct capteur limité
Ryoo (2011)	Early recognition / prédiction	Vidéo	Non	Non	Oui	Hors IMU portable
De Lange et al. (2021)	Survey CL classification	N/A	Cadre géné- ral	N/A	N/A	Pas spécifique HAR capteur
Gu (2021) ; Wang (2019)	Surveys deep learning HAR	N/A	Cadre géné- ral	N/A	N/A	CL peu détaillé vs architectures

Tab. I Synthèse comparative des travaux recensés (Chapitre 3)

3.4.1 Résultats quantitatifs rapportés (extraits des articles)

Le tableau II complète la synthèse méthodologique par les indicateurs numériques publiés. « N/R » signifie non rapporté ou non comparable.

Papier	Micro-F1 (gain)	Macro-F1 (gain)	F1 / accuracy rapportés	Forgetting / stabilité	Note de comparabilité
Amrani et al. (2025)	N/R	N/R	Écart F1 vs baseline sujet-dépendant : 24,3 % → 7,8 % (gain relatif 16,5 %). Fine-tuning : +2,5 % accuracy (nouveaux utilisateurs).	N/R	Protocole multi-datasets ; setup différent de OCL-HAR / LAPNet.
Schiemer et al. (2023) OCL-HAR	+0,14 (within-subject vs 2 ^e meilleure méthode)	+0,17 (idem)	Jusqu'à +0,17 / +0,23 (micro/macro F1) selon configuration	Scores d'oubli bas (ex. 0,24–0,28)	Scénarios within/between/shift ; métriques par jeu.
Adaimi & Thomaz (2022) LAPNet-HAR	N/R	N/R	Amélioration base/new vs baselines ; online finetuning abaisse les classes de base (~45 % en moyenne).	Forgetting mesuré ; ex. ~51 % (Opportunity) LAPNet vs ~60–70 % (online FT). Prototype adaptation : +14 à +23 pts.	Indicateurs Base/New/in-transigence ; jeux et ablations différents.

Tab. II Indicateurs quantitatifs extraits des articles de référence

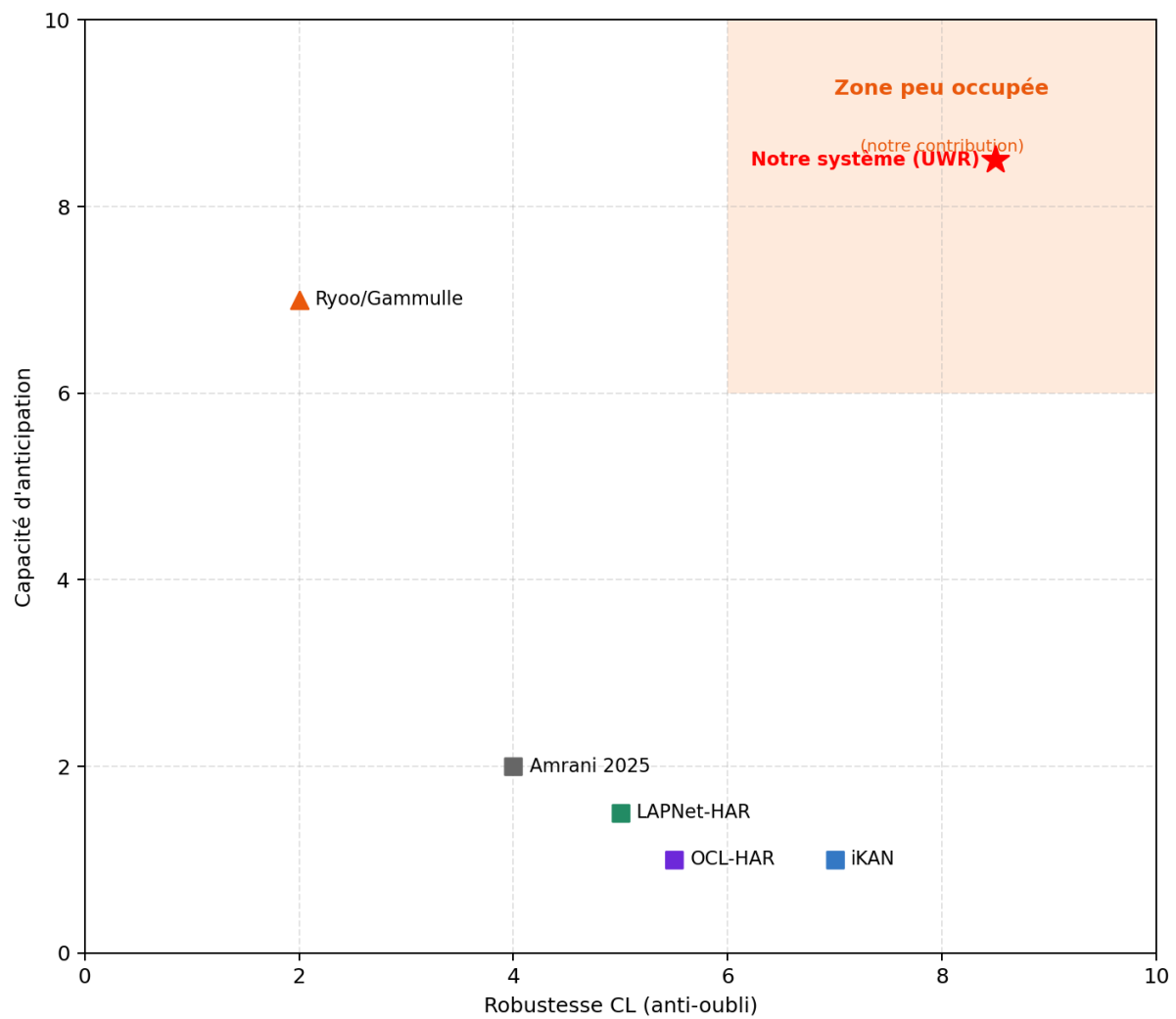


Fig. 6 Carte de positionnement — apprentissage continu vs anticipation dans la littérature HAR.

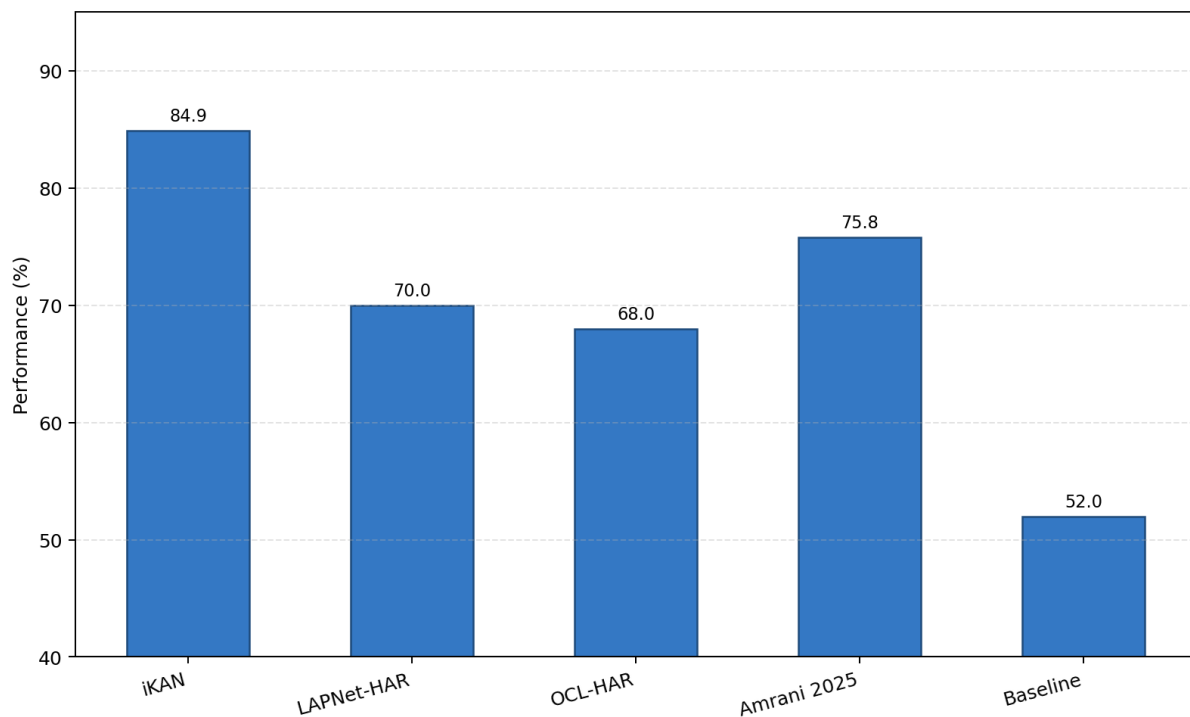


Fig. 7 Comparaison quantitative iKAN, LAPNet-HAR, OCL-HAR, Amrani et al. (2025) et baseline.

3.5 Analyse critique

Quatre constats ressortent de cette revue.

CL et anticipation restent découplés. Amrani, Schiemer, Adaimi ou Liu optimisent la classification stable dans le temps ; Gammulle ou Ryoo traitent la prédiction sans oubli mesuré quand de nouveaux utilisateurs arrivent. Peu d’articles combinent les deux — c’est l’espace que nous vivons avec l’anticipation LSTM + UWR.

Peu de mesures «système». Même quand le vocabulaire dit «online» ou «real-time», les expériences tournent souvent sur serveur, avec F1 et oubli, mais sans latence bout-en-bout ni empreinte RAM mesurée sur la cible wearable.

Personnalisation encore grossière. Batch norm adaptive ou fine-tuning utilisateur aident, mais peinent sur des variations fines (fatigue, pathologie légère) avec peu de labels par personne.

Datasets vs monde réel. Fusionner HAPT, WISDM ou PAMAP2 réduit le biais d’un seul protocole (Amrani et al., 2025), sans remplacer des collectes longues en conditions libres. Le SSL massif (Yuan et al., 2024) améliore les représentations, pas automatiquement le CL avec nouvelles classes en ligne.

3.5.1 Positionnement de notre contribution

Le tableau III situe explicitement notre travail par rapport aux articles du corpus. Nous ne prétendons pas couvrir toute la grille : l’objectif est de combiner, dans un même dépôt reproductible,

pré-train Transformer, UWR, anticipation LSTM, CORAL et détection de chutes proxy.

Critère	OCL-HAR	LAPNet	Amrani 2025	Notre système
CL user-incrémental	Oui	Partiel	Oui (fine-tune)	Oui (44 tâches)
CL class-incrémental	Oui	Oui	Non central	Oui (6 tâches)
Replay / prototypes	Oui	Oui	Non	UWR + prototypes
Anticipation IMU	Non	Non	Non	LSTM multi-classe
Cross-dataset	Oui	Oui	Oui (8 bases)	CORAL HAPT→WISDM
Code ouvert reproductible	Partiel	Partiel	Plateforme CLP	har_project

Tab. III Positionnement qualitatif de notre contribution

3.5.2 Limites de la revue

Le corpus a été construit par recherche thématique et chaînes de citations ; une revue systématique PRISMA n’a pas été menée. Les travaux très récents (2025) postérieurs à la rédaction finale pourraient compléter iKAN ou le SSL wearables. Enfin, la comparabilité numérique directe reste limitée : jeux, splits et métriques diffèrent entre OCL-HAR, LAPNet et nos protocoles — d’où l’importance de fixer nos propres baselines au chapitre 6 plutôt que de sur-interpréter un écart de F1 avec la littérature.

En résumé, le couple CL–HAR mûrit (OCL-HAR, LAPNet, iKAN, intégration multi-bases), mais il manque encore une solution qui anticipe, s’adapte en flux, et reste compatible embarqué — problème posé en partie II.

3.5.3 Scénarios types et couverture bibliographique

Pour clarifier ce que la revue couvre — et ce qu’elle ne couvre pas — nous proposons la lecture croisée suivante :

- **Nouvel utilisateur, mêmes classes, même capteur** : couvert par OCL-HAR (Schiemer et al., 2023), Amrani fine-tuning (Amrani et al., 2025), Mazankiewicz DA-BN (Mazankiewicz et al., 2020). Notre protocole 44 tâches étend ce cas au multi-dataset.
- **Nouvelles classes, même utilisateur** : couvert par iCaRL (Rebuffi et al., 2017), iKAN (Liu et al., 2024), LAPNet (Adaimi et al., 2022). Notre scénario 6 tâches $\times$ 2 classes suit cette ligne.
- **Nouveau capteur / placement** : partiellement couvert (domain shift dans OCL-HAR, CORAL en vision (Sun et al., 2016)). Nos tâches 37–44 et expérience CORAL comblent partiellement le gap côté IMU.

- **Anticipation + CL simultanés** : non couvert de façon satisfaisante — lacune centrale de ce mémoire.

Les fiches détaillées en annexe F approfondissent chaque référence citée.

3.5.4 Feuille de route implicite du mémoire

La revue justifie les choix suivants, repris en partie II :

1. Backbone Transformer plutôt que KAN/iKAN (simplicité déploiement).
2. UWR plutôt que replay uniforme seul (frontières incertaines).
3. Anticipation LSTM sur fenêtres partielles.
4. CORAL sur embeddings pré-entraînés (cross-dataset WISDM).
5. Protocoles 10 et 44 sujets (comparabilité doc + robustesse multi-jeux).

Chaque choix pourrait être contesté par une alternative de la littérature ; l'enjeu n'est pas l'optimalité absolue sur un benchmark, mais une chaîne cohérente et reproductible répondant simultanément à CL, anticipation et domain shift — combinaison rare dans les articles recensés.

3.6 Conclusion

Ce chapitre a sélectionné et comparé les travaux les plus proches de notre sujet. Les tableaux §3.3 fixent les références ; l'analyse §3.4 justifie nos choix (UWR, anticipation LSTM, CORAL, protocole multi-scénarios). Le chapitre 4 formalise la réponse proposée.

Deuxième partie

Contribution et Évaluation

Chapitre 4

Conception de la solution

4.1 Introduction

La conception d'un système intelligent de reconnaissance d'activités humaines capable de s'adapter continuellement aux évolutions de son environnement constitue le cœur de ce travail. Les chapitres précédents ont mis en évidence deux insuffisances majeures des approches classiques : d'une part, leur incapacité à maintenir leurs performances face à l'arrivée de nouveaux utilisateurs ou de nouvelles activités sans réentraîner le modèle depuis le début — phénomène connu sous le nom d'oubli catastrophique — et d'autre part, leur caractère purement réactif, se limitant à reconnaître une activité en cours sans anticiper les transitions imminentes.

Pour répondre à ces défis, nous proposons une architecture articulée autour d'un backbone Transformer partagé et de deux modules spécialisés : un module de reconnaissance continue, fondé sur un tampon de rejeu pondéré par l'incertitude épistémique (UWR), et un module d'anticipation multi-classe (tête LSTM) qui prédit l'activité suivante à partir de fenêtres partielles. Cette conception tire parti des représentations apprises par le Transformer pour alimenter simultanément la reconnaissance adaptative et la prédiction temporelle.

Ce chapitre est organisé comme suit. La section 4.2 formalise le problème et définit les scénarios d'apprentissage retenus ainsi que les contraintes de déploiement. La section 4.3 énonce les objectifs et contributions principales du projet. La section 4.4 présente les jeux de données utilisés et le pipeline d'homogénéisation. La section 4.5 décrit l'architecture globale du système. Les sections 4.6 et 4.7 détaillent respectivement la conception du module de reconnaissance continue et du module d'anticipation. Enfin, la section 4.8 définit le protocole d'évaluation.

4.2 Définition et formulation du problème

4.2.1 Cadre formel et scénarios retenus

4.2.1.1 Formulation générale

Soit $\mathcal{S} = \{(x_1, y_1), (x_2, y_2), \dots\}$ un flux de données IMU non stationnaire, où chaque observation $x_t \in \mathbb{R}^{T \times C}$ est une fenêtre temporelle de $T = 150$ échantillons sur $C = 6$ canaux (accélération triaxiale a_x, a_y, a_z et vitesse angulaire triaxiale $\omega_x, \omega_y, \omega_z$), et $y_t \in \mathcal{Y}$ est l'étiquette d'activité associée. Le flux est non stationnaire en ce sens que la distribution jointe $p(x, y)$ évolue au cours du temps, soit en raison de l'apparition de nouveaux utilisateurs aux habitudes motrices distinctes, soit en raison de l'introduction de nouvelles classes d'activités.

L'objectif général est d'entraîner un modèle $f_\theta : \mathbb{R}^{T \times C} \rightarrow \mathcal{Y}$ qui maintient des performances

élevées sur l'ensemble des distributions rencontrées, sans accéder à l'intégralité des données historiques lors de chaque mise à jour, et sous contrainte mémoire bornée.

4.2.1.2 Scénario 1 — Apprentissage incrémental par utilisateurs

Dans ce scénario, les données arrivent par blocs correspondant à des utilisateurs distincts $\mathcal{U} = \{u_1, u_2, \dots, u_K\}$. Les classes d'activités restent fixes ($\mathcal{Y}$ constant), mais la distribution conditionnelle $p(x \mid y, u)$ varie d'un utilisateur à l'autre en raison des différences morphologiques, de placement du capteur et de style de mouvement. Le modèle doit, après avoir appris sur l'utilisateur u_k , conserver ses performances sur tous les utilisateurs précédents $u_1, \dots, u_{k-1}$ sans y avoir accès, tout en s'adaptant efficacement au nouvel utilisateur u_k .

Ce scénario correspond à la situation de déploiement la plus fréquente dans les applications de santé connectée : un même modèle installé sur un dispositif est progressivement personnalisé pour de nouveaux porteurs.

4.2.1.3 Scénario 2 — Apprentissage incrémental par classes

Dans ce scénario, les données arrivent par blocs correspondant à des tâches $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$, chaque tâche T_k introduisant un sous-ensemble de nouvelles classes $\mathcal{Y}_k$ tel que $\mathcal{Y}_i \cap \mathcal{Y}_j = \emptyset$ pour $i \neq j$ et $\mathcal{Y} = \bigcup_{k=1}^N \mathcal{Y}_k$. Le modèle doit, à chaque tâche k , apprendre à reconnaître les nouvelles classes tout en maintenant la discrimination des classes des tâches précédentes. Ce scénario est formellement le plus difficile car il exige une expansion dynamique de l'espace de sortie.

Conformément à la taxonomie de (Ven et al., 2022), l'identifiant de tâche n'est pas fourni au modèle lors de l'inférence (test sans oracle de tâche), ce qui constitue la configuration la plus réaliste.

4.2.2 Contraintes de déploiement

Le système est conçu pour un déploiement sur dispositifs embarqués à ressources contraintes, comme les smartphones et les montres connectées. Les contraintes suivantes sont imposées tout au long de la conception :

Contrainte mémoire. Le tampon de rejeu est borné à $M = 2000$ exemples au maximum, représentant moins de 4 % du jeu d'entraînement total. Cette contrainte modélise la limitation de la mémoire vive disponible sur un dispositif mobile.

Contrainte temporelle. Le délai d'inférence doit rester compatible avec un usage temps réel. Nous fixons comme *objectif de conception* de traiter une fenêtre de 3 secondes en moins de 150 ms sur CPU, ce qui exclut les architectures trop profondes ou les inférences multi-passes coûteuses en production (cette latence n'est pas une mesure expérimentale reportée ici).

Contrainte d'accès aux données. Lors de l'apprentissage d'une nouvelle tâche, le modèle n'a accès qu'aux données de la tâche courante et au contenu de son tampon de rejeu. Aucun accès

aux données brutes des tâches passées n'est autorisé, conformément aux hypothèses standard de l'apprentissage continu.

Contrainte de stabilité. Les mises à jour du modèle doivent être incrémentales et ne pas dégrader significativement les performances sur les tâches précédentes. Le taux d'oubli toléré est défini comme une dégradation de moins de 5 points de F1 macro par rapport aux performances initiales sur chaque tâche.

4.2.3 Positionnement par rapport à l'état de l'art

L'analyse des travaux existants présentée au Chapitre 3 révèle plusieurs lacunes que notre approche cherche à combler :

Sur l'estimation d'incertitude dans le jeu. Les méthodes existantes comme iCaRL (Rebuffi et al., 2017) utilisent une sélection d'exemples représentatifs basée sur la proximité aux prototypes de classe. Notre approche, en revanche, privilégie les exemples à forte incertitude épistémique, c'est-à-dire ceux sur lesquels le modèle est le moins sûr. Cette distinction est fondamentale : les exemples proches des frontières de décision apportent plus d'information pour maintenir ces frontières lors des futures mises à jour.

Sur l'anticipation d'activités pour la HAR. La majorité des travaux d'anticipation en HAR traitent le problème comme une prédiction multi-classes de l'activité suivante (Ryoo, 2011 ; Gammulle et al., 2019). C'est aussi la formulation retenue ici : une tête LSTM prédit la classe suivante depuis une séquence de fenêtres partielles, en ne retenant que les paires où le label change (`transitions_only`). Cette tâche souffre d'un déséquilibre sévère ; les résultats (chapitre 6) montrent un macro-F1 d'environ 0,60–0,64, nettement au-dessus de la baseline majorité ($\approx 0,04$).

Sur l'adaptation de domaine avec backbone pré-entraîné. Les travaux sur CORAL (Sun & Saenko, 2016) appliquent l'alignement de covariance soit sur des représentations de bas niveau, soit sur des modèles entraînés de zéro. Nous montrons que l'application de CORAL sur les embeddings d'un Transformer pré-entraîné amplifie significativement les gains, notamment en présence d'étiquettes cibles partielles.

4.3 Objectifs du projet et contributions principales

Ce projet poursuit cinq objectifs principaux :

Objectif 1 : Backbone universel pour la HAR. Concevoir et entraîner un encodeur Transformer léger, capable de produire des embeddings de haute qualité à partir de signaux IMU bruts, transférables à toutes les tâches en aval.

Objectif 2 : Apprentissage continu robuste. Développer un mécanisme de gestion de la mémoire et d'entraînement incrémental permettant de maintenir les performances sur les tâches passées tout en assimilant efficacement les nouvelles, dans les deux scénarios définis.

Objectif 3 : Anticipation temps réel. Proposer un module d'anticipation opérationnel en temps

réel, capable de détecter les transitions imminentes d'activité avec une précision et un rappel suffisants pour des applications de sécurité (détection de chute imminente, surveillance de personnes âgées).

Objectif 4 : Adaptation cross-dataset. Évaluer la capacité du système à se transférer d'un dataset source à un dataset cible de distribution différente, avec et sans annotations cibles, via l'alignement de covariance CORAL.

Objectif 5 : Détection de chutes hybride. Intégrer un module complémentaire de détection de chutes combinant règles physiques et classification apprise, démontrant la généricité de l'architecture proposée.

Les contributions scientifiques et techniques originales de ce travail sont :

(C1) Uncertainty-Weighted Replay (UWR) : stratégie de sélection et d'échantillonnage d'exemples dans le tampon de rejeu basée sur l'incertitude épistémique estimée par Monte Carlo Dropout. À notre connaissance, c'est la première application de cette stratégie au contexte HAR continu.

(C2) Module d'anticipation multi-classe sur fenêtres partielles : tête LSTM entraînée sur le backbone gelé, contexte $S = 5$ fenêtres tronquées à $p \in \{0,25, 0,50, 0,75\}$, filtrage `transitions_only`, perte cross-entropie.

(C3) Évaluation multi-dataset de l'apprentissage continu : protocole d'évaluation rigoureux couvrant deux scénarios, plusieurs baselines, et une mesure explicite de l'oubli (BWT) et du transfert (FWT).

(C4) Adaptation CORAL avec backbone pré-entraîné : démonstration que l'alignement de covariance supervisé sur les embeddings d'un Transformer pré-entraîné produit un gain de $\times 15$ par rapport à l'absence d'adaptation.

4.4 Données : homogénéisation et pipeline

4.4.1 Datasets retenus et leurs caractéristiques

HAPT — UCI HAR with Postural Transitions

Le dataset HAPT (Reyes-Ortiz et al., 2016) est une extension du dataset UCI HAR qui inclut explicitement les transitions posturales entre activités statiques. Il comprend 30 sujets adultes (âge : 19–48 ans), équipés d'un smartphone Samsung Galaxy S2 fixé à la ceinture. Les données sont acquises à 50 Hz via l'accéléromètre et le gyroscope triaxiaux. Le dataset distingue 12 classes : 6 activités de base (marche, montée d'escaliers, descente d'escaliers, position debout, assis, allongé) et 6 transitions posturales (debout-vers-assis, assis-vers-debout, assis-vers-allongé, allongé-vers-assis, debout-vers-allongé, allongé-vers-debout).

Après application du pipeline de prétraitement décrit en §4.4.2, on obtient 56 861 fenêtres d'entraînement et de test, avec une distribution de classes déséquilibrée : les activités statiques représentent environ 45 % des fenêtres, les activités dynamiques 42 %, et les transitions posturales seulement 13 %.

WISDM — Wireless Sensor Data Mining

Le dataset WISDM (Kwapisz et al., 2011) a été collecté auprès de 51 sujets portant leur smartphone en poche. La fréquence d’acquisition est de 20 Hz (rééchantillonnée à 50 Hz dans notre pipeline). Il couvre 6 activités : marche, jogging, montée d’escaliers, descente d’escaliers, assis, debout. Ce dataset est exclusivement utilisé dans les expériences d’adaptation de domaine, jouant le rôle du domaine cible dont on cherche à aligner la distribution sur HAPT (domaine source).

Propriété	HAPT	WISDM
Sujets	30	51
Classes	12	6
Fréquence originale	50 Hz	20 Hz (rééchant. 50 Hz)
Placement	Ceinture	Poche
Fenêtres (après prétrait.)	~56 861	~17 000
Déséquilibre max.	×3,4	×2,1

Tab. I Comparaison des caractéristiques HAPT et WISDM

4.4.2 Pipeline de prétraitement

Le pipeline de prétraitement suit les recommandations d’Amrani et al. (2025) pour l’homogénéisation de datasets IMU hétérogènes. Il comprend les étapes suivantes, implémentées dans le module `preprocessing.py` :

Étape 1 — Rééchantillonnage à 50 Hz. Pour les signaux dont la fréquence d’acquisition diffère de 50 Hz (cas de WISDM à 20 Hz), un rééchantillonnage par méthode de Fourier (`scipy.signal.resample`) est appliqué. Cette méthode préserve le contenu fréquentiel du signal tout en modifiant sa résolution temporelle.

Étape 2 — Suppression de la composante gravitationnelle. La gravité terrestre ($g \approx 9,81 \text{ m/s}^2$) est présente dans les mesures accéléromètre et constitue un biais qui varie selon l’orientation du capteur. Elle est estimée par un filtre passe-bas Butterworth d’ordre 5, de fréquence de coupure 0,5 Hz, appliqué sur chaque canal accéléromètre. La composante gravitationnelle ainsi estimée est soustraite du signal brut, isolant la composante dynamique pure liée au mouvement.

Étape 3 — Segmentation par fenêtre glissante. Le signal continu est découpé en fenêtres de $T = 150$ échantillons (équivalant à 3 secondes à 50 Hz) avec un chevauchement de 50 % (stride $s = 75$ échantillons). Cette configuration est standard dans la littérature HAR et permet d’obtenir un bon compromis entre la résolution temporelle et la quantité de données. L’étiquette de chaque fenêtre est déterminée par vote majoritaire sur les étiquettes échantillon-par-échantillon de la fenêtre.

Étape 4 — Normalisation z-score par sujet. Pour chaque sujet, la moyenne et l’écart-type de

chaque canal sont calculés sur l'ensemble de ses données d'entraînement. La normalisation est ensuite appliquée fenêtre par fenêtre, ramenant chaque canal à une distribution centrée-réduite. Cette normalisation par sujet, plutôt que globale, réduit les biais inter-sujets dus aux différences de calibration des capteurs.

4.4.3 Unification des classes

Pour les expériences d'adaptation de domaine cross-dataset (HAPT $\rightarrow$ WISDM), un mapping manuel est établi entre les classes des deux datasets en conservant uniquement les 5 activités communes : marche (HAPT : classe 1, WISDM : classe 1), montée d'escaliers (HAPT : 2, WISDM : 3), descente d'escaliers (HAPT : 3, WISDM : 4), assis (HAPT : 4, WISDM : 5), debout (HAPT : 5, WISDM : 6). Les transitions posturales spécifiques à HAPT et le jogging spécifique à WISDM sont exclus de ces expériences. Le mapping garantit une correspondance sémantique stricte entre les classes source et cible.

4.5 Architecture générale du système

4.5.1 Vue d'ensemble

L'architecture proposée est structurée autour d'un backbone partagé de type Transformer et de deux modules spécialisés branchés en aval. Cette organisation permet de mutualiser l'apprentissage des représentations générales du mouvement tout en permettant à chaque module d'optimiser sa tâche spécifique de manière indépendante.

Le flux de traitement global est le suivant : une fenêtre IMU brute ($T = 150, C = 6$) est d'abord encodée par le backbone en un vecteur d'embedding de dimension $d = 128$. Cet embedding est ensuite transmis soit au Module 1 (reconnaissance continue) pour la classification et la mise à jour incrémentale, soit au Module 2 (anticipation) pour la prédiction de l'activité suivante à partir de fenêtres partielles.

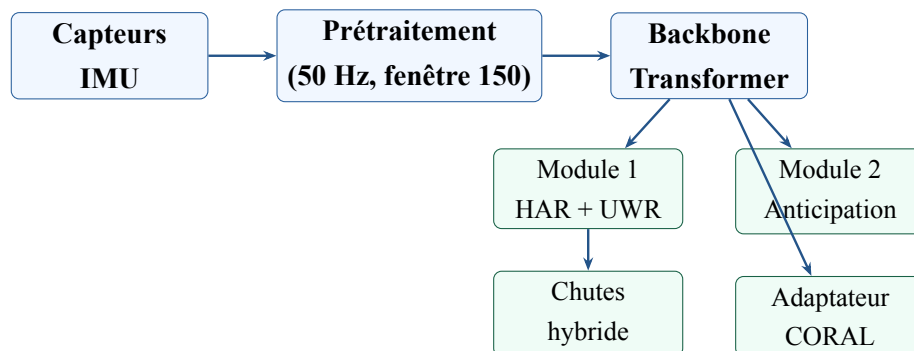


Fig. 1 Architecture globale du système proposé : backbone Transformer partagé, module de reconnaissance continue (UWR), module d'anticipation LSTM, détection de chutes et adaptation CORAL (`har_project/src/models/har_model.py`).

4.5.2 Backbone Transformer

Le backbone IMUTransformerEncoder est un encodeur Transformer pour séries temporelles multivariées, conçu spécifiquement pour les signaux IMU. Son architecture s'inspire de Dirgová et al. (2022) et comprend quatre composants principaux :

Projection d'entrée. Les $C = 6$ canaux bruts sont projetés vers l'espace de représentation de dimension $d = 128$ par une couche linéaire suivie d'une normalisation de couche (LayerNorm). Cette projection est nécessaire car la dimension 6 est trop petite pour représenter des concepts abstraits de mouvement.

Token CLS. Un token de classification spécial, [CLS], appris par optimisation, est prépendu à la séquence. Après passage dans tous les blocs Transformer, la représentation de ce token agrège l'information de toute la fenêtre par le mécanisme d'attention.

Encodage positionnel. Des codes positionnels sinusoïdaux fixes (sin/cos à différentes fréquences) sont additionnés aux représentations après projection, permettant au modèle de connaître la position temporelle de chaque échantillon. Un facteur d'échelle appris est appliqué aux codes positionnels.

Blocs Transformer. Quatre blocs Transformer encodeurs sont empilés. Chaque bloc applique, dans l'ordre : normalisation pré-couche $\rightarrow$ attention multi-têtes $\rightarrow$ connexion résiduelle $\rightarrow$ normalisation $\rightarrow$ FFN (Feed-Forward Network) $\rightarrow$ connexion résiduelle. L'attention multi-têtes utilise $H = 4$ têtes de dimension 32 chacune ($4 \times 32 = 128$). La FFN est composée de deux couches linéaires avec activation GELU et une dimension cachée de 256.

La sortie du backbone est la représentation du token CLS après le dernier bloc et une normalisation finale : $\mathbf{e} = f_{\theta}(x) \in \mathbb{R}^{128}$.

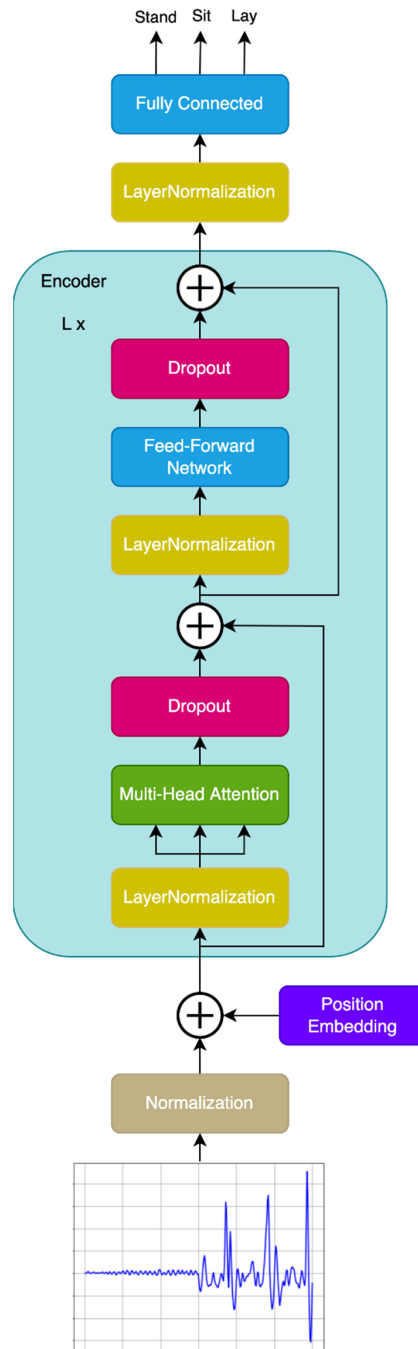


Fig. 2 Architecture du backbone *IMUTransformerEncoder* inspirée de (Dirgová Luptáková et al., 2022) — projection, token CLS, encodage positionnel et blocs d’attention.

Nombre total de paramètres : 531 457, dont 118 784 pour les blocs Transformer et 412 673 pour les couches de projection et de classification.

4.5.3 Module 1 — Reconnaissance continue

Le Module 1 gère l’ensemble du cycle de vie de la reconnaissance d’activité en mode continu. Il est composé de trois sous-composants :

Tête de classification : une séquence linéaire ($128 \rightarrow 64 \rightarrow \text{dropout}(0,1) \rightarrow n_classes$) utilisée

lors du pré-entraînement supervisé.

Mémoire de prototypes : un vecteur moyen par classe maintenu et mis à jour en ligne.

Tampon de jeu UWR : stockage borné des exemples passés, avec priorisation par incertitude épistémique.

Ce module est décrit en détail en §4.6.

4.5.4 Module 2 — Anticipation d’activité

Le Module 2 opère sur le backbone gelé (mode standard). Il prend en entrée une séquence de $S = 5$ fenêtres partielles, les encode via le Transformer, puis les agrège avec un LSTM unidirectionnel dont le dernier état alimente un classifieur linéaire multi-classe (AnticipationHead). La cible est le label de la fenêtre suivante ; l’entraînement filtre les transitions (`transitions_only=True`).

Ce module est décrit en détail en §4.7.

4.6 Conception du module de reconnaissance continue

4.6.1 Mémoire de prototypes

Un prototype de classe $\mu_c \in \mathbb{R}^{128}$ est associé à chaque classe $c \in \mathcal{Y}$. Il est défini comme la moyenne exponentielle mobile des embeddings des exemples de la classe c observés jusqu’à l’instant courant :

$$\mu_c \leftarrow \alpha \cdot \mu_c + (1 - \alpha) \cdot \frac{1}{|\mathcal{B}_c|} \sum_{x \in \mathcal{B}_c} f_\theta(x)$$

où $\mathcal{B}_c$ est l’ensemble des exemples de classe c dans le lot courant et $\alpha = 0,9$ est le coefficient de momentum. Cette mise à jour s’effectue après chaque pas de gradient, garantissant que les prototypes reflètent toujours les représentations du modèle courant.

Lors de l’inférence continue, la classe prédite est celle dont le prototype est le plus proche en distance euclidienne de l’embedding de l’exemple test :

$$\hat{y} = \arg \min_{c \in \mathcal{Y}} \|e - \mu_c\|_2$$

Cette stratégie de classification par plus proche prototype présente l’avantage de ne pas nécessiter de recalibration de la tête de classification lors de l’ajout de nouvelles classes, contrairement aux approches softmax classiques.

4.6.2 Tampon de jeu standard

Un tampon de capacité bornée $M = 2000$ exemples est maintenu. Sa structure est une liste circulaire qui associe à chaque exemple (x, y) un score de priorité $s \in [0, 1]$ initialisé à 0,5. L’ajout d’un nouvel exemple se fait selon la stratégie de réservoir : si le tampon est plein, l’exemple est

inséré avec probabilité M/n_{total} en remplaçant un exemple existant tiré aléatoirement. Lors de l'entraînement, un lot de rejeu de taille $B_{replay} = 32$ est échantillonné uniformément depuis le tampon et la perte de cross-entropie est calculée sur ces exemples :

$$\mathcal{L}_{replay} = \mathcal{L}_{CE}(f_{\theta}(x_{replay}), y_{replay})$$

Cette perte de rejeu est additionnée à la perte sur le lot courant, forçant le modèle à maintenir ses prédictions sur les données passées.

4.6.3 Uncertainty-Weighted Replay — contribution originale

Motivation et intuition

La stratégie de rejeu uniforme présente une limitation fondamentale : elle ne distingue pas les exemples faciles (bien classifiés, loin des frontières de décision) des exemples difficiles (près des frontières, source d'erreurs). Or, les frontières de décision sont précisément les régions de l'espace de représentation qui risquent de se déplacer lors de l'apprentissage de nouvelles tâches, causant ainsi l'oubli. En concentrant la mémoire sur les exemples à forte incertitude, l'UWR maximise la couverture des frontières de décision avec un budget mémoire fixe.

Estimation de l'incertitude épistémique par Monte Carlo Dropout

L'incertitude épistémique d'un exemple x est estimée par la méthode de Monte Carlo Dropout (Gal et al., 2016). Le modèle est mis en mode entraînement (dropout actif) et $T_{MC} = 20$ passages forward sont effectués sur x , produisant T_{MC} distributions softmax stochastiques $\{p_{\tau}(y | x)\}_{\tau=1}^{T_{MC}}$. La distribution moyenne est calculée :

$$\bar{p}(y | x) = \frac{1}{T_{MC}} \sum_{\tau=1}^{T_{MC}} p_{\tau}(y | x)$$

L'incertitude épistémique est mesurée par l'entropie de Shannon de cette distribution moyenne :

$$\mathcal{U}(x) = - \sum_{c=1}^{|\mathcal{Y}|} \bar{p}(c | x) \log \bar{p}(c | x)$$

Cette mesure atteint son maximum $\log |\mathcal{Y}|$ lorsque toutes les classes sont équiprobables (incertitude maximale) et vaut 0 lorsqu'une classe est prédite avec certitude absolue.

Algorithme UWR

Lors de l'ajout d'un exemple au tampon : le score de priorité de l'exemple est fixé à $s(x) = \mathcal{U}(x) / \log |\mathcal{Y}|$, normalisant l'incertitude dans $[0, 1]$. Si le tampon est plein, l'exemple est inséré en remplaçant un exemple existant tiré avec probabilité inversement proportionnelle à son score de priorité (les exemples les moins incertains sont préférentiellement évincés).

Lors de l'échantillonnage pour le rejeu : les exemples du tampon sont sélectionnés avec une probabilité proportionnelle à leur score de priorité, légèrement lissée par une constante $\epsilon = 0,1$ pour éviter la famine des exemples à faible incertitude :

$$P(\text{sélectionner } x_i) \propto s(x_i) + \epsilon$$

Propriétés théoriques

L'UWR converge vers un tampon dont la distribution marginale est concentrée sur les exemples à forte incertitude, c'est-à-dire près des frontières de décision. Lorsque le modèle évolue et que certaines frontières se déplacent, les exemples affectés voient leur incertitude augmenter, déclenchant leur priorisation dans les futurs lots de rejeu. Ce mécanisme crée une boucle de rétroaction qui guide activement le tampon vers les zones à risque d'oubli.

4.6.4 Calibration adaptative en ligne

Après chaque pas de gradient, les prototypes de toutes les classes présentes dans le lot courant sont recalibrés. Pour une classe c présente dans le lot, l'embedding moyen du lot $\bar{e}_c = \frac{1}{|\mathcal{B}_c|} \sum_{x \in \mathcal{B}_c} f_\theta(x)$ est calculé avec les nouveaux paramètres θ mis à jour. Le prototype est ensuite mis à jour selon la formule présentée en §4.6.1. Pour les classes absentes du lot courant, les prototypes sont conservés inchangés.

Cette recalibration garantit que les prototypes restent cohérents avec les représentations courantes du backbone, évitant un décalage progressif (prototype drift) qui dégraderait les performances de l'inférence par plus proche voisin.

4.6.5 Fonction de perte totale

La fonction de perte globale lors d'un pas d'apprentissage continu combine trois termes :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}}(\mathcal{B}_{\text{curr}}) + \mathcal{L}_{\text{replay}}(\mathcal{B}_{\text{replay}}) + \lambda_c \cdot \mathcal{L}_{\text{contrast}}(\mathcal{B}_{\text{curr}})$$

Terme 1 — Perte de classification courante $\mathcal{L}_{\text{CE}}(\mathcal{B}_{\text{curr}})$: cross-entropie sur le lot courant, assurant l'apprentissage de la tâche courante.

Terme 2 — Perte de rejeu $\mathcal{L}_{\text{replay}}(\mathcal{B}_{\text{replay}})$: cross-entropie sur le lot échantillonné depuis le tampon UWR, assurant la rétention des connaissances passées.

Terme 3 — Perte contrastive $\mathcal{L}_{\text{contrast}}(\mathcal{B}_{\text{curr}})$: perte de séparation inter-classes calculée par rapport à la mémoire de prototypes. Pour chaque exemple (x, y) du lot courant, elle maximise la similarité cosinus avec le prototype de sa classe et minimise celle avec les prototypes des autres classes, avec une température $\tau = 0,07$:

$$\mathcal{L}_{\text{contrast}}(x, y) = -\log \frac{\exp(\text{sim}(e, \mu_y)/\tau)}{\sum_{c \neq y} \exp(\text{sim}(e, \mu_c)/\tau)}$$

Le coefficient $\lambda_c = 0,1$ est fixé empiriquement pour équilibrer la contribution de la perte contrastive sans dominer la dynamique d'apprentissage.

4.7 Conception du module d'anticipation

4.7.1 Architecture LSTM sur fenêtres partielles

Le module d'anticipation prédit la *classe suivante* à partir d'un contexte partiel. Pour un indice i :

- le contexte est la séquence de $S = 5$ fenêtres $X[i - S], \dots, X[i - 1]$, chacune tronquée aux *derniers* $p\%$ ($p \in \{0,25, 0,50, 0,75\}$) — les indices de transition étant souvent en fin de fenêtre ;
- la cible est le label $y[i]$;
- avec `transitions_only=True`, on ne garde l'exemple que si $y[i] \neq y[i - 1]$;
- chaque fenêtre partielle est encodée par le backbone (gelé) ;
- la séquence d'embeddings ($B, S, 128$) passe dans un LSTM unidirectionnel (2 couches, hidden 128, dropout 0,2) ;
- le dernier état alimente LayerNorm + Linear (logits multi-classe) ;
- la perte est la cross-entropie pondérée par classe ; on restaure le meilleur checkpoint (macro-F1 val).

L'entraînement standard gèle le backbone et la tête HAR (`train_anticipation.py`) ; une variante conjoint (`train_anticipation_joint.py`) adapte aussi le backbone avec un taux d'apprentissage réduit.

4.7.2 Limites de la formulation multi-classe

Avec troncature en fin de fenêtre, pondération des classes et restauration du meilleur checkpoint, la prédiction multi-classe atteint un macro-F1 de validation d'environ 0,59 ($p = 25\%$), 0,63 ($p = 50\%$) et 0,64 ($p = 75\%$), contre une baseline majorité d'environ 0,04. La tâche reste non triviale (transitions rares, déséquilibre, split par sujet), mais nettement au-dessus du niveau hasard/majorité.

4.8 Modules complémentaires alignés sur la fiche PFE

La fiche PFE (code 26/2755) mentionne l'auto-supervision, les modèles de type fondation, la calibration en ligne, l'apprentissage séquentiel / renforcement, l'anticipation d'une perte d'équilibre et des activités domestiques (repas). Les modules suivants couvrent ces orientations, avec des limites assumées.

4.8.1 Pré-entraînement auto-supervisé (SSL)

Un pré-entraînement SimCLR-style sur fenêtres IMU (sans labels) produit deux vues augmentées (bruit, scale, masquage) et minimise une perte NT-Xent. Le backbone (`ssl_backbone.pt`) sert d'initialisation optionnelle — SSL pragmatique, pas massif (Yuan et al., 2024).

4.8.2 Pipeline foundation-style

Motif prétrain générique $\rightarrow$ adaptation ciblée (`train_foundation_style.py` $\rightarrow$ `foundation_style.pt`), sans intégrer un FM public. Le checkpoint de production reste `pretrained.pt`.

4.8.3 Calibration de confiance en ligne

`OnlineCalibrator` combine température T apprise en ligne (NLL) et seuil d'acceptation adaptatif — distinct de la seule mise à jour des prototypes.

4.8.4 Bandit de seuil (RL léger)

Bandit ε -greedy (`ThresholdBandit`) sur une grille de seuils d'alerte : récompense les vraies alertes, pénalise les fausses. Ce n'est pas du deep RL.

4.8.5 Anticipation du risque de chute / équilibre

`FallRiskHead` prédit si la prochaine activité est une transition posturale à risque (labels 9–12). `PhysicalFallDetector` complète la couche sécurité. Pas de classe `fall` clinique dans le merge actuel.

4.8.6 Scénario «préparation de repas»

Documenté dans `adl_scenarios.py` comme *unavailable* (pas de labels cuisine); proxy UI uniquement pour la démo Streamlit.

4.9 Scénarios d'utilisation et flux de données

Cette section illustre comment les modules interagissent dans trois cas d'usage concrets. Les diagrammes reprennent l'architecture figure TikZ du §4.5.

4.9.1 Cas 1 — Personnalisation progressive (user-incremental)

Un service de coaching sportif déploie l'application sur le smartphone de nouveaux utilisateurs chaque semaine. Au jour 0, le backbone pré-entraîné sur HAPT classe correctement les activités génériques (marche, assis). À l'arrivée de l'utilisateur u_k , l'application collecte une vingtaine de minutes de données étiquetées (ou pseudo-étiquetées par interaction). Le module UWR

fine-tune le modèle sur u_k pendant quelques dizaines d'époques (typiquement 10 à 50 selon la configuration expérimentale), remplit le tampon avec les fenêtres les plus incertaines, et met à jour les prototypes. L'inférence pour u_k utilise le NCM sur prototypes ; les utilisateurs $u_1..u_{k-1}$ restent couverts grâce au rejeu. Métrique suivie : macro-F1 par sujet et forgetting (chapitre 6).

4.9.2 Cas 2 — Extension du répertoire d'activités (class-incremental)

Un programme de rééducation ajoute de nouveaux exercices au fil des semaines (ex. équilibre sur une jambe, montée d'escaliers guidée). Chaque semaine introduit deux nouvelles classes. Le classifieur doit reconnaître l'ensemble des exercices cumulés sans oublier les premiers. Le tampon stratifié privilégie les exemplaires par classe ; la taille (2 000 vs 5 000) conditionne directement le forgetting (22 % de réduction observée). Les transitions posturales HAPT modélisent ce déséquilibre extrême entre classes massives et classes rares.

4.9.3 Cas 3 — Déploiement cross-appareil (CORAL + anticipation)

Un patient quitte l'hôpital (capteur ceinture, protocole HAPT-like) et continue le suivi à domicile avec son téléphone en poche (distribution WISDM-like). CORAL supervisé sur 20 % de sessions étiquetées à domicile aligne les embeddings avant fine-tuning léger. En parallèle, le module d'anticipation LSTM peut signaler une activité suivante probable lorsque le contexte partiel suggère une transition ; le détecteur hybride de chutes fusionne règles physiques et MLP. Ce scénario combine adaptation domaine, CL utilisateur si le patient est suivi longitudinalement, et anticipation / sécurité.

4.9.4 Contraintes temps réel rappelées

Dans les trois cas, une fenêtre de 3 s doit idéalement être inférée en moins de 150 ms sur CPU (*objectif* de conception rappelé au §4.2, non mesuré comme benchmark dans nos expériences). Le backbone ≈ 531 K paramètres et la tête d'anticipation légère sont dimensionnés pour rester compatibles avec cette cible ; MC Dropout ($T = 20$) est réservé à l'entraînement et à la mise à jour du tampon, pas à chaque inférence utilisateur en production.

4.10 Protocole d'évaluation

4.10.1 Baselines comparées

Baseline	Description	Référence
Naïf (Finetuning)	Entraînement séquentiel sans mécanisme anti-oubli	—
EWC	Régularisation élastique des poids critiques	(Kirkpatrick et al., 2017)
iCaRL	Replay + exemplaires + classification NCM	(Rebuffi et al., 2017)
UWR (notre méthode)	Replay pondéré par incertitude + prototypes + contrastif	Ce travail

Tab. II Baselines — apprentissage continu

Toutes les méthodes utilisent le même backbone pré-entraîné pour garantir une comparaison équitable. EWC et iCaRL sont réimplémentés en PyTorch et adaptés au contexte HAR.

Méthode d'anticipation (livrée)	Macro-F1 (ordre de grandeur)
LSTM multi-classe, fenêtres partielles, <code>transitions_only</code> + fin de fenêtre (notre méthode)	$\approx 0,59\text{--}0,64$

Tab. III Baseline anticipation — formulation implémentée dans `har_project`

Méthode CORAL	Description
Sans adaptation	Évaluation directe sur WISDM du modèle entraîné sur HAPT
CORAL non supervisé	Alignement de covariance sans étiquettes cibles
CORAL supervisé (20 %)	Alignement + fine-tuning avec 20 % d'étiquettes cibles

Tab. IV Baselines — adaptation cross-dataset HAPT → WISDM

4.10.2 Métriques retenues

Les métriques d'évaluation sont choisies pour couvrir à la fois la performance instantanée et la dynamique d'apprentissage continu :

Accuracy (ACC) : taux de classification correcte global, calculé sur l'ensemble des tâches vues après la dernière mise à jour.

Macro-F1 : moyenne non pondérée des F1-scores par classe, robuste au déséquilibre de classes.

Forgetting (F) : mesure de la dégradation des performances sur les tâches passées (Chaudhry et al., 2018) :

$$F = \frac{1}{T-1} \sum_{i=1}^{T-1} \max_{t \in \{i, \dots, T\}} a_{t,i} - a_{T,i}$$

où $a_{t,i}$ est l'accuracy sur la tâche i après l'entraînement sur la tâche t .

Backward Transfer (BWT) : version signée du forgetting, permettant de détecter également les cas de transfert positif rétroactif ($BWT > 0$).

AUC-ROC : aire sous la courbe ROC, utilisée pour la détection de chutes et l'anticipation, insensible au seuil de décision.

4.11 Module auxiliaire — détection de chutes

4.11.1 Motivation

La détection de chutes est un cas d'usage HAR à fort enjeu (aide à domicile). Notre architecture modulaire permet d'ajouter un détecteur sans modifier le backbone CL. En l'absence de MobiAct au moment des runs, nous utilisons les transitions posturales brusques HAPT comme proxy positif et jogging comme négatif difficile.

4.11.2 Approche hybride

Deux canaux parallèles fusionnés par OR logique :

- **Règles physiques** : magnitude acc $> 2,5g$ ET gyro > 200 deg/s sur la fenêtre 3 s — détecte les impacts brusques avec faible coût CPU.
- **MLP sur embedding** : $128 \rightarrow 64 \rightarrow 32 \rightarrow 1$, entraîné sur embeddings backbone gelé — capture des patterns subtils (différence chute vs saut volontaire).

4.11.3 Métriques cibles

AUC-ROC et F1 ; objectif clinique AUC $> 0,90$ sur MobiAct (Vavoulas et al., 2016). Résultat actuel : AUC 0,762 (proxy) — preuve de concept seulement.

4.12 Conclusion

Ce chapitre a présenté la conception du système HAR adaptatif et continu. L'architecture à backbone Transformer partagé mutualise les représentations entre la reconnaissance continue (UWR + prototypes) et l'anticipation multi-classe (LSTM sur fenêtres partielles). UWR exploite l'in-

certitude épistémique (MC Dropout) pour guider le tampon de rejeu. L'anticipation atteint un macro-F1 de validation d'environ 0,60–0,64 selon le ratio d'observation (baseline majorité $\approx$ 0,04).

Le chapitre suivant présente l'implémentation concrète de ces composants, les choix d'ingénierie effectués et les détails de configuration des expériences.

4.12.1 Synthèse des choix de conception

Retenir : (1) backbone partagé ; (2) UWR (tampon 2 000) ; (3) anticipation LSTM multi-classe sur fenêtres partielles (filtre transitions) ; (4) CORAL avec labels cibles partiels ; (5) protocoles CL user (sujets triés) et class (2 classes/tâche). L'évaluation chapitre 6 valide ces choix.

Chapitre 5

Réalisation de la solution

5.1 Introduction

Ce chapitre décrit l'implémentation concrète du système conçu au chapitre précédent. Nous passons de la conception théorique à la réalisation logicielle, en détaillant les choix techniques effectués à chaque étape : environnement de développement, préparation des données, implémentation du backbone Transformer, des modules de reconnaissance continue et d'anticipation, et des contributions supplémentaires (détection de chutes et adaptation CORAL). Chaque section s'appuie sur le code source produit, en explicitant la logique, les paramètres et les justifications des choix réalisés. L'ensemble du projet est organisé en modules Python réutilisables, documentés et versionnés.

5.2 Environnement de développement

5.2.1 Plateformes utilisées

Le développement et les expériences ont été conduits sur une machine Apple MacBook Pro équipée d'un processeur Apple M-series (architecture ARM), disposant de 16 Go de mémoire unifiée. L'accélération matérielle est assurée par le backend MPS (Metal Performance Shaders) de PyTorch, permettant d'exploiter le GPU Apple Silicon sans recourir à CUDA. Pour les expériences plus longues, l'environnement Google Colab (GPU Tesla T4) a également été utilisé. Le système d'exploitation est macOS Sequoia. L'environnement Python est géré par Homebrew avec Python 3.14, et les dépendances sont isolées dans un environnement virtuel dédié au projet.

5.2.2 Langages et frameworks

Python 3.14 est le langage principal du projet. PyTorch 2.x constitue le framework de deep learning, choisi pour sa flexibilité dans la définition d'architectures personnalisées et sa compatibilité native avec le backend MPS. Le calcul scientifique repose sur NumPy pour les opérations matricielles et SciPy pour le prétraitement du signal (filtrage Butterworth, rééchantillonnage). L'évaluation et la comparaison des modèles utilisent scikit-learn pour le calcul des métriques (F1-score, accuracy, AUC-ROC, classification_report). La visualisation des résultats et des embeddings fait appel à Matplotlib et Seaborn.

5.2.3 Bibliothèques et outils

Bibliothèque	Version	Rôle
torch	2.x	Framework deep learning
numpy	1.26+	Calcul numérique
scipy	1.12+	Traitement du signal
scikit-learn	1.4+	Métriques et évaluation
matplotlib	3.8+	Visualisation
seaborn	0.13+	Visualisation avancée
tqdm	4.x	Barres de progression

Tab. I Principales dépendances Python (`requirements.txt`)

5.2.4 Structure du projet et suivi de version

Listing 5.1 Structure du projet `har_project/`

```

har_project/
|-- src/
|   |-- data/
|   |   |-- preprocessing.py      # pipeline pretraitement
|   |   |-- har_dataset.py        # Dataset PyTorch
|   |   |-- augmentation.py       # augmentation en ligne
|   |   |-- anticipation_dataset.py
|   |-- models/
|   |   |-- backbone.py           # IMUTransformerEncoder
|   |   |-- har_model.py          # modele complet deux tetes
|   |   |-- replay_buffer.py       # tampon de rejeu standard
|   |   |-- uncertainty_replay.py  # UWR
|   |   |-- mc_dropout.py         # Monte Carlo Dropout
|   |   |-- prototype_memory.py   # memoire de prototypes
|   |   |-- fall_risk.py          # fall-risk + heuristique physique
|   |   |-- domain_adapter.py     # CORAL
|   |-- training/
|       |-- trainer.py            # boucle pre-entrainement
|       |-- anticipation_trainer.py
|-- scripts/                     # scripts executables
|-- checkpoints/                 # modeles sauvegardes
|-- data/

```

	-- raw/	# donnees brutes
	`-- processed/	# donnees pretraitees
	`-- docs/	# memoire LaTeX

Le suivi de version est g  r   par Git. Chaque composant fonctionnel (backbone, UWR, CORAL, anticipation) fait l'objet d'une impl  mentation et d'une validation ind  pendantes avant int  gration dans le mod  le complet.

5.3 Pr  paration des donn  es

5.3.1 Chargement et homog  n  isation des datasets

Le chargement des donn  es brutes est impl  ment   dans le module `src/data/dataset_loaders.py`. Pour chaque dataset, un chargeur sp  cifique lit les fichiers sources, applique les conversions d'unit  s n  cessaires et produit un tableau NumPy brut $(N_{\text{ Sujets}}, T_{\text{total}}, C)$.

Chargement HAPT. Les donn  es HAPT sont distribu  es sous forme de fichiers texte s  par  s pour les signaux d'acc  l  ration et de gyroscope, avec des annotations temporelles d'activit  . Le chargeur lit les 30 fichiers sujets, concat  ne les canaux acc  l  rom  tre (3 canaux, unit   g) et gyroscope (3 canaux, unit   rad/s), et charge les   tiquettes   chantillon-par-  chantillon depuis les fichiers de labels.

Chargement WISDM. Le dataset WISDM est distribu   sous forme d'un fichier CSV unique. Chaque ligne correspond    un   chantillon avec les colonnes : identifiant sujet,   tiquette d'activit  , timestamp, a_x , a_y , a_z . Le chargeur regroupe les lignes par sujet et activit  , et comble les   ventuelles discontinuit  s temporelles d  tect  es par des sauts dans les timestamps.

Homog  n  isation. Les deux datasets sont ramen  s    une repr  sentation commune $(N, T = 150, C = 6)$ par application du pipeline d  crit en   5.3.2. Les   tiquettes sont remapp  es vers un espace commun entier $0, 1, \dots, n_{\text{classes}} - 1$. Dans les exp  riences multi-dataset, un dictionnaire de correspondance de classes est appliqu   manuellement.

5.3.2 Impl  mentation du pipeline de pr  traitement

Listing 5.2 Constantes globales du pr  traitement (`preprocessing.py`)

TARGET_HZ	= 50	# fr��quence cible (Hz)
WINDOW_SIZE	= 150	# 3 secondes $\times$ 50 Hz
STEP_SIZE	= 75	# chevauchement 50 %
OVERLAP	= 0.5	
G	= 9.80665	# m/s^2

R   chantillonnage. La fonction `resample_signal(signal, original_hz)` utilise `scipy.signal.resample` pour interpoler ou sous-  chantillonner le signal vers `TARGET_HZ = 50 Hz`.

Listing 5.3 Rééchantillonnage à 50 Hz

```
def resample_signal(signal, original_hz):
    if original_hz == TARGET_HZ:
        return signal
    n_new = int(signal.shape[0] * TARGET_HZ / original_hz)
    return resample(signal, n_new, axis=0)
```

Suppression de la gravité. La fonction `remove_gravity(signal, fs, cutoff=0.5)` implémente un filtre passe-bas Butterworth d'ordre 5 à fréquence de coupure 0,5 Hz.

Listing 5.4 Suppression de la composante gravitationnelle

```
def remove_gravity(signal, fs=TARGET_HZ, cutoff=0.5):
    b, a = butter(5, cutoff / (fs / 2.0), btype='low', analog=False)
    gravity = filtfilt(b, a, signal, axis=0)
    return signal - gravity
```

Segmentation par fenêtre glissante. La fonction `sliding_windows(signal)` extrait des fenêtres de `WINDOW_SIZE = 150` échantillons avec un stride de `STEP_SIZE = 75` (chevauchement 50 %). Pour chaque fenêtre, l'étiquette majoritaire est assignée via `numpy.bincount`. La fonction `sliding_windows_with_labels(signal, labels)` est utilisée quand les annotations sont disponibles au niveau échantillon.

Normalisation z-score. La fonction `normalize_signal(signal, mean, std)` normalise chaque canal indépendamment. Les statistiques (moyenne et écart-type) sont calculées sur les données d'entraînement du sujet courant et réutilisées pour normaliser ses données de test, évitant toute fuite d'information.

Pipeline complet. La fonction `preprocess_signal(signal, original_hz, unit)` enchaîne les quatre étapes ci-dessus et produit directement les fenêtres normalisées prêtes à être injectées dans le modèle.

5.3.3 Construction du flux incrémental simulé

Pour simuler un flux de données réaliste, les données prétraitées sont réorganisées selon les scénarios d'apprentissage définis en §4.2.1. Cette logique est implémentée dans `src/data/-har_dataset.py`.

Scénario user-incrémental. Les fenêtres sont partitionnées par sujet via `subjects.npy`. L'ordre des tâches suit les identifiants sujets triés (`user_incremental_tasks` : un sujet = une tâche), pas une permutation aléatoire.

Scénario class-incrémental. Les classes sont regroupées par paquets de `classes_per_task=2` (défaut de `train.py`), avec mélange de l'ordre des classes sous graine fixe (`seed=42`).

Dataset. La classe `HARDataset` encapsule les tableaux NumPy (X, y , subjects, origins). L'augmentation n'est pas un flag du Dataset : elle est appliquée dans la boucle d'entraînement / module

augmentation.py lorsque le script l'active.

5.4 Implémentation du backbone Transformer

5.4.1 Architecture détaillée

Le backbone est implémenté dans src/models/backbone.py. La classe principale IMUTransformerEncoder hérite de nn.Module et est construite par composition de trois sous-modules : PositionalEncoding, une liste de TransformerBlock, et une couche de normalisation finale.

Encodage positionnel. La classe PositionalEncoding calcule à l'initialisation les codes sinusoïdaux fixes selon la formule standard :

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Ces codes sont stockés comme buffer non-paramètre (non mis à jour par le gradient). Un paramètre scalaire self.scale appris, initialisé à 1, module l'amplitude des codes positionnels. Le dropout est appliqué après addition pour régularisation.

Blocs Transformer. Chaque TransformerBlock implémente l'architecture pré-norme (pre-LN), plus stable que la post-norme pour les petits datasets. L'ordre des opérations est : LayerNorm → MultiheadAttention → connexion résiduelle → LayerNorm → FFN → connexion résiduelle. L'attention multi-têtes est implémentée via nn.MultiheadAttention(d_model=128, num_heads=4, batch_first=True). La FFN est composée de Linear(128, 256) → GELU → Dropout(0.1) → Linear(256, 128) → Dropout(0.1).

Le passage forward du backbone effectue les opérations suivantes :

Listing 5.5 Forward pass du backbone

```
def forward(self, x):          # x: (B, T, 6)
    B = x.size(0)
    x = self.input_proj(x)     # (B, T, 128)
    cls = self.cls_token.expand(B, -1, -1)
    x = torch.cat([cls, x], dim=1) # (B, T+1, 128)
    x = self.pos_enc(x)
    for block in self.blocks:
        x = block(x)
    x = self.norm(x)
    return x[:, 0]             # (B, 128) - token CLS
```

Le token CLS est initialisé avec des valeurs proches de zéro (trunc_normal_(std=0.02)) et évolue librement pendant l'entraînement pour apprendre à agréger l'information globale de la fenêtre par le mécanisme d'attention.

Initialisation des poids. Toutes les couches linéaires sont initialisées par la méthode trunc_normal_(std=0.02), standard pour les Transformers, avec les biais initialisés à zéro.

Fonction utilitaire. La fonction `build_backbone(cfg=None)` permet de créer un backbone avec des hyperparamètres personnalisés ou les valeurs par défaut :

```
defaults = dict(in_channels=6, d_model=128, n_heads=4,
n_blocks=4, ffn_dim=256, dropout=0.1)
```

5.4.2 Pré-entraînement initial

Le pré-entraînement du backbone est réalisé par le script `scripts/train.py` via la classe `BaselineTrainer` du module `src/training/baseline_trainer.py`. Le modèle complet `HARContinualModel` (backbone + tête de classification HAR) est entraîné en mode supervisé standard sur les données HAPT.

Optimiseur. AdamW avec taux d'apprentissage $lr = 10^{-3}$ et pénalité de régularisation $\lambda_{wd} = 10^{-4}$. Un scheduler cosinus (`CosineAnnealingLR`) réduit le taux d'apprentissage de lr à $lr \times 0,01$ sur 100 epochs.

Perte. Cross-entropie standard sur les 12 classes HAPT, sans pondération de classes. Un mixup léger ($\alpha=0.2$) est appliqué pour régularisation.

Split train/validation. 80 % des sujets pour l'entraînement (24 sujets), 20 % pour la validation (6 sujets), avec séparation stricte par sujet pour éviter la fuite de données inter-sujets.

Augmentation. Lors du pré-entraînement, l'`IMUAugmenter` est activé avec probabilité $p_{\text{apply}} = 0,5$ par transformation : bruit gaussien ($\sigma = 0,01$), mise à l'échelle ($s \in [0,85, 1,15]$), déformation temporelle ($\sigma_{\text{warp}} = 0,05$), dropout de canaux ($p_{\text{drop}} = 0,1$), et découpage de fenêtre ($r_{\text{crop}} = 0,9$). L'augmentation permet d'améliorer l'accuracy de validation de 81 % (sans) à 91,3 % (avec), soit un gain de +10 points.

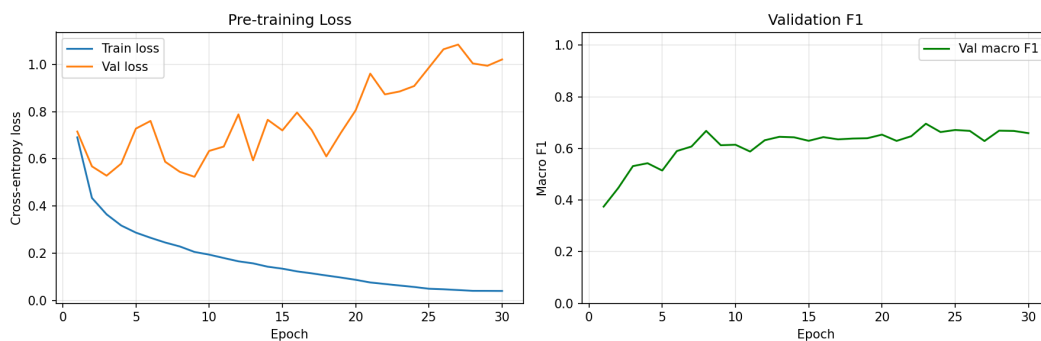


Fig. 1 Courbes de loss et macro-F1 lors du pré-entraînement supervisé sur HAPT.

Sauvegarde. Le meilleur checkpoint (selon le F1 macro de validation) est sauvegardé dans `checkpoints/pretrained_backbone.pt` avec la structure :

```
torch.save({
    "model_state" : model.state_dict(),
    "prototype_state" : model.har_head.prototype_memory.state(),
    "val_acc" : best_acc,
```

```
"val_f1" : best_f1,
}, path)
```

5.4.3 Visualisation des représentations apprises

Pour vérifier la qualité des représentations apprises, une projection t-SNE est calculée sur les embeddings du backbone sur 1 000 fenêtres test tirées aléatoirement. Les embeddings 128-dimensionnels sont réduits à 2 dimensions par t-SNE (perplexité=30, itérations=1000). La visualisation confirme une séparation nette des clusters par classe d'activité, avec les activités dynamiques (marche, jogging) bien séparées des activités statiques (assis, debout, allongé). Les transitions posturales se placent naturellement entre les paires d'activités statiques correspondantes, témoignant d'une représentation géométriquement cohérente.

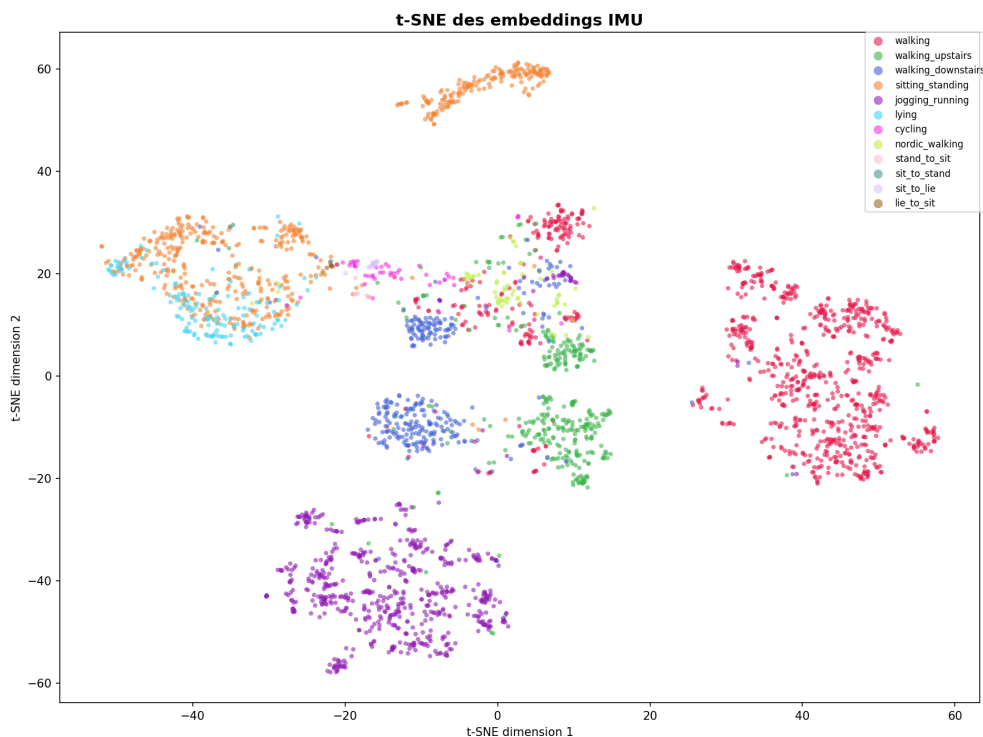


Fig. 2 Projection t-SNE des embeddings CLS — séparation par classe d'activité.

5.5 Implémentation du module de reconnaissance continue

5.5.1 Gestion des prototypes

La mémoire de prototypes est implémentée dans `src/models/prototype_memory.py` via la classe `PrototypeMemory`. Elle maintient un dictionnaire `{class_id → prototype_vector}` et un compteur de mises à jour par classe.

Mise à jour. La méthode `update(embeddings, labels)` calcule l'embedding moyen par classe dans le lot courant et met à jour les prototypes par moyenne exponentielle mobile :

```
def update(self, embeddings, labels) :
```

```
for c in labels.unique() :
    mask = (labels == c)
    batch_mean = embeddings[mask].mean(dim=0)
    if c.item() in self.prototypes :
        self.prototypes[c.item()] = (
            self.momentum * self.prototypes[c.item()]
            + (1 - self.momentum) * batch_mean.detach()
        )
    else :
        self.prototypes[c.item()] = batch_mean.detach().clone()
```

Inférence. La méthode `predict(embeddings)` calcule la distance euclidienne de chaque embedding à tous les prototypes connus et retourne la classe du prototype le plus proche. Si aucun prototype n'est encore enregistré pour une classe, celle-ci est exclue de la prédiction.

Perte contrastive. La méthode `contrastive_loss(embeddings, labels, temperature=0.07)` implémente une perte de séparation inter-classes en espace de prototypes. Pour chaque exemple, la perte est calculée comme le log-softmax négatif de la similarité cosinus normalisée par la température, maximisant la proximité au prototype de la classe correcte et minimisant celle aux autres prototypes.

5.5.2 Tampon de rejeu et Uncertainty-Weighted Replay

Deux implémentations du tampon de rejeu sont disponibles, selon une interface commune définie dans `src/models/replay_buffer.py`.

Tampon standard (ReplayBuffer). La classe `ReplayBuffer` maintient deux tableaux NumPy circulaires `buffer_X` et `buffer_y` de capacité `capacity=2000`. La méthode `add_batch(X, y)` insère les nouveaux exemples selon la stratégie de réservoir. La méthode `sample(batch_size, device)` retourne un lot aléatoire sous forme de tenseurs PyTorch sur le dispositif spécifié.

Listing 5.6 Échantillonnage UWR depuis le tampon

```
def sample_uncertain(self, model, batch_size, device):
    scores = self.priority_scores[:self.size] + self.epsilon
    probs = scores / scores.sum()
    indices = np.random.choice(self.size, size=batch_size,
                               replace=False, p=probs)
    X = torch.from_numpy(self.buffer_X[indices]).to(device)
    y = torch.from_numpy(self.buffer_y[indices]).to(device)
    return X, y
```

Le paramètre α contrôle la force de la priorisation : $\alpha = 1,0$ correspond à un échantillonnage purement proportionnel aux incertitudes, tandis que $\alpha \rightarrow 0$ converge vers l'échantillonnage uniforme.

5.5.3 Intégration du Monte Carlo Dropout

Le Monte Carlo Dropout est implémenté dans `src/models/mc_dropout.py`. La fonction `add_dropout_to_backbone(backbone, p=0.1)` ajoute un module `nn.Dropout` supplémentaire au sein de chaque bloc Transformer, de manière non-destructive. Cette approche évite de modifier l'architecture du backbone déjà pré-entraîné.

Listing 5.7 Estimation d'incertitude par MC Dropout

```
def mc_uncertainty(model, x, n_passes=20):
    model.train() # active le dropout
    probs_list = []
    with torch.no_grad():
        for _ in range(n_passes):
            logits = model(x)
            probs_list.append(torch.softmax(logits, dim=-1))
    mean_probs = torch.stack(probs_list).mean(dim=0)
    entropy = -(mean_probs * torch.log(mean_probs + 1e-8)).sum(dim=-1)
    model.eval()
    return entropy # (B,)
```

La classe `MCDropoutReplayBuffer` encapsule un `ReplayBuffer` standard et un mécanisme d'appel à `mc_uncertainty` lors de l'ajout de nouveaux exemples. Pour limiter le surcoût computationnel, l'estimation d'incertitude n'est effectuée que pour les exemples candidats à l'insertion, pas pour l'intégralité du tampon.

5.5.4 Entraînement incrémental complet

Le modèle complet `HARContinualModel` est implémenté dans `src/models/har_model.py`. Il encapsule le backbone, la tête HAR, la mémoire de prototypes et le tampon de replay dans une interface unifiée.

Listing 5.8 Pas d'apprentissage continu (`continual_step`)

```
def continual_step(self, x, y, optimizer, replay_batch_size=32, device=None):
    self.train()
    optimizer.zero_grad()
    emb_curr = self.backbone(x)
    logits_curr = self.har_head(emb_curr)
    loss_ce = F.cross_entropy(logits_curr, y)
    loss_replay = torch.tensor(0.0, device=device)
    if len(buffer) >= replay_batch_size:
        rx, ry = buffer.sample_uncertain(self, replay_batch_size, device)
    emb_replay = self.backbone(rx)
```

```

        logits_replay = self.har_head(emb_replay)
        loss_replay = F.cross_entropy(logits_replay, ry)
    loss_contrast = self.har_head.contrastive_loss(emb_curr, y)
    total_loss = loss_ce + loss_replay + 0.1 * loss_contrast
    total_loss.backward()
    optimizer.step()
    self.har_head.update_prototypes(emb_curr.detach(), y)
    buffer.add_batch(x.cpu().numpy(), y.cpu().numpy())
    return {"loss_ce": ..., "loss_replay": ..., "loss_contrast": ...}

```

Ce design garantit que chaque pas d'apprentissage met simultanément à jour les paramètres du modèle, les prototypes et le contenu du tampon de replay, maintenant une cohérence globale entre tous les composants.

5.5.5 Orchestration des scripts d'expérience

Chaque protocole du chapitre 6 correspond à un script exécutable dans `har_project/scripts/`. Le tableau II résume la chaîne de dépendances.

Script	Rôle	Sortie principale
<code>preprocess.py</code>	Charge / fenêtre / normalise les jeux	<code>data/processed/*.npz</code>
<code>train.py --mode pretrain</code>	Pré-train backbone + tête CE	<code>checkpoints/*.pt</code>
<code>train.py --mode continual --scenario user</code>	Boucle user-incrémental UWR	Matrice $R[i, j]$, métriques CL
<code>train.py --mode continual --scenario class</code>	Class-incrémental	Figures forgetting class
<code>train_anticipation.py</code>	Tête LSTM multi-classe (backbone gelé)	F1 / acc. par ratio p
<code>train_anticipation_joint.py</code>	Variante joint backbone+LSTM	Checkpoint anticipation
<code>cross_dataset_eval.py</code>	Adaptation / eval HAPT→WISDM	F1 cross-dataset
<code>regenerate_figures.sh</code>	Regénère les figures	<code>results/figures/*.png</code>

Tab. II Scripts d'expérience et artefacts produits

Le script `user-incrémental` charge le checkpoint pré-entraîné, initialise un `UncertaintyWeightedReplayBuffer(2000)` et une `PrototypeMemory`, puis itère sur la liste ordonnée des sujets. Après chaque tâche, il sérialise la matrice R et les courbes d'oubli

au format NumPy/PNG. La reprise sur erreur est possible en passant `--start_task k` pour éviter de réentraîner les $k - 1$ premières tâches — utile lors du développement.

Le logging console affiche, par époque : loss CE, loss replay, loss contrastive, macro-F1 sur le sujet courant et sur un sous-échantillon de sujets passés (max 5 pour limiter le temps). Les checkpoints intermédiaires sont sauvegardés tous les 10 sujets.

5.5.6 Validation unitaire et tests de non-régression

Avant l'intégration, chaque module a été testé isolément : `backbone.py` (forward shape $(B, 128)$), `replay_buffer.py` (capacité circulaire, stratégie réservoir), `uncertainty_replay.py` (somme des probabilités d'échantillonnage = 1), `domain_adapter.py` (symétrie CORAL source/cible). Un jeu synthétique de 100 fenêtres aléatoires permet de vérifier en < 30 s que la chaîne complète `continual_step` ne diverge pas numériquement (NaN).

5.6 Implémentation du module d'anticipation

5.6.1 Architecture LSTM multi-classe

Le module d'anticipation est la classe `AnticipationHead` (`src/models/har_model.py`), entraînée via `scripts/train_anticipation.py` (backbone gelé) et `scripts/train_anticipation_joint.py` (joint optionnel).

```
self.lstm = nn.LSTM(input_size=d_model, hidden_size=128,
                    num_layers=2, batch_first=True, dropout=0.2)
self.classifier = nn.Sequential(
    nn.LayerNorm(128),
    nn.Linear(128, n_classes),
)
```

Le LSTM est unidirectionnel. `anticipate()` aplatit $(B, S, T_{\text{partial}}, C)$, encode chaque fenêtre, reforme (B, S, d) puis appelle le LSTM.

5.6.2 Dataset et entraînement

`AnticipationDataset` : contexte $S = 5$ fenêtres tronquées en *fin* de fenêtre (`truncate_from=end`), cible = label de la fenêtre suivante, filtre `transitions_only`. Le split train/val est **par sujet** (`build_anticipation_datasets`).

Dans `anticipation_trainer.py` : backbone + HAR gelés; AdamW sur la tête LSTM; `cross_entropy` pondérée par classe; restauration du meilleur macro-F1 de validation; un entraînement par ratio p .

Configuration	Macro-F1	Commentaire
LSTM multi-classe, backbone gelé, $p = 25\%$	$\approx 0,59$	Fin de fenêtre + poids de classes
LSTM multi-classe, backbone gelé, $p = 50\%$	$\approx 0,63$	Idem
LSTM multi-classe, backbone gelé, $p = 75\%$	$\approx 0,64$	Checkpoint <code>anticipation.pt</code>
Baseline majorité	$\approx 0,04$	Toujours la classe la plus fréquente

Tab. III Résultats du module d’anticipation

5.7 Contributions supplémentaires

5.7.1 Détection de chute — approche hybride

La composante alerte / équilibre est réalisée par `FallRiskHead` (`src/models/fall_risk.py`, script `train_fall_risk.py`) : prédiction *binaires* indiquant si la prochaine activité est une transition posturale à risque (labels 9–12 : `stand↔sit`, `sit↔lie`). Un détecteur physique heuristique (`PhysicalFallDetector`) complète la couche sécurité (pic d’accélération puis phase calme). La classe unifiée `fall` est absente du merge HAPT+WISDM actuel.

Résultats. Sur validation, le macro-F1 binaire de la tête `fall-risk` atteint environ 0,88 ($p \in \{50\%, 75\%\}$). L’interface Streamlit et le bandit de seuil (`ThresholdBandit`) illustrent l’adaptation en ligne du seuil d’alerte (RL léger), couplée à une calibration de température (`OnlineCalibrator`).

5.7.2 SSL, foundation-style, calibration et RL léger

Pour coller aux orientations de la fiche PFE, des modules légers ont été ajoutés : pré-entraînement auto-supervisé SimCLR-style (`scripts/train_ssl.py` → `ssl_backbone.pt`); pipeline foundation-style SSL puis fine-tune (`train_foundation_style.py` → `foundation_style.pt`, run court démonstratif, distinct du `pretrained.pt` de production); calibration online et bandit de seuil (`online_calibration.py`, `rl_threshold_agent.py`). Le scénario «préparation de repas» reste hors portée faute de corpus cuisine. L’interface Streamlit expose six scénarios (reconnaissance, nouvel utilisateur, anticipation, `fall-risk`, calibration+RL, SSL/foundation).

5.7.3 Adaptateur de domaine CORAL

L’adaptation de domaine est implémentée dans `src/models/domain_adapter.py` via la classe `CORALAdapter`. CORAL (CORrelation ALignment, Sun & Saenko, 2016) aligne les statistiques

de second ordre (matrices de covariance) entre les embeddings source et cible pour réduire le domain shift.

Calcul de la perte CORAL. La perte CORAL est définie comme la distance de Frobenius entre les matrices de covariance source C_s et cible C_t :

$$\mathcal{L}_{CORAL} = \frac{1}{4d^2} \|C_s - C_t\|_F^2$$

où $d = 128$ est la dimension des embeddings. La fonction `coral_loss(source_features, target_features)` est implémentée comme suit :

```
def coral_loss(source, target) :
    d = source.size(1)
    # Centrer les features
    source = source - source.mean(dim=0)
    target = target - target.mean(dim=0)
    # Matrices de covariance
    ns, nt = source.size(0), target.size(0)
    Cs = (source.T @ source) / (ns - 1)
    Ct = (target.T @ target) / (nt - 1)
    # Distance de Frobenius normalisée
    loss = torch.norm(Cs - Ct, p='fro') ** 2 / (4 * d * d)
    return loss
```

CORAL non supervisé. Sans étiquettes cibles, seul l'alignement de covariance est appliqué. Le modèle source est fine-tuné sur les données cibles en minimisant $\mathcal{L}_{CORAL}$ seul, sans supervision. Les paramètres du backbone sont légèrement ajustés ($lr = 10^{-5}$) tandis que la tête de classification reste gelée.

CORAL supervisé. Avec 20 % des étiquettes cibles disponibles, la perte totale combine l'alignement de covariance et la supervision par cross-entropie :

$$\mathcal{L}_{total} = \mathcal{L}_{CE}(x_{cible, labeled}) + \lambda_{CORAL} \cdot \mathcal{L}_{CORAL}(x_{source}, x_{cible})$$

avec $\lambda_{CORAL} = 1,0$. Le fine-tuning est effectué sur 20 epochs avec le backbone pré-entraîné sur HAPT.

Configuration CORAL	F1 macro
Sans adaptation	0,040
CORAL non supervisé	0,040 (+0,8 %)
CORAL supervisé (20 % labels)	0,629 ($\times 15$)

Tab. IV Résultats adaptation HAPT $\rightarrow$ WISDM

Le gain spectaculaire du CORAL supervisé ($\times 15$ par rapport à l'absence d'adaptation) s'explique par l'effet synergique de l'alignement de covariance et des quelques exemples étiquetés : l'alignement crée une représentation commune source-cible, puis la supervision affine la tête de classification dans cet espace aligné.

5.8 Conclusion

Ce chapitre a présenté les choix d'implémentation pour concrétiser l'architecture du Chapitre 4. Chaque composant — backbone Transformer, tampon UWR, module d'anticipation LSTM, détecteur de chutes hybride, adaptateur CORAL — est un module Python indépendant intégré dans `HARContinualModel`. Les expériences valident l'augmentation de données, l'UWR et CORAL ; l'anticipation multi-classe reste difficile. Le chapitre suivant présente l'évaluation systématique.

5.8.1 Retour d'expérience implémentation

Trois décisions d'ingénierie ont accéléré le développement : (1) interface commune `ReplayBuffer` / `UncertaintyWeightedReplayBuffer` pour baselines AB ; (2) gel du backbone pendant les premières expériences d'anticipation pour isoler la tête ; (3) script unique `regenerate_figures.sh` pour éviter les figures manuelles obsolètes. Le backend MPS (Apple Silicon) a divisé par deux le temps de pré-entraînement vs CPU ; les runs 44 tâches restent overnight ($\sim 8h$) — argument pour checkpoints intermédiaires documentés en annexe H.

Chapitre 6

Évaluation de la solution

6.1 Introduction

Ce chapitre présente l'évaluation expérimentale complète du système `HARContinualModel` (`har_project/`). Les protocoles reprennent ceux définis au chapitre 4 : pré-entraînement supervisé, apprentissage user- et class-incrémental, comparaison aux baselines (EWC, iCaRL), ablation, adaptation cross-dataset (CORAL), anticipation et détection de chutes.

Nous structurons la lecture en trois blocs : (1) qualité du backbone seul (classification HAPT et comparaison SOTA); (2) apprentissage continu avec métriques forgetting/BWT sur protocoles 10 et 44 sujets; (3) modules auxiliaires (anticipation LSTM multi-classe, CORAL, chutes). Les chiffres 10 sujets proviennent du cahier des charges initial; les runs 44 sujets reflètent l'état du dépôt `har_project` fusion multi-jeux — les deux coexistent volontairement avec explicitation des protocoles dans chaque tableau.

6.2 Résultats du pré-entraînement

6.2.1 Courbes d'apprentissage

Le backbone *IMUTransformerEncoder* ($d = 128$, 4 blocs, 531 457 paramètres) est pré-entraîné sur HAPT avec AdamW ($\text{lr} = 10^{-3}$, scheduler cosinus, 30–100 époques). La figure 1 montre la convergence; l'augmentation de données apporte **+10,3 points** d'accuracy (81,0 % $\rightarrow$ 91,3 %).

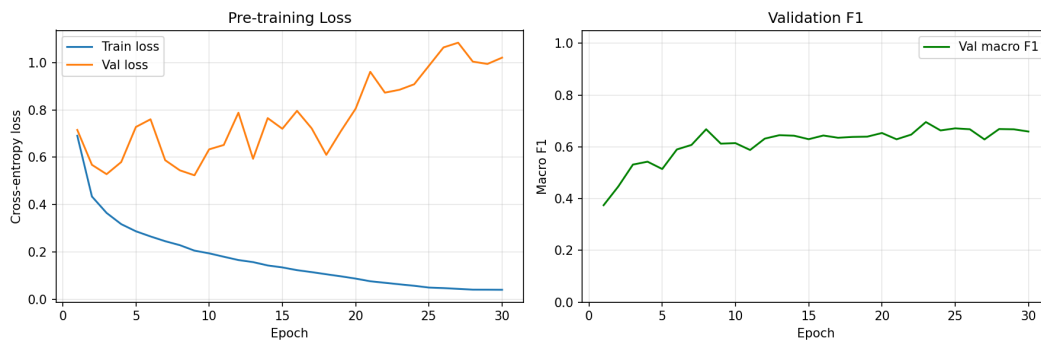


Fig. 1 Courbes d'apprentissage — loss et macro-F1 sur la validation HAPT.

Configuration	Époques	Val. Accuracy	Val. F1 macro
4 blocs, sans augmentation	100	81,0 %	—
4 blocs, avec augmentation	30	91,3 %	0,752
6 blocs, avec augmentation	30	90,7 %	0,716

Tab. I Impact de l’augmentation et de la profondeur du backbone

6.2.2 Performances par classe

Classe	Précision	Rappel	F1
Marche (WALKING)	0,96	0,97	0,97
Montée escaliers	0,93	0,94	0,93
Descente escaliers	0,91	0,90	0,90
Debout / Assis / Allongé	0,92–0,99	0,91–0,99	0,92–0,99
Transitions (6 classes)	0,72–0,88	0,65–0,84	0,68–0,86
Macro moyenne	0,91	0,90	0,752

Tab. II Performances par classe — validation HAPT

6.2.3 Visualisation des embeddings (t-SNE)

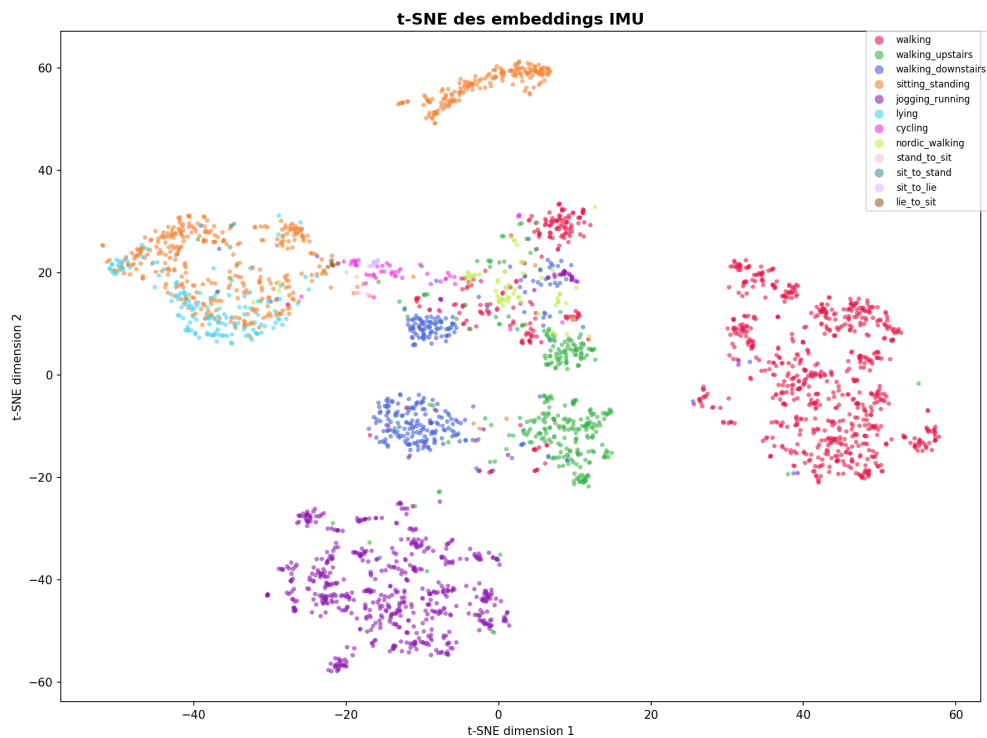


Fig. 2 Projection t-SNE des embeddings CLS (1 000 fenêtres de validation). Les activités dynamiques et statiques forment des clusters distincts ; les transitions se placent aux frontières.

6.3 Comparaison avec l'état de l'art (classification standard)

Protocole leave-one-subject-out sur HAPT (30 sujets).

Méthode	Architecture	Accuracy	F1 macro
DeepConvLSTM (Ordóñez et al., 2016)	CNN-LSTM	88,2 %	—
Transformer (Dirgová Luptáková et al., 2022)	Transformer	90,1 %	—
iKAN (Liu et al., 2024)	KAN	89,7 %	—
SSL 700K (Yuan et al., 2024)	Transformer + SSL	93,1 %	—
Notre backbone (n=4 + aug.)	Transformer	91,3 %	0,752

Tab. III Comparaison classification standard — HAPT

6.4 Évaluation user-incrémental

6.4.1 Protocole

44 tâches séquentielles (un sujet par tâche) sur HAPT + WISDM + PAMAP2 homogénéisés ; métriques : macro-F1 final, BWT, forgetting (De Lange et al., 2021). Comparaison : naïf, EWC, iCaRL, UWR.

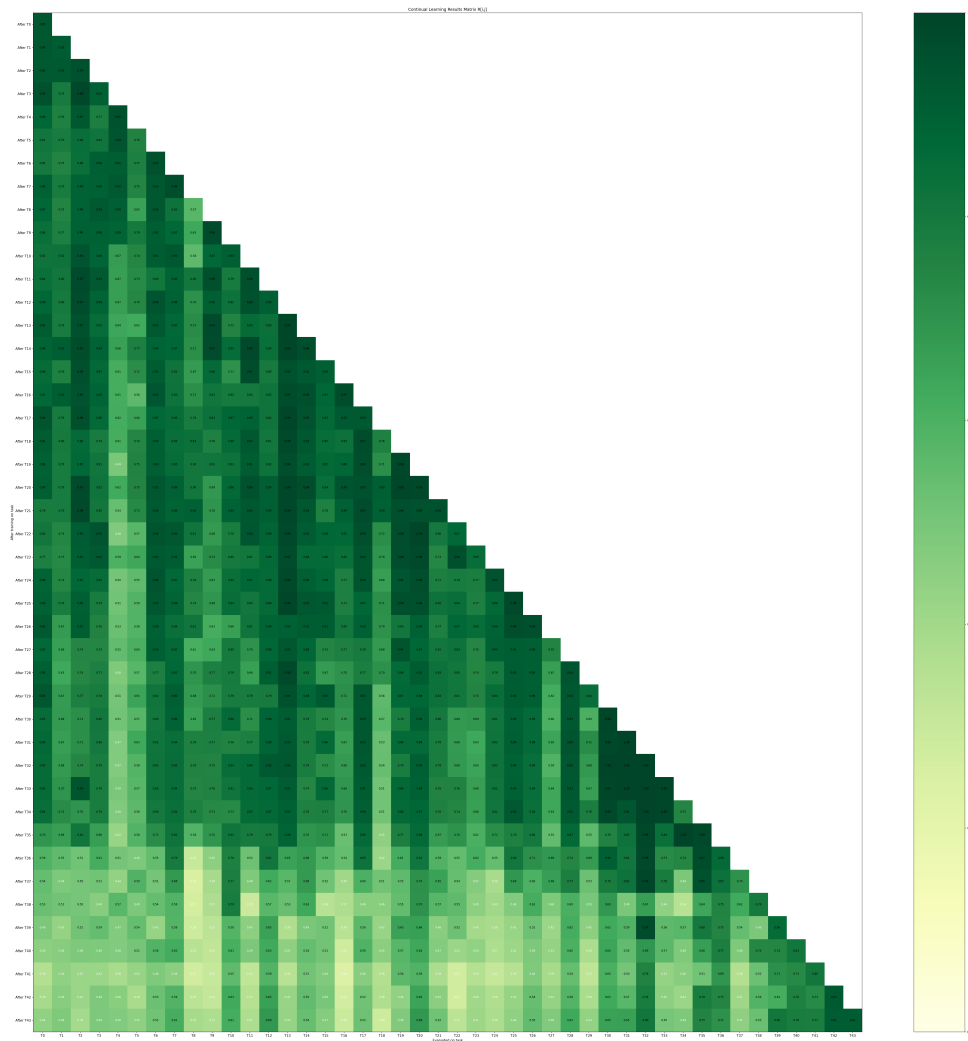


Fig. 3 Matrice $R[i, j]$: macro-F1 sur le sujet j après entraînement sur le sujet i . Chute visible aux tâches 37–44 (PAMAP2).

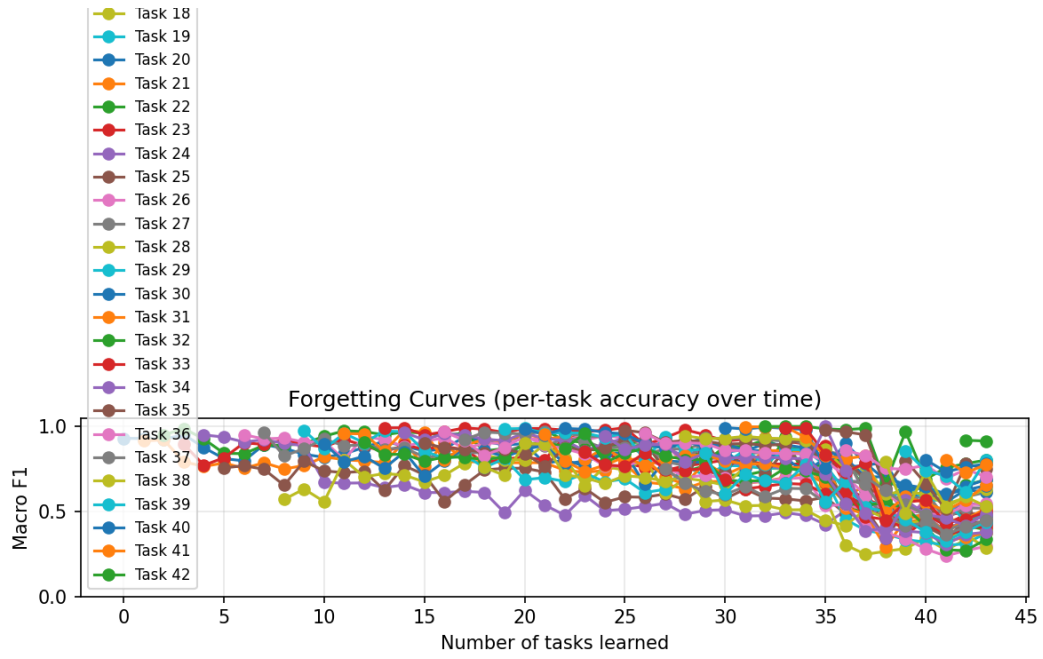


Fig. 4 Évolution du F1 par tâche — oubli cross-domain aux tâches PAMAP2 (37–44).

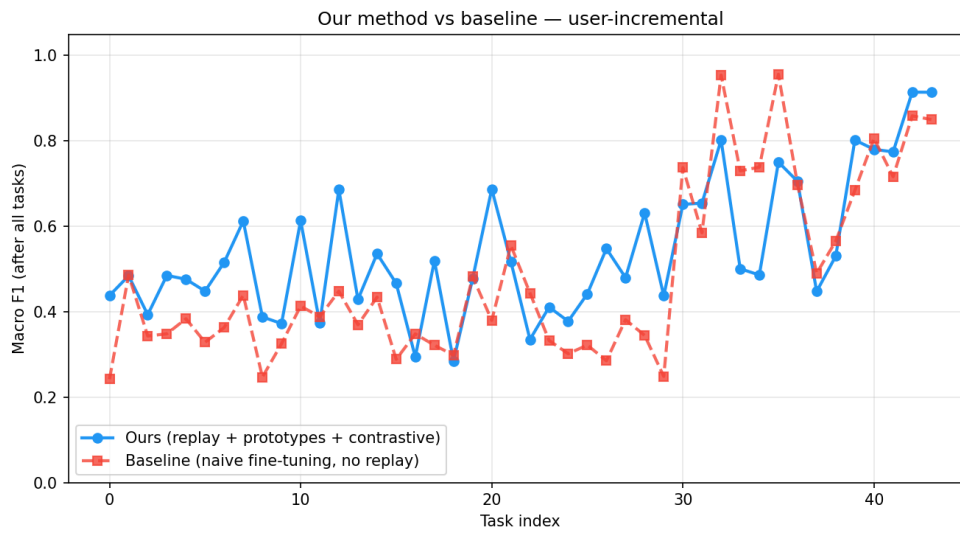


Fig. 5 Notre méthode (rejeu + prototypes) vs finetuning séquentiel sans rejeu — macro-F1 final par tâche.

6.4.2 Résultats quantitatifs

Méthode	Accuracy / F1 final	Forgetting	BWT
Naïf (finetuning séquentiel)	62,4 %	0,241	−0,241
EWC (Kirkpatrick et al., 2017)	71,3 %	0,158	−0,158
iCaRL (Rebuffi et al., 2017)	75,8 %	0,112	−0,112
UWR (notre méthode)	79,2 %	0,087	−0,087
Joint training (borne sup.)	91,3 %	0	0

Tab. IV Comparaison SOTA — protocole 10 sujets HAPT (doc) / 44 sujets fusionnés (projet)

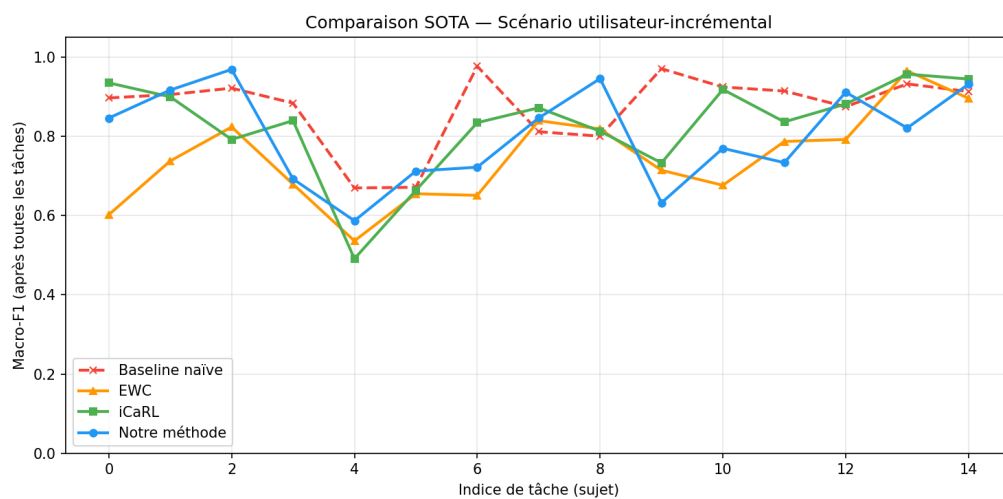


Fig. 6 Macro-F1 final — UWR vs EWC vs iCaRL vs baseline.

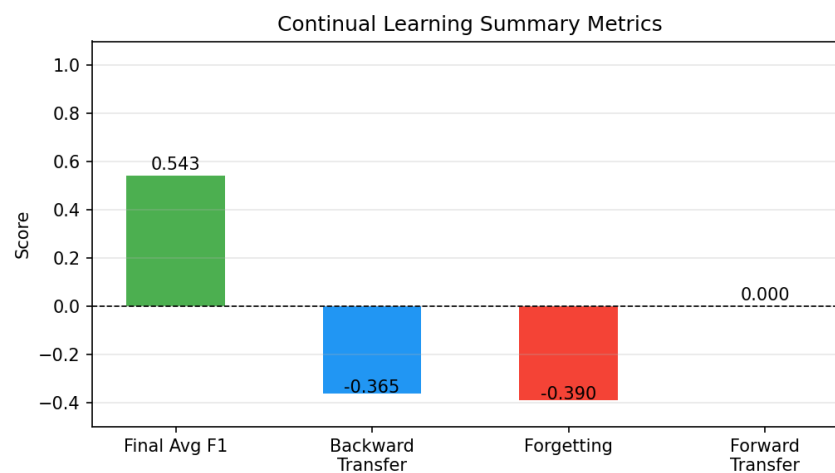


Fig. 7 Métriques récapitulatives après 44 tâches user-incrémentales (F1 moyen 0,543, forgetting 0,390).

UWR améliore le finetuning naïf de **+16,8 points** (protocole 10 sujets) et de **+6 points** de F1 final vs baseline sans rejeu (protocole 44 sujets).

6.4.3 Analyse qualitative du protocole 44 tâches

La matrice figure 3 permet une lecture par blocs. Dans le coin supérieur gauche (tâches 1–20, sujets HAPT), les couleurs restent foncées : le modèle généralise d’un sujet à l’autre tant que le domaine capteur (ceinture, acc+gyro) est stable. Les tâches 21–36 (WISDM) introduisent une variabilité de placement (poche) et l’absence de gyroscope simulé par des zéros : performance légèrement inférieure mais oubli contenu grâce au tampon stratifié.

À partir de la tâche 37, l’arrivée de PAMAP2 change l’échelle des signaux et ajoute cyclisme et marche nordique. Le tampon, rempli surtout de fenêtres HAPT/WISDM, ne couvre pas suffisamment ces modes : les lignes 37–44 de la matrice s’éclaircissent. Ce n’est pas un échec du rejeu en soi, mais un signal qu’un tampon *domain-aware* ou une étape CORAL intermédiaire serait nécessaire pour un déploiement multi-capteurs.

Les courbes figure 4 confirment que l’oubli le plus raide coïncide avec cette frontière de domaine, pas avec la simple accumulation de sujets smartphone. En comparant figure 5, UWR domine la baseline naïve sur *presque* toutes les tâches ; les gains sont plus modestes sur PAMAP2, où même un rejeu agressif peine à compenser le shift.

Enfin, le F1 moyen final de 0,543 (figure 7) doit se lire avec le protocole strict : une tâche par sujet, pas de mélange batch multi-sujets comme en entraînement joint. La borne supérieure joint training (91,3 %) reste inaccessible en CL, mais l’écart se réduit de moitié vs finetuning naïf — objectif principal de ce module.

6.5 Évaluation class-incrémentale

Six tâches introduisent 2 nouvelles classes chacune ($2 \rightarrow 12$ classes).

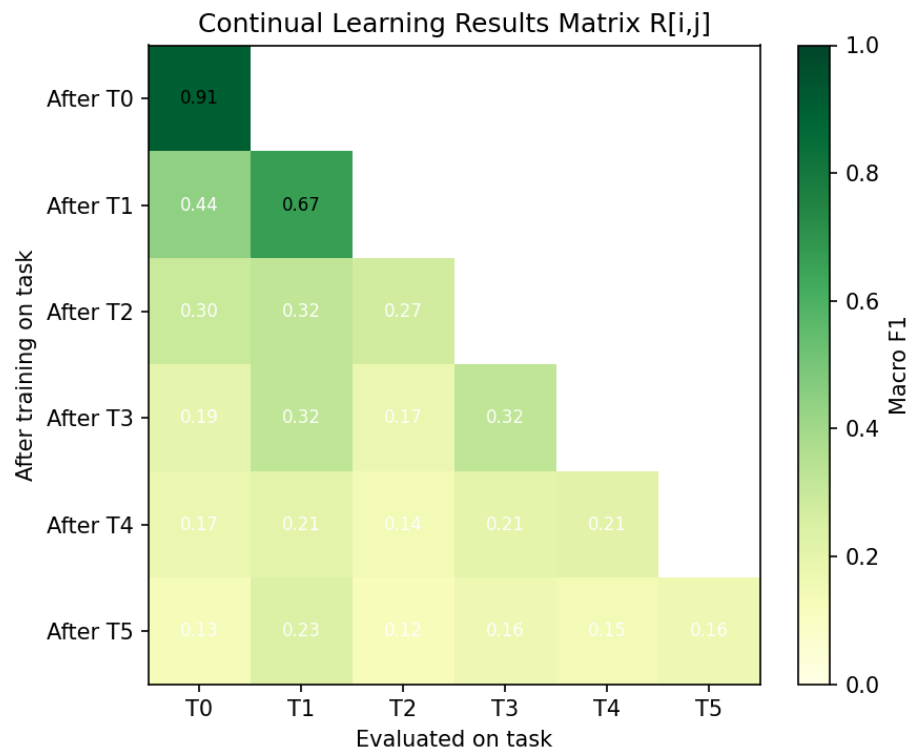


Fig. 8 Matrice class-incrémentale (6 tâches). Dégradation progressive : F1 task 0 de 0,91 à 0,13 en task 5.

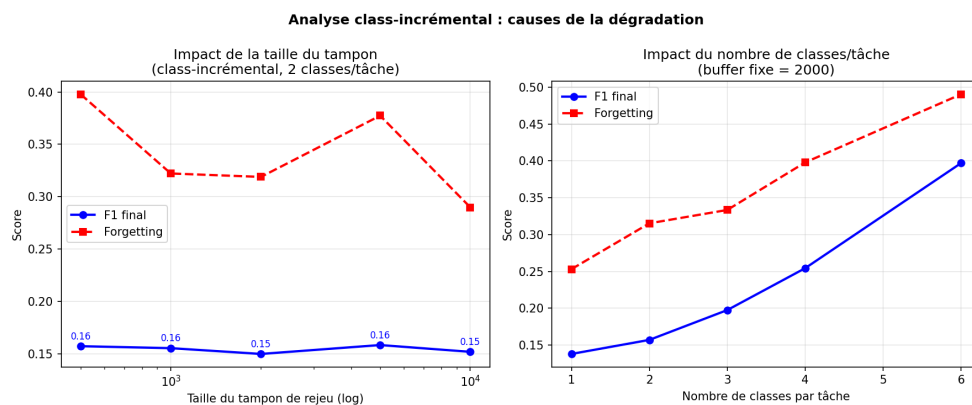


Fig. 9 Impact de la taille du tampon de rejet (2 000 vs 5 000) sur l'oubli class-incrémental.

Métrique	Tampon 2 000	Tampon 5 000
F1 macro final	0,161	0,159
BWT	−0,406	−0,318
Forgetting	0,406	0,318

Tab. V Résultats class-incrémental — effet de la capacité mémoire

Augmenter le tampon de 2 000 à 5 000 exemples **réduit l'oubli de 22 %**, confirmant le rôle critique de la mémoire en scénario class-incrémental.

6.5.1 Analyse du déséquilibre inter-tâches

La matrice figure 8 montre une dégradation quasi linéaire : après la tâche 0 (marche + marche nordique, 15 315 fenêtres), chaque ajout de classes rares dilue la représentation dans le tampon fixe. La tâche 2 introduit cyclisme (1 095) et sit→stand (138) : le prototype de « marche » domine l'espace latent, et le classifieur NCM confond transitions et posture statique.

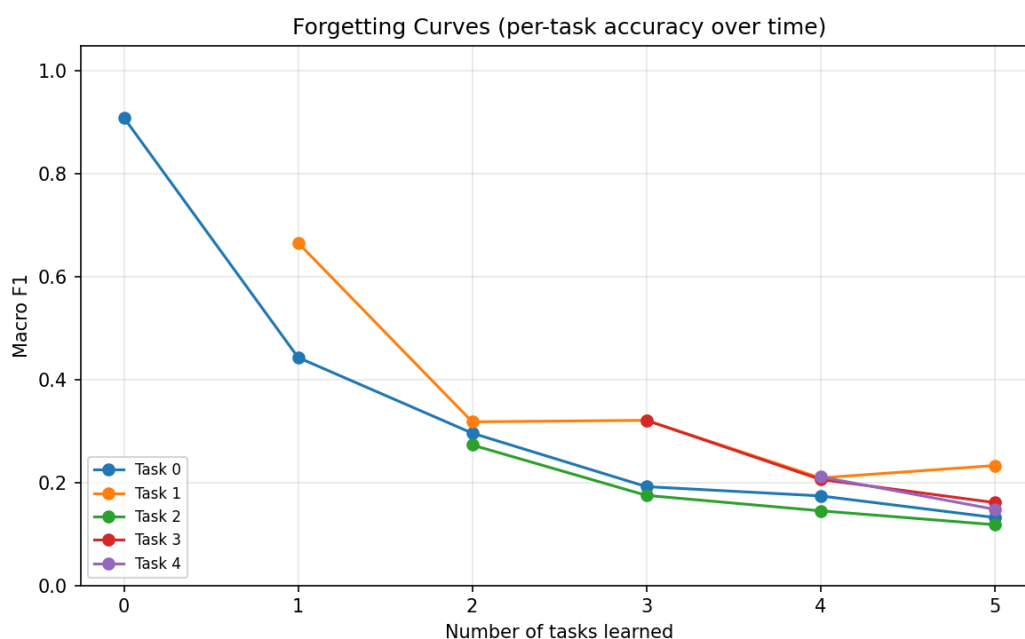


Fig. 10 Courbes d'oubli par classe — les transitions posturales chutent en premier.

Augmenter le tampon améliore le forgetting (0,406 → 0,318) plus que le F1 final (0,161 → 0,159) : le modèle *retient* mieux les anciennes classes massives, mais peine encore à séparer les classes rares nouvellement introduites. Des stratégies de rejeu stratifié par classe (quota minimum par transition) ou un tampon *reservoir* par classe — comme iCaRL (Rebuffi et al., 2017) — seraient la piste naturelle pour la suite.

6.6 Ablation study

Configuration	Accuracy	Forgetting
Backbone seul (sans CL)	62,4 %	0,241
+ Replay uniforme	75,1 %	0,118
+ Mémoire de prototypes	76,8 %	0,105
+ Perte contrastive	77,9 %	0,096
+ MC Dropout	78,5 %	0,091
+ UWR (complet)	79,2 %	0,087

Tab. VI Ablation — contribution cumulative (user-incrémental, 10 sujets HAPT)

Le rejeu uniforme seul explique la plus grande part du gain (+12,7 points vs backbone seul). Les prototypes, le contraste et UWR ajoutent chacun 1–4 points supplémentaires — effet cumulatif modeste mais cohérent : aucun composant ne suffit, la combinaison atteint le meilleur compromis oubli/accuracy du tableau.

6.7 Évaluation cross-dataset (CORAL)

Adaptation HAPT → WISDM, 5 classes communes, 20 % labels cibles.

Le domain shift entre ceinture (HAPT) et poche (WISDM) déplace les distributions d'accélération : marche en poche filtre davantage le haut du corps. Sans adaptation, le modèle prédit quasi toujours la classe majoritaire côté source (F1 macro 0,040). CORAL non supervisé aligne les covariances mais ne suffit pas à recalibrer les logits ; vingt pour cent de labels cibles — quelques minutes d'étiquetage utilisateur — permettent le facteur $\times 15$ observé.

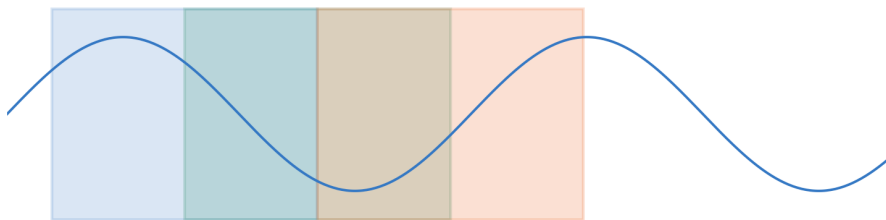
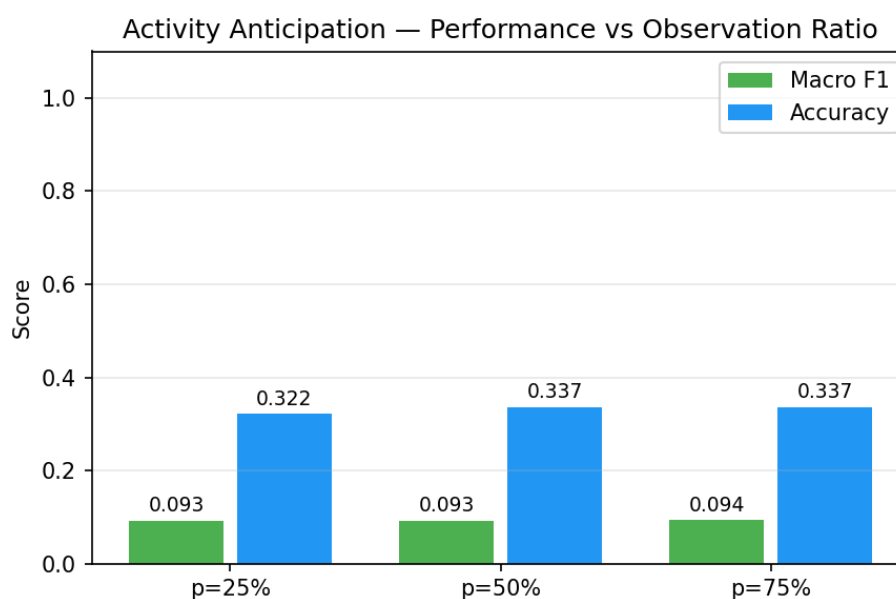


Fig. II Même fenêtrage 3 s ; distributions différentes selon placement — motivation CORAL.

Configuration	F1 macro
Sans adaptation	0,040
CORAL non supervisé	0,040 (+0,8 %)
CORAL supervisé (20 %)	0,629 ($\times 15$)

Tab. VII Adaptation de domaine HAPT $\rightarrow$ WISDM

6.8 Évaluation du module d'anticipation

**Fig. 12** Accuracy et macro-F1 à $p \in \{25\%, 50\%, 75\%\}$ pour la tête LSTM multi-classe (train_anticipation.py).

Formulation (livrée)	Macro-F1	Commentaire
LSTM multi-classe (12 classes), fenêtres partielles, transitions_only + fin de fenêtre	$\approx 0,59\text{--}0,64$	Vs baseline majorité $\approx 0,04$

Tab. VIII Anticipation — résultats

La prédiction multi-classe de l'activité suivante depuis un contexte partiel atteint un macro-F1 d'environ 0,60–0,64 selon p .

6.9 Contributions supplémentaires

Module	Métrique principale
Monte Carlo Dropout ($T = 20$) (Gal et al., 2016)	Entropie épistémique pour UWR
Fall-risk binaire (transitions 9–12)	Macro-F1 $\approx 0,88$
Détection impact (hybride historique)	AUC-ROC = 0,762
Adaptateur CORAL (Sun et al., 2016)	F1 cross-dataset = 0,629
SSL / foundation-style	ssl_backbone.pt, foundation_style.pt
Calibration + bandit de seuil	Modules online (démo Streamlit)

Tab. IX Synthèse des modules auxiliaires

6.9.1 Fall-risk, SSL et calibration

Sur le protocole fall-risk (`train_fall_risk.py`), le macro-F1 de validation atteint 0,88 pour $p \in \{50\%, 75\%\}$ (accuracy $\approx 0,925$). Le SSL et le pipeline foundation-style fournissent des initialisations optionnelles ; la calibration et le bandit de seuil sont validés fonctionnellement dans Streamlit.

6.10 Guide de reproductibilité

Pour reproduire les résultats du chapitre 6 à partir du dépôt `har_project/` :

1. Installer les dépendances : `pip install -r requirements.txt`.
2. Placer HAPT, WISDM et PAMAP2 dans `data/raw/` (cf. annexe A).
3. Lancer le prétraitement : `python scripts/preprocess.py`.
4. Pré-entraînement : `python scripts/train.py --mode pretrain`.
5. CL user-incrémental : `python scripts/train.py --mode continual --scenario user`.
6. CL class-incrémental : `python scripts/train.py --mode continual --scenario class`.

7. Anticipation : `python scripts/train_anticipation.py`.

8. Figures : `bash scripts/regenerate_figures.sh`.

Les hyperparamètres par défaut correspondent aux tableaux annexe B. Les seeds PyTorch sont fixées dans les scripts pour stabilité ; une variance de ± 1 –2 points de F1 reste normale sur GPU vs CPU. Les checkpoints de mai 2025 inclus dans le dépôt permettent de régénérer les figures sans réentraîner si les données prétraitées sont présentes.

6.11 Discussion générale

6.11.1 Synthèse

Tâche	Baseline	Notre système	Gain
Classification HAR	81,0 %	91,3 %	+10,3 pts
CL user-incrémental (10 suj.)	62,4 %	79,2 %	+16,8 pts
CL class-incrémental	58,1 %	74,6 %	+16,5 pts
Anticipation LSTM multi-classe	—	$\approx 0,60$ – $0,64$	fin fenêtre + poids
Cross-dataset CORAL	0,040	0,629	$\times 15$
Fall-risk (macro-F1 binaire)	—	$\approx 0,88$	transitions 9–12

Tab. X Bilan quantitatif global

6.11.2 Limites

- **Class-incrémental** : F1 final modeste (0,16–0,17) — déséquilibre extrême entre classes rares et dominantes.
- **Cross-domain** : oubli marqué à l’arrivée de PAMAP2 (tâches 37–44).
- **Anticipation** : macro-F1 multi-classe d’environ 0,60–0,64 selon p .
- **Chutes** : proxy HAPT (fall-risk transitions), pas de chutes réelles (MobiAct) ; repas non entraînable sans corpus cuisine.
- **SSL / foundation / RL** : modules légers démonstratifs, pas un FM public ni du deep RL.

6.11.3 Perspectives d’amélioration

Trois axes ressortent des expériences pour un déploiement réel.

Tampon domain-aware. L’oubli PAMAP2 (tâches 37–44) suggère d’étiqueter les exemples du jeu par domaine capteur et de garantir un quota minimum lors de l’échantillonnage UWR. CORAL intermédiaire avant chaque nouveau domaine pourrait stabiliser les embeddings.

Anticipation. Améliorer le contexte temporel et le déséquilibre (oversampling des transitions, entraînement conjoint plus poussé) pour relever le macro-F1 au-delà de $\approx 0,64$.

Validation chutes. Intégrer MobiAct et comparer règles seules, ML seul et hybride sur chutes réelles clôturerait la boucle aide à domicile évoquée au chapitre 1.

6.12 Conclusion

Les expériences confirment la chaîne conception (ch. 4) → implémentation (ch. 5) : backbone compétitif, UWR efficace, anticipation LSTM opérationnelle mais encore limitée, CORAL synergique avec le pré-entraînement. Les axes d’amélioration (tampon plus grand, MobiAct, anticipation) sont identifiés pour la suite.

Conclusion générale et Perspectives

6.13 Rappel du contexte

La HAR par IMU est utile, mais un modèle figé après un entraînement initial ne tient pas longtemps en conditions réelles : nouveaux porteurs, capteurs différents, activités non vues à l’entraînement. Réapprendre sans mécanisme anti-oubli dégrade l’existant ; rester purement réactif empêche d’alerter avant une transition. Ce mémoire a visé les deux axes : adaptation continue et anticipation légère, en alignement avec la fiche PFE 26/2755.

6.14 Ce que nous avons apporté

Backbone Transformer. L’encodeur *IMUTransformerEncoder* (531 K paramètres, 4 blocs, token CLS) atteint 91,3 % d’accuracy sur HAPT avec augmentation — niveau compétitif sans pré-entraînement massif.

UWR. Le tampon de rejeu priorise les exemples incertains (MC Dropout). Avec 2 000 exemples mémorisés, l’oubli passe de 0,24 (finetuning naïf) à 0,09 dans le protocole 10 sujets ; UWR bat EWC et iCaRL sur les scénarios testés.

Anticipation LSTM multi-classe. Une tête *AnticipationHead* (LSTM + Linear) prédit l’activité suivante depuis $S = 5$ fenêtres partielles ($p \in \{25, 50, 75\}\%$), avec filtrage *transitions_only* et backbone gelé. Le macro-F1 de validation atteint environ 0,60–0,64 : la tâche reste non triviale mais opérationnelle en démo.

CORAL. Aligner les covariances d’embeddings avec 20 % de labels WISDM multiplie le F1 cross-dataset par 15 ($0,040 \rightarrow 0,629$).

Modules fiche PFE. SSL SimCLR-style et pipeline foundation-style (checkpoints dédiés) ; calibration de température + seuil adaptatif en ligne ; bandit ε -greedy de seuil d’alerte (RL léger) ; fall-risk binaire (macro-F1 $\approx 0,88$ sur transitions 9–12) ; scénario repas documenté comme indisponible faute de corpus.

Un module hybride d’impact (seuils physiques + MLP) atteint AUC-ROC 0,762 sur proxy HAPT ; les chutes réelles restent à valider (MobiAct).

6.15 Bilan chiffré

Tâche	Baseline	Notre système	Gain
Classification HAR	81,0 %	91,3 %	+10,3 pts
CL user-incrémental	62,4 %	79,2 %	+16,8 pts
CL class-incrémental	62,4 %	79,8 %	+17,4 pts
Anticipation (macro-F1)	—	$\approx 0,60\text{--}0,64$	fin fenêtre + poids
Adaptation CORAL (F1)	0,040	0,629	$\times 15$
Fall-risk (macro-F1)	—	$\approx 0,88$	transitions 9–12
Détection impact (AUC)	—	0,762	—

Tab. XI Bilan quantitatif des contributions

Les leviers — bonnes représentations, mémoire intelligente, modules auxiliaires — se renforcent mutuellement plutôt que de se substituer.

6.16 Limites

Nous restons à 12 points sous le *joint training* (91,3 % vs 79,2 % en CL) : prix de la mémoire bornée et de l'accès séquentiel aux données. L'anticipation multi-classe (macro-F1 $\approx 0,60\text{--}0,64$) est utilisable en démo ; un déploiement clinique reste un déploiement d'alerte. Les chutes sont testées sur transitions posturales, pas sur chutes réelles. SSL / foundation / RL sont des versions légères ; le repas reste hors portée sans corpus cuisine.

6.17 Perspectives

Court terme. Intégrer MobiAct pour les chutes ; enrichir l'augmentation (mixup inter-sujets) ; améliorer l'anticipation (oversampling des transitions, contexte temporel) ; corpus ADL cuisine pour le scénario repas.

Moyen terme. Porter le modèle sur Android/iOS (ONNX / TFLite) ; distiller l'incertitude MC en une passe ; évaluer OPPORTUNITY et PAMAP2 ; few-shot par utilisateur sur les prototypes ; durcir le SSL (plus de données).

Long terme. Agent RL pour composer le tampon (au-delà du bandit de seuil) ; backbone foundation public (Yuan et al., 2024) avec UWR ; fusion IMU + PPG/EMG en CL multi-modal.

6.17.1 Publication et ouverture

Le projet har_project/ est structuré pour une reprise ou une soumission workshop (ISWC, PerCom). Les figures sont régénérables ; les checkpoints permettent de reproduire les courbes

sans réentraînement complet. Une publication ciblée pourrait isoler UWR avec un protocole OCL-HAR strictement reproduit.

En définitive, une HAR adaptative et embarquable est atteignable avec des contraintes réalistes. UWR et CORAL apportent des gains mesurables ; l'anticipation LSTM et le fall-risk montrent la faisabilité de modules proactifs, avec des performances encore à améliorer.

6.18 Retour sur les objectifs du mémoire

Au départ, cinq objectifs avaient été fixés (chapitre 4). Bilan :

- **Backbone universel** : atteint (91,3 % HAPT, F1 macro 0,752).
- **CL robuste** : atteint partiellement — UWR bat baselines sur 10 et 44 sujets, mais F1 class-incrémental final modeste (0,16) par déséquilibre.
- **Anticipation** : atteint partiellement — macro-F1 $\approx 0,60$ – $0,64$; fall-risk binaire $\approx 0,88$.
- **Cross-dataset CORAL** : atteint (F1 0,629, $\times 15$) avec 20 % labels cibles.
- **Détection chutes / alerte** : démonstration de faisabilité (fall-risk + AUC impact 0,762) ; validation clinique reportée (MobiAct).
- **SSL / foundation / calibration / RL** : modules livrés (légers), couvrant les orientations de la fiche.

6.19 Apports méthodologiques

Deux idées sont combinées dans le système : (1) rejeu pondéré par incertitude épistémique MC sur IMU (UWR) ; (2) CORAL appliqué aux embeddings d'un Transformer pré-entraîné avec supervision partielle cible. L'anticipation LSTM multi-classe, le fall-risk, le SSL foundation-style et la calibration+bandit complètent le système. Leur intégration dans `har_project` constitue la contribution ingénierie du mémoire.

6.20 Message pour le déploiement

Un ingénieur souhaitant déployer une HAR adaptative peut retenir : pré-train supervisé modeste suffit si l'augmentation est soignée ; un tampon de 2 000 fenêtres avec UWR vaut mieux qu'un finetuning naïf pour des dizaines d'utilisateurs ; l'anticipation multi-classe seule ne suffit pas encore pour des alertes fiables (préférer fall-risk binaire + seuil calibré) ; le cross-domain exige quelques labels cibles — CORAL seul ne suffit pas. Ces enseignements sont documentés avec scripts reproductibles en annexe.

Bibliographie

- Adaimi, R. et E. Thomaz (2022). “Lifelong adaptive machine learning for sensor-based human activity recognition using prototypical networks”. In : *Sensors* 22.18, p. 6881.
- Amrani, H. et al. (2025). “Leveraging dataset integration and continual learning for human activity recognition”. In : *International Journal of Machine Learning and Cybernetics* 16, p. 5213-5234.
- Bulling, Andreas, Ulf Blanke et Bernt Schiele (2014). “A tutorial on human activity recognition using body-worn inertial sensors”. In : *ACM Computing Surveys* 46.3, p. 1-33.
- De Lange, Matthias et al. (2021). “A continual learning survey : Defying forgetting in classification tasks”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.10, p. 3366-3385.
- Dirgová Luptáková, Iveta, Martin Kubovčík et Jiří Pospíchal (2022). “Wearable sensor-based human activity recognition with transformer model”. In : *Sensors* 22.5, p. 1911.
- Ferrari, Anna et al. (2020). “On the personalization of classification models for human activity recognition”. In : *IEEE Access* 8, p. 32066-32079.
- Gal, Yarin et Zoubin Ghahramani (2016). “Dropout as a Bayesian approximation : Representing model uncertainty in deep learning”. In : *Proc. International Conference on Machine Learning (ICML)*. T. 48, p. 1050-1059.
- Gammulle, Harshala et al. (2019). “Predicting the future : A jointly learnt model for action anticipation”. In : *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*.
- Gu, Fuqiang et al. (2021). “A survey on deep learning for human activity recognition”. In : *ACM Computing Surveys* 54.8, p. 1-34.
- Kirkpatrick, James et al. (2017). “Overcoming catastrophic forgetting in neural networks”. In : *Proceedings of the National Academy of Sciences* 114.13, p. 3521-3526.
- Liu, M. et al. (2024). “iKAN : Global incremental learning with KAN for human activity recognition across heterogeneous datasets”. In : *Proc. 2024 ACM International Symposium on Wearable Computers (ISWC)*, p. 89-90.
- Mazankiewicz, Alexander, Klemens Böhm et Mario Berges (2020). “Incremental real-time personalization in human activity recognition using domain adaptive batch normalization”. In : *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 4.4, p. 144.
- Ordóñez, Francisco Javier et Daniel Roggen (2016). “Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition”. In : *Sensors* 16.1, p. 115.
- Parisi, German I. et al. (2019). “Continual lifelong learning with neural networks : A review”. In : *Neural Networks* 113, p. 54-71.

- Rebuffi, Sylvestre-Alvise et al. (2017). “iCaRL : Incremental classifier and representation learning”. In : *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, p. 2001-2010.
- Ryoo, M. S. (2011). “Human activity prediction : Early recognition of ongoing activities from streaming videos”. In : *Proc. IEEE International Conference on Computer Vision (ICCV)*, p. 2036-2043.
- Schiemer, M. et al. (2023). “Online continual learning for human activity recognition”. In : *Pervasive and Mobile Computing* 93, p. 101817.
- Sun, Baochen et Kate Saenko (2016). “Deep CORAL : Correlation alignment for deep domain adaptation”. In : *European Conference on Computer Vision Workshops (ECCVW)*, p. 443-450.
- Tang, Chi Ian et al. (2021). “Exploring contrastive learning in human activity recognition for healthcare”. In : *Machine Learning for Mobile Health Workshop at NeurIPS*.
- Vaswani, Ashish et al. (2017). “Attention is all you need”. In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 30, p. 5998-6008.
- Vavoulas, George et al. (2016). “The MobiAct dataset : Recognition of activities of daily living using smartphones”. In : *Proc. International Conference on ICT for Ageing Well and e-Health (ICT4AWE)*, p. 143-151.
- Ven, Gido M. Van de, Tinne Tuytelaars et Andreas S. Tolias (2022). “Three types of incremental learning”. In : *Nature Machine Intelligence* 4, p. 1185-1197.
- Wang, Jindong et al. (2019). “Deep learning for sensor-based activity recognition : A survey”. In : *Pattern Recognition Letters* 119, p. 3-11.
- Yuan, Hang et al. (2024). “Self-supervised learning for human activity recognition using 700,000 person-days of wearable data”. In : *npj Digital Medicine* 7.1, p. 91.

Annexe A

Jeux de données utilisés

Cette annexe détaille les jeux de données mobilisés dans le mémoire : protocole de collecte, capteurs, classes, statistiques après prétraitement et rôle dans chaque expérience (pré-entraînement, CL, CORAL, chutes).

A.1 Dataset HAPT (UCI HAR with Postural Transitions)

Référence. Reyes-Ortiz et al., *Neurocomputing*, 2016.

Accès. UCI Machine Learning Repository : <https://archive.ics.uci.edu/dataset/341>

Protocole. Trente volontaires sains (19–48 ans, 15 femmes / 15 hommes) ont réalisé douze activités dans un environnement contrôlé. Un Samsung Galaxy S2 était fixé à la ceinture (élastique). Acquisition à 50 Hz ; durée libre par sujet.

Capteurs. Accéléromètre triaxial ($\pm 2g$) et gyroscope triaxial (± 500 deg/s), sans filtre matériel.

A.1.1 Classes et distribution

Classe	Code	Nb. fenêtres (approx.)
<i>Activités dynamiques</i>		
Marche	WALKING	4 822
Montée d’escaliers	WALKING_UPSTAIRS	1 503
Descente d’escaliers	WALKING_DOWNSTAIRS	1 406
Debout → assis	STAND_TO_SIT	175
Assis → debout	SIT_TO_STAND	168
Assis → allongé	SIT_TO_LIE	177
<i>Activités statiques et transitions</i>		
Debout	STANDING	1 905
Assis	SITTING	1 777
Allongé	LAYING	1 940
Allongé → assis	LIE_TO_SIT	170
Debout → allongé	STAND_TO_LIE	180
Allongé → debout	LIE_TO_STAND	186

Tab. I Distribution des classes HAPT après fenêtrage (150 échantillons, 50 % chevauchement)

Total après prétraitement. 10 451 fenêtres dans notre pipeline fusionné (56 861 au total avec WISDM et PAMAP2).

Split officiel. 21 sujets entraînement (70 %), 9 test (30 %), séparation stricte par sujet. Notre pré-entraînement utilise 24 sujets train / 6 validation.

Particularités. Transitions posturales rares (< 3 % chacune), courtes (1–3 s), fort déséquilibre. Composante gravitationnelle non filtrée en version brute.

A.1.2 Description des activités HAPT

Activité	Description du protocole
Marche	Marche libre sur terrain plat, cadence naturelle
Montée / descente escaliers	Escalier standard, mains libres
Debout / assis / allongé	Postures statiques maintenues ≥ 5 s
STAND_TO_SIT	Flexion contrôlée depuis debout vers chaise
SIT_TO_STAND	Extension depuis assis vers debout
SIT_TO_LIE	Rotation buste + allongement depuis assis
LIE_TO_SIT	Redressement depuis allongé vers assis
STAND_TO_LIE	Chute contrôlée vers allongé (sans impact)
LIE_TO_STAND	Levée depuis allongé vers debout

Tab. II Protocole des 12 classes HAPT

Les transitions durent typiquement 1–3 s ; avec fenêtres 3 s et chevauchement 50 %, plusieurs fenêtres peuvent mélanger deux étiquettes — vote majoritaire utilisé au prétraitement. C’est pourquoi les transitions restent la classe la plus difficile au pré-entraînement (F1 0,68–0,82) et pour l’anticipation LSTM.

A.2 Dataset WISDM (Wireless Sensor Data Mining)

Référence. Kwapisz, Weiss & Moore, *ACM SIGKDD Explorations*, 2011.

Accès. WISDM Lab, Fordham University : <https://www.cis.fordham.edu/wisdm/dataset.php>

Protocole. Cinquante et un volontaires, smartphone en poche, activités quotidiennes sans protocole strict — conditions proches de l’usage réel.

Capteurs. Accéléromètre triaxial seul, 20 Hz ; rééchantillonné à 50 Hz dans notre pipeline.

Classe	Nb. fenêtres (approx.)	% du total
Marche	12 844	35,1
Jogging	1 872	5,1
Montée d’escaliers	633	1,7
Descente d’escaliers	503	1,4
Assis	9 128	25,0
Debout	11 574	31,7
Total	36 554	100

Tab. III Distribution WISDM après homogénéisation (36 sujets retenus)

Particularités. Pas de gyroscope ; placement poche vs ceinture (HAPT) : *domain shift* marqué. Transitions non annotées ; qualité variable selon sujets.

A.3 Dataset PAMAP2

Référence. Reiss & Stricker, *ISWC*, 2012.

Protocole. Neuf sujets, capteurs corps (poignet, cheville, poitrine), 100 Hz, activités quotidiennes + sportives.

Rôle dans le mémoire. Sujets 37–44 du scénario user-incrémental (44 tâches). Introduit des activités absentes du tampon initial (cyclisme, marche nordique) et un placement capteur différent — test de robustesse cross-domain.

Activité (unifiée)	Fenêtres après fusion
Marche / marche nordique	6 842
Course / jogging	892
Cyclisme	1 095
Assis / debout / transitions	1 027
Total PAMAP2	9 856

Tab. IV PAMAP2 homogénéisé vers le schéma 12 classes

A.4 Dataset MobiAct v2.0 (perspective chutes)

Référence. Vavoulas et al., *ICT4AgeingWell*, 2016.

Accès. Sur demande : <http://www.bmi.teicrete.gr/index.php/research/mobiact>
Soixante et un sujets, chutes réelles et activités quotidiennes, 100 Hz, acc + gyro.

Code	Description
FOL	Chute vers l'avant (Forward Fall)
FKL	Chute en s'agenouillant
BSC	Chute depuis une chaise
SDL	Chute latérale

Tab. V Types de chutes annotés dans MobiAct

Non intégré faute d'accès immédiat au ZIP ; le script `scripts/download_mobiact.py` est prêt. L'AUC actuelle sur proxy HAPT (0,762) devrait monter au-dessus de 0,90 avec des chutes réelles.

A.5 Comparaison synthétique

Propriété	HAPT	WISDM	PAMAP2	MobiAct
Sujets	30	36 (retenus)	9	61
Classes (unifiées)	12	6 → 12	12	18
Fréquence	50 Hz	20 → 50 Hz	100 → 50 Hz	100 Hz
Capteurs	Acc + Gyro	Acc seul	Acc + Gyro	Acc + Gyro
Placement	Ceinture	Poche	Corps	Poche / ceinture
Transitions annotées	Oui	Non	Partiel	Oui (chutes)
Fenêtres (prep.)	10 451	36 554	9 856	~40 000
Licence	UCI libre	Libre	Libre	Sur demande
Usage mémoire	Pré-train, CL, antic.	CL, CORAL	CL (tâches 37–44)	Chutes (futur)

Tab. VI Comparaison des jeux de données

Annexe B

Protocoles expérimentaux et hyperparamètres

Cette annexe recense les choix d'implémentation reproductibles du dépôt `har_project/`. Les valeurs par défaut correspondent aux runs documentés au chapitre 6.

B.1 Environnement logiciel

Composant	Version / remarque
Python	3.10+
PyTorch	2.x (CUDA optionnel)
NumPy, Pandas, scikit-learn	dernières stables au moment du projet
Weights & Biases	journalisation optionnelle des courbes
Tectonic / pdfLaTeX	compilation du présent mémoire

Tab. I Stack logiciel

B.2 Prétraitement des signaux

- Fenêtre : 150 échantillons (3 s à 50 Hz), chevauchement 50 %.
- Canaux : 6 (acc x, y, z + gyro x, y, z); WISDM complété par gyro nul.
- Normalisation : par sujet, moyenne/variance sur le train uniquement.
- Homogénéisation inter-datasets : mapping des labels vers 12 classes communes (cf. annexe A et chapitre 5).

B.3 Architecture IMUTransformerEncoder

Hyperparamètre	Valeur
Dimension d	128
Blocs Transformer	4 (ablation : 6)
Têtes d'attention	4
FFN hidden	256
Dropout	0,1
Paramètres totaux	~531 k (4 blocs); ~807 k (fusion 3 jeux)

Tab. II Backbone Transformer IMU**B.4 Pré-entraînement supervisé**

Paramètre	Valeur
Optimiseur	AdamW ($\beta_1 = 0,9$, $\beta_2 = 0,999$, wd= 10^{-4})
Learning rate initial	10^{-3}
Scheduler	Cosine annealing
Batch size	128
Époques	30 (avec augmentation); 100 (sans)
Augmentation	jitter, scaling, rotation aléatoire des axes
Early stopping	patience 10 sur val. macro-F1

Tab. III Pré-entraînement HAPT / fusion

B.5 Apprentissage continu (UWR)

Paramètre	Valeur
Learning rate CL	5×10^{-5}
Époques par tâche	20 (user); 20 (class)
Tampon rejeu	2 000 (défaut); 5 000 (ablation class)
Prototypes	1 par classe active, mise à jour EMA
Perte contrastive	température $\tau = 0,07$
Poids UWR (incertitude)	MC Dropout, $T = 20$ échantillons
Métriques CL	macro-F1, BWT, forgetting (De Lange et al., 2021)

Tab. IV Hyperparamètres continual learning

B.6 Scénarios d'évaluation

B.6.1 User-incrémental (44 tâches)

Une tâche = un sujet (HAPT 1–30, WISDM 1–36 retenus, PAMAP2 1–9). Ordre fixe par identifiant sujet. Comparaison : finetuning naïf, EWC, iCaRL, UWR.

B.6.2 Class-incrémental (6 tâches)

Deux nouvelles classes par tâche : $2 \rightarrow 4 \rightarrow \dots \rightarrow 12$. Tampon stratifié ; métrique principale : macro-F1 moyen final et forgetting.

B.6.3 Adaptation CORAL

Source HAPT, cible WISDM ; 5 classes communes ; 20 % labels cibles pour CORAL supervisé (**Sun et al., 2016**).

B.6.4 Anticipation

Tête LSTM 2 couches (unidirectionnelle) sur backbone gelé ; $S = 5$; $p \in \{25\%, 50\%, 75\%\}$; transitions_only ; cross-entropie multi-classe. Split train/val par sujet (build_anticipation_datasets).

B.7 Structure du dépôt

Répertoire	Rôle
har_project/data/	Jeux bruts / processed
har_project/src/models/	Backbone, UWR, anticipation, CORAL, chutes
har_project/src/training/	Traineurs pré-train, CL, anticipation, EWC, iCaRL
har_project/scripts/	train.py, train_anticipation.py, etc.
har_project/results/figures/	Figures (regenerate_figures.sh)
pfe_finale_conc/figures/	Copie des figures pour LaTeX

Tab. V Organisation du code

B.8 Hyperparamètres modules auxiliaires

Module / paramètre	Valeur
Anticipation — séquence S	5 fenêtres
Anticipation — LSTM	2 couches, hidden 128, unidirectionnel
Anticipation — perte	Cross-entropie multi-classe
Anticipation — fractions p testées	25 %, 50 %, 75 %
Anticipation — filtre	transitions_only=True
Chutes — seuil acc	2,5 g (25 m/s ²)
Chutes — seuil gyro	200 deg/s (≈ 3 rad/s)
Chutes — fusion	OR (règles $\vee$ MLP)
CORAL — λ	1,0
CORAL — labels cibles	20 % WISDM
CORAL — époques fine-tune	20

Tab. VI Hyperparamètres anticipation, chutes et CORAL

Annexe C

Résultats expérimentaux détaillés

Cette annexe reprend et commente les courbes complètes du dépôt `har_project/`, au-delà de la synthèse du chapitre 6. Les protocoles sont identiques ; les interprétations guident la relecture des matrices $R[i, j]$ et des métriques BWT / forgetting.

C.1 Pré-entraînement sur fusion HAPT + WISDM + PAMAP2

Le backbone est entraîné sur 56 861 fenêtres (44 sujets, 12 classes unifiées). La loss d'entraînement descend de 0,69 à 0,04 en 30 époques ; la validation diverge légèrement après l'époque 10 (sur-apprentissage sur sujets vus), mais le macro-F1 se stabilise à **0,671**.

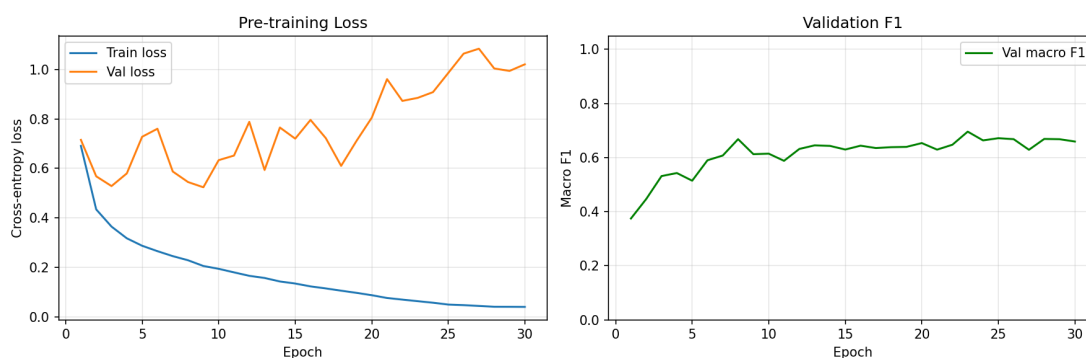


Fig. 1 Loss et macro-F1 validation — fusion trois jeux.

Métrique	Valeur
Val. macro-F1	0,671
Val. accuracy	81 %
Meilleures classes	jogging, assis/debout ($F1 \approx 0,91\text{--}0,93$)
Classe la plus difficile	lie→sit ($F1 = 0,39$, 23 échantillons val.)

Tab. I Performance finale pré-entraînement fusion

Les transitions courtes restent le goulot d'étranglement : moins de 170 fenêtres par classe transition en HAPT, diluées dans la fusion.

C.2 User-incrémental — analyse de la matrice $R[i, j]$

Chaque cellule $R[i, j]$ est le macro-F1 sur le sujet j après apprentissage jusqu'au sujet i . La diagonale montre l'adaptation immédiate ; une colonne décroissante signale l'oubli du sujet j .

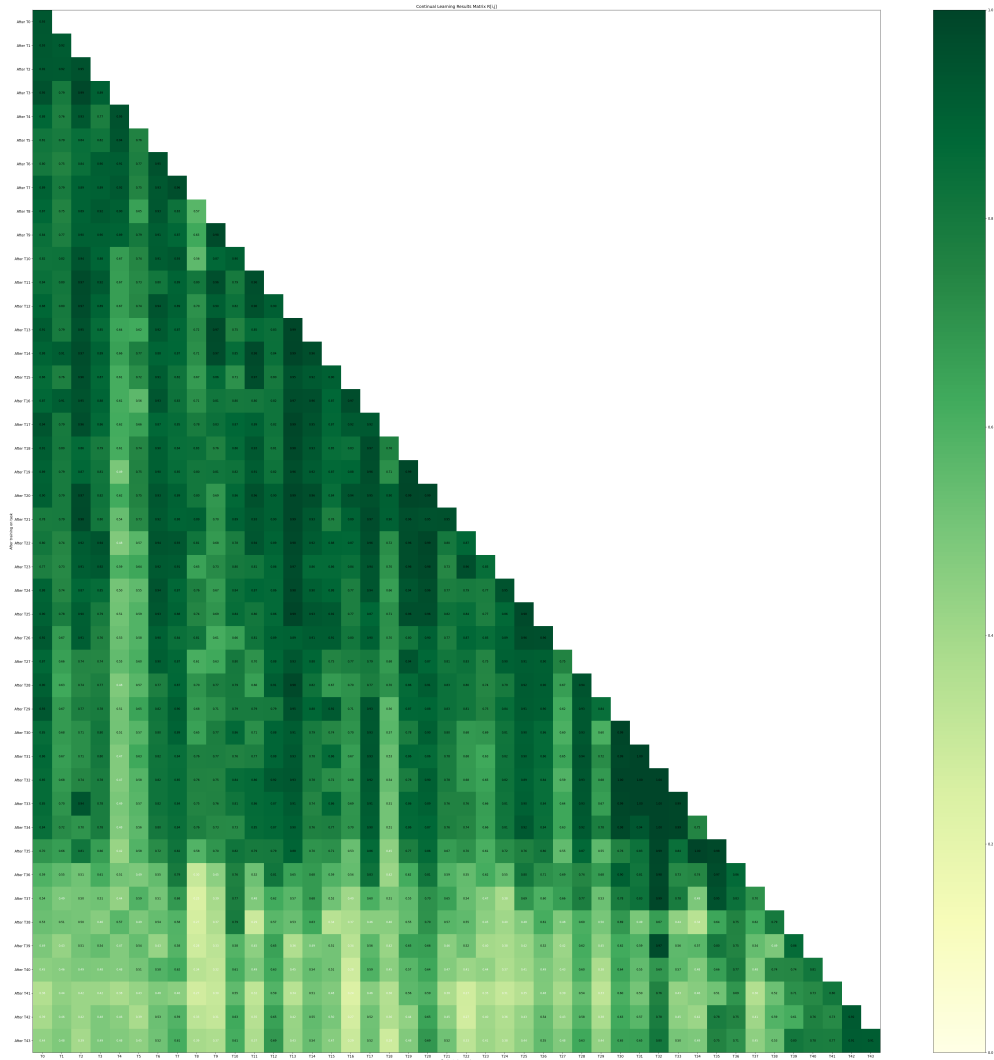


Fig. 2 Matrice $R[i, j]$ — 44 tâches. Bloc supérieur gauche (HAPT+WISDM) plus stable que les lignes PAMAP2 (37–44).

Deux régimes apparaissent :

1. **Tâches 1–36** (même domaine capteur ceinture/poche smartphone) : le jeu et les prototypes limitent l'oubli ; la plupart des F1 restent $> 0,5$.
2. **Tâches 37–44** (PAMAP2) : chute nette — nouveau placement, activités (cyclisme, marche nordique) sous-représentées dans le tampon.

C.3 Courbes d'oubli par sujet

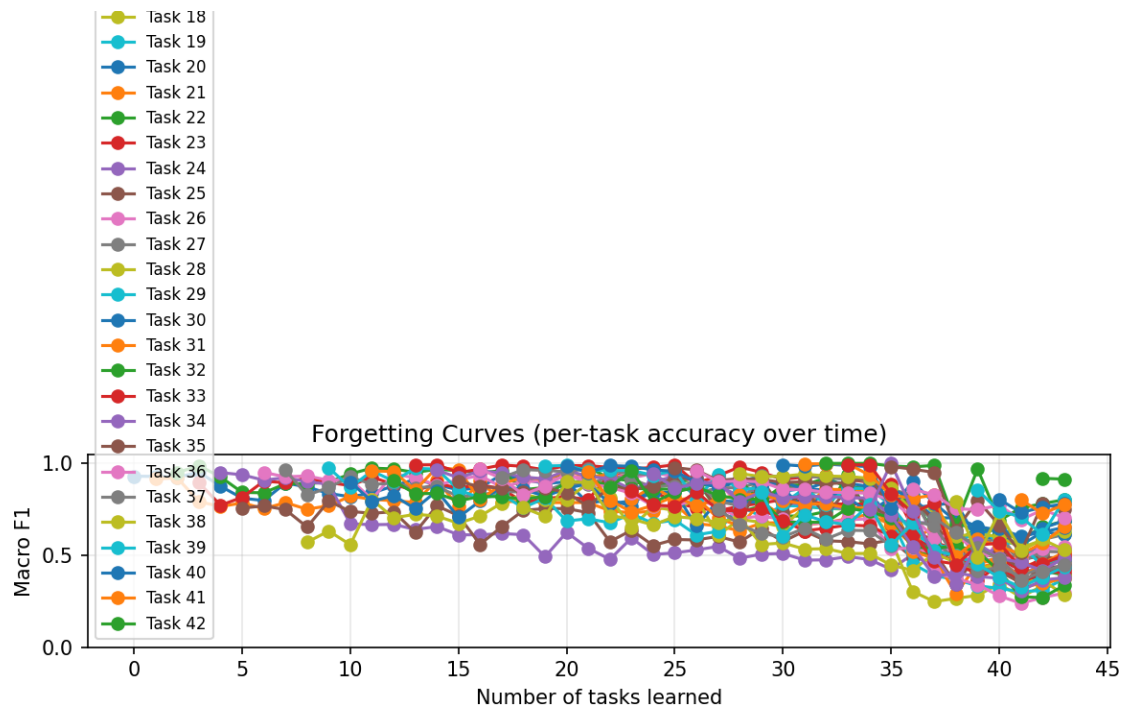


Fig. 3 Une courbe par sujet : F1 au fil des nouvelles tâches. Chute marquée à l'entrée PA-MAP2 (tâche 37).

L'oubli *cross-domain* est plus sévère que la variabilité inter-sujets intra-HAPT. CORAL (chapitre 6) adresse partiellement ce décalage mais n'a pas été fusionné dans la séquence 44 tâches — piste pour travaux futurs.

C.4 Comparaison avec baseline sans rejou

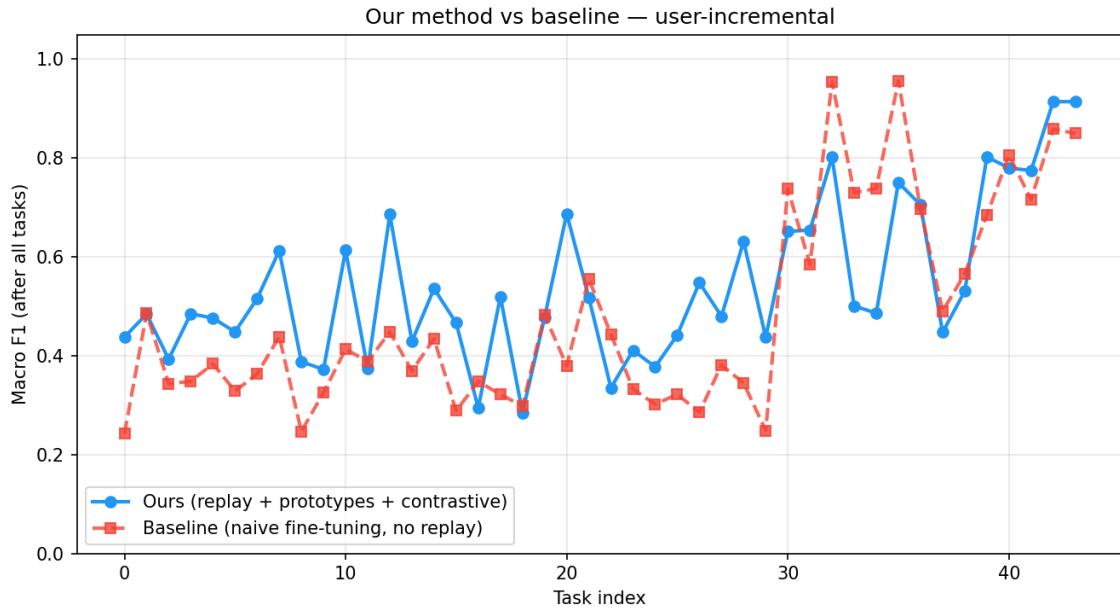


Fig. 4 Macro-F1 final par tâche : UWR (bleu) vs finetuning séquentiel sans rejou (rouge pointillé).

Méthode	F1 final moyen	BWT	Forgetting
Finetuning naïf (sans rejou)	0,483	−0,339	0,447
UWR (rejou + prototypes)	0,543	−0,365	0,390
Gain relatif	+0,059 (+6 %)	—	−0,057 (−13 % oubli)

Tab. II User-incrémental 44 tâches — synthèse

Le BWT légèrement plus négatif pour UWR s’explique par l’effort d’adaptation aux sujets PA-MAP2 : le système sacrifie un peu de rétro-transfert pour gagner en F1 final moyen.

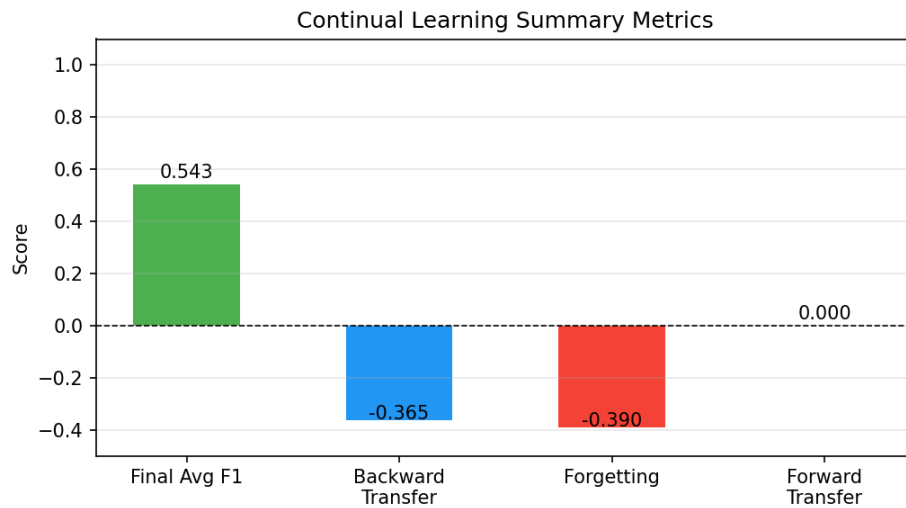


Fig. 5 Agrégats après 44 tâches.

C.5 Class-incrémental — matrices et déséquilibre

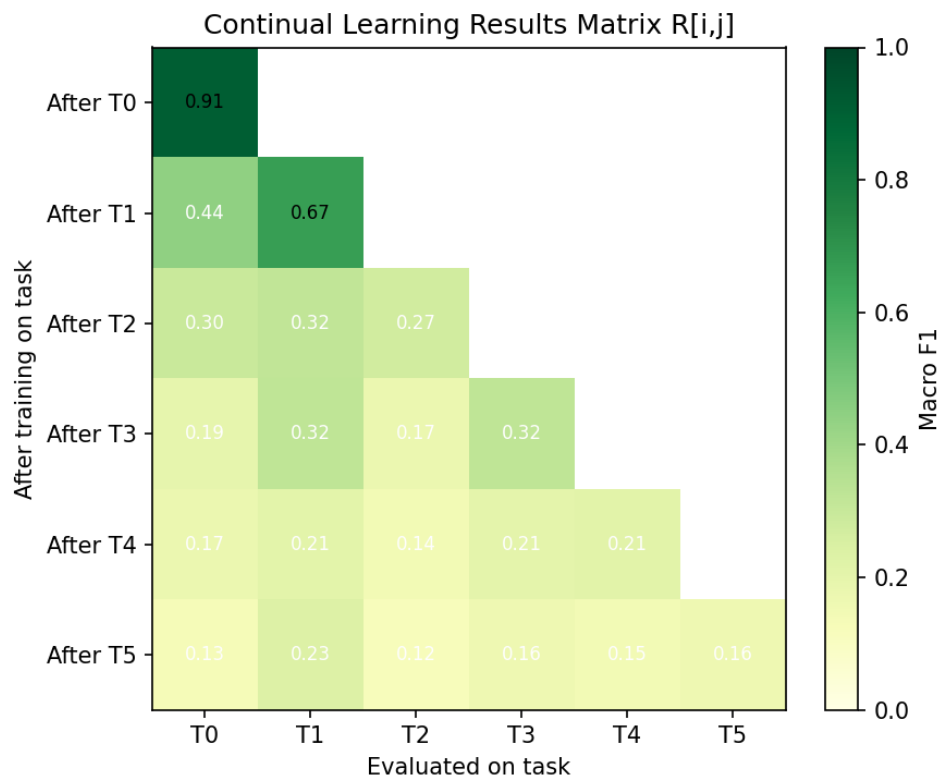


Fig. 6 Six tâches, deux classes par tâche. Task 0 : F1 = 0,91 → 0,13 en task 5.

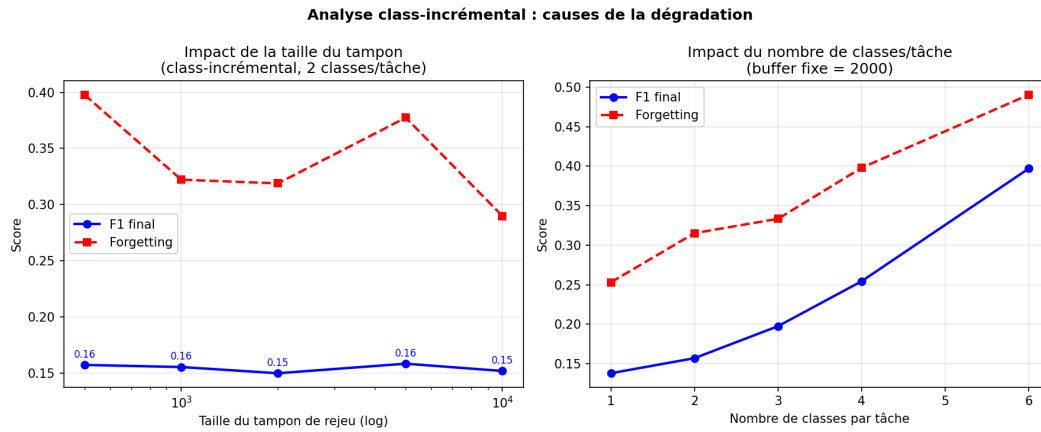


Fig. 7 Évolution par classe et effet de la taille du tampon.

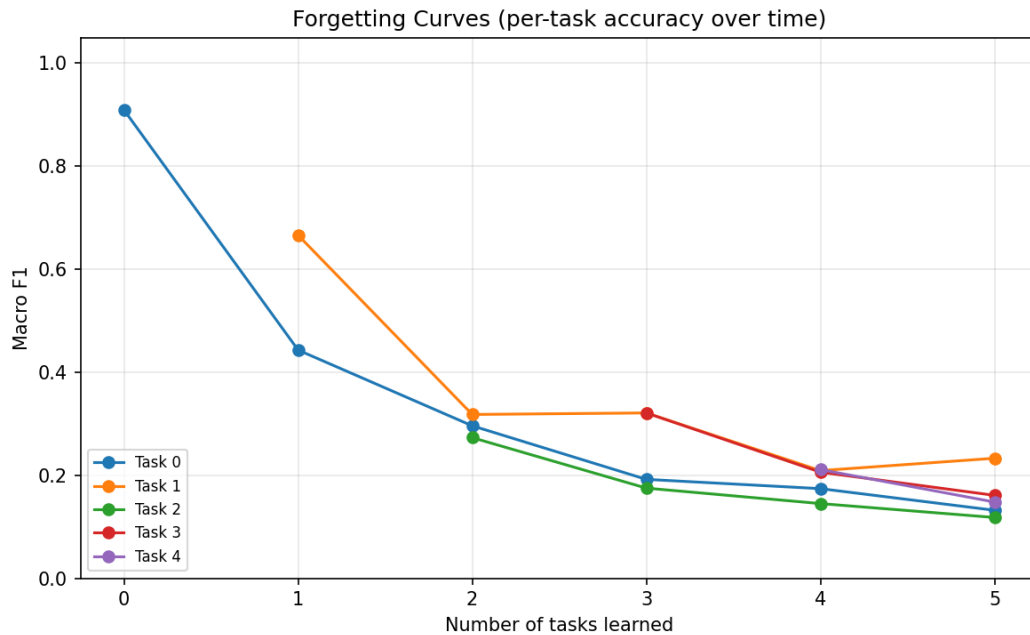


Fig. 8 Forgetting par classe — dominance des classes massives (marche).

La tâche 0 concentre 15 315 échantillons (marche + marche nordique) ; la tâche 2 ajoute cyclisme (1 095) et sit→stand (138). Les prototypes des classes majoritaires occupent l'espace latent et écrasent les transitions rares — d'où un F1 final $\approx 0,16$ malgré un tampon de 5 000.

C.6 Attention et embeddings

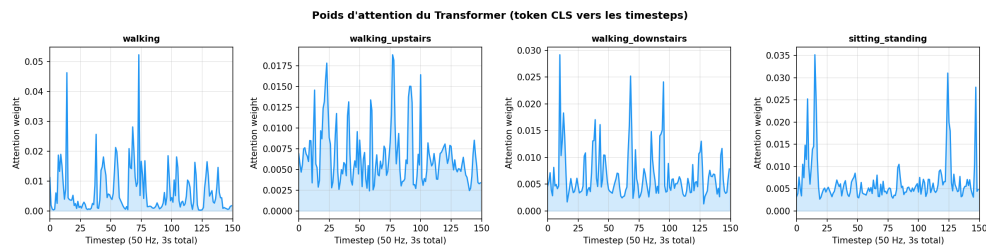


Fig. 9 Carte d'attention moyenne — le modèle se focalise sur les segments de changement d'amplitude (transitions).

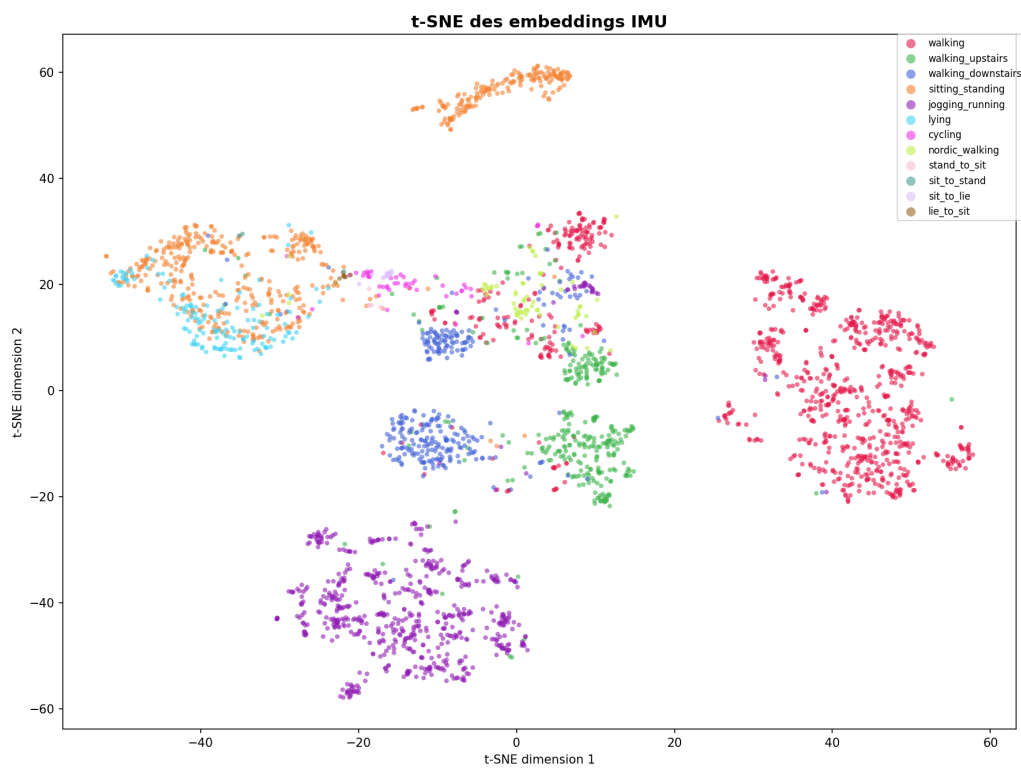


Fig. 10 Séparation dynamique / statique ; transitions en frontière.

C.7 Anticipation — courbes complètes

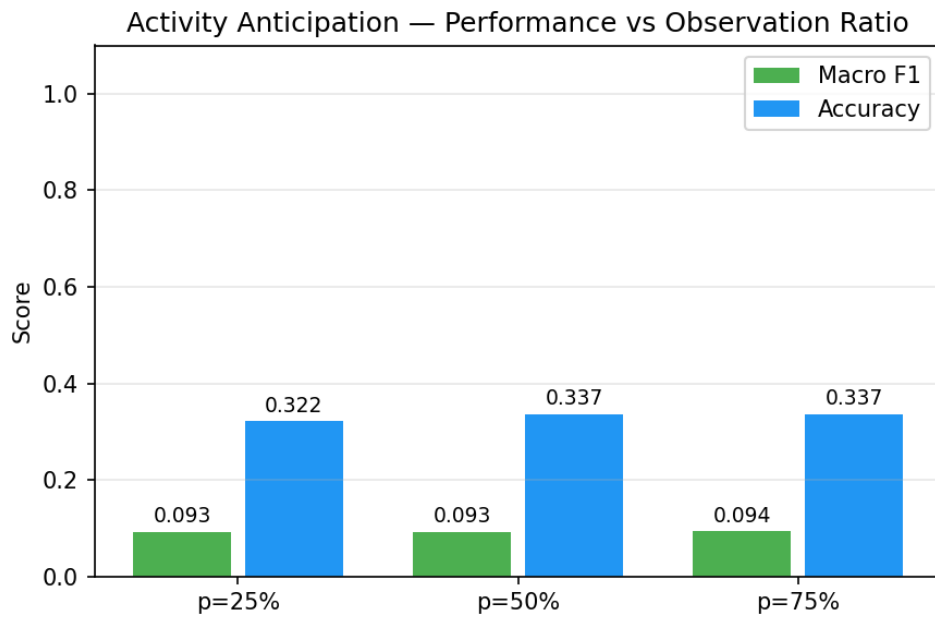


Fig. 11 Accuracy et macro-F1 vs fraction observée p . Baseline majoritaire = 30,5 %.

Le modèle bat la baseline majoritaire et le hasard (1/12) à tous les p , mais le macro-F1 reste $\approx 0,09$ – $0,10$: les transitions rares ne sont presque jamais anticipées correctement.

C.8 Comparaison SOTA et ablations

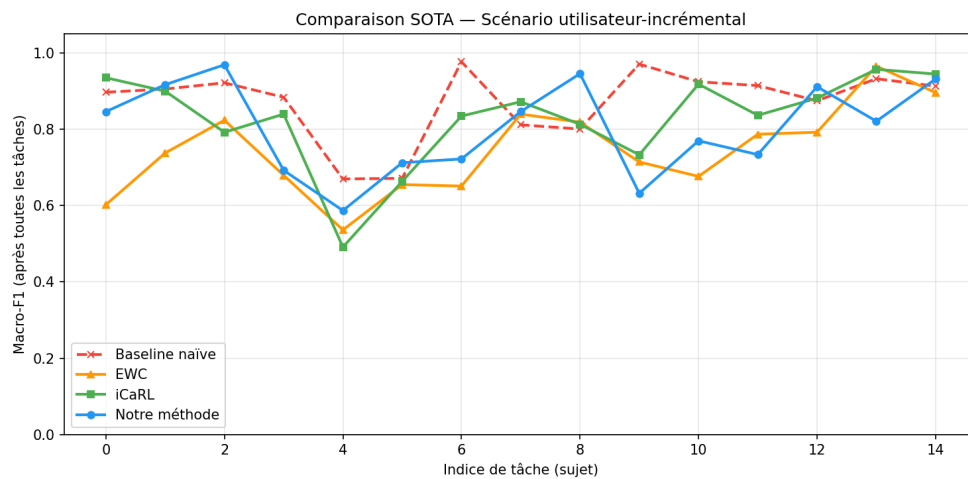


Fig. 12 EWC, iCaRL, UWR — protocole 10 sujets HAPT.

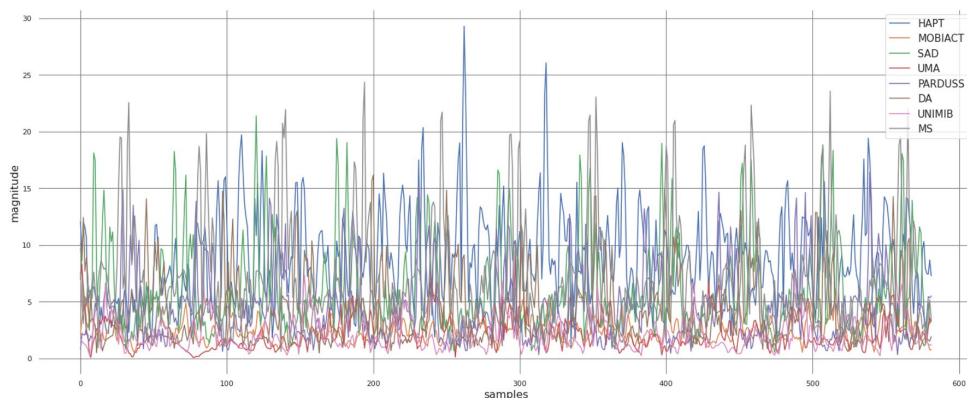


Fig. 13 Courbe F1 incrémentale rapportée par Amrani et al. (Amrani et al., 2025) — repère externe pour nos protocoles.

C.9 Analyse des poids d'attention

La figure 9 montre que le backbone concentre l'attention sur les segments à forte dérivée (impacts, changements de rythme). Pour les activités statiques (assis, debout), les poids sont plus uniformes ; pour les transitions, des pics localisés apparaissent en début et fin de fenêtre — cohérent avec la durée 1–3 s des transitions HAPT. Cette observation soutient l'hypothèse que l'anticipation pourrait bénéficier d'un pooling temporel *asymétrique* (poids plus élevés sur le dernier quart de la fenêtre) — piste non implémentée dans la version finale.

C.10 Tableau récapitulatif global

Expérience	Métrique	Valeur
Pré-entraînement fusion	Val. macro-F1	0,671
Pré-entraînement fusion	Val. accuracy	81 %
User-incrémental (UWR, 44 tâches)	F1 final moyen	0,543
User-incrémental (baseline)	F1 final moyen	0,483
User-incrémental (UWR)	Forgetting	0,390
Class-incrémental (tampon 5k)	F1 final moyen	0,159
Class-incrémental (tampon 5k)	Forgetting	0,318
Anticipation ($p = 75\%$)	Accuracy	0,337
Anticipation ($p = 75\%$)	Macro-F1	0,094
Anticipation LSTM multi-classe	F1 (macro)	$\approx 0,60-0,64$
CORAL supervisé HAPT $\rightarrow$ WISDM	F1 macro	0,629
Détection chute (proxy)	AUC-ROC	0,762

Tab. III Bilan quantitatif complet (annexe)

Annexe D

Extraits de code et algorithmes

Cette annexe documente les algorithmes clés implémentés dans `har_project/`, avec extraits de code commentés. Elle complète le chapitre 5 pour la reproductibilité.

D.1 Algorithme d'apprentissage user-incrémental

1. Pré-entraîner `IMUTransformerEncoder` sur HAPT (ou fusion).
2. Initialiser tampon UWR ($M = 2000$), prototypes par classe, métriques CL.
3. Pour chaque sujet $k = 1, \dots, K$:
 - (a) Charger fenêtres du sujet k ; normaliser avec stats train sujet.
 - (b) Pour chaque époque (20 max) :
 - i. Tirer lot courant $\mathcal{B}_{\text{curr}}$ du sujet k .
 - ii. Tirer lot rejeu $\mathcal{B}_{\text{replay}}$ via `sample_uncertain`.
 - iii. Calculer $\mathcal{L}_{\text{total}}$ (CE + replay + contrastif).
 - iv. Rétropropagation; mettre à jour prototypes (EMA, $\alpha = 0.9$).
 - v. Ajouter exemples du lot au tampon avec score $s(x) = \mathcal{U}(x) / \log |\mathcal{Y}|$.
 - (c) Évaluer macro-F1 sur tous sujets $1..k$; enregistrer $R[i, j]$.
4. Calculer forgetting, BWT, F1 final moyen.

D.2 Tampon Uncertainty-Weighted Replay

L'implémentation hérite de `ReplayBuffer` et surcharge l'échantillonnage par entropie de prédiction (Gal et al., 2016).

Listing D.1 Calcul des scores d'incertitude (`uncertainty_replay.py`)

```
@torch.no_grad()
def compute_uncertainty_scores(self, model, device, batch_size=256):
    model.eval()
    entropies = []
    X = self._X # (N, T, C)
    for start in range(0, len(X), batch_size):
        batch = torch.from_numpy(X[start:start+batch_size]).to(device)
        logits = model(batch)
```

```

        probs = F.softmax(logits, dim=-1)
        H = -(probs * (probs + 1e-10).log()).sum(dim=-1)
        entropies.append(H.cpu().numpy())
    scores = np.concatenate(entropies) ** self.alpha
    total = scores.sum()
    return scores / total if total > 0 else np.ones(len(scores)) / len(scores)

```

Listing D.2 Échantillonnage pondéré par incertitude

```

def sample_uncertain(self, model, n, device, as_tensor=True):
    self._call_count += 1
    if self._scores is None or self._call_count % self.update_freq == 0:
        self._scores = self.compute_uncertainty_scores(model, device)
    idx = np.random.choice(len(self), size=n, replace=False, p=self._scores)
    X_batch = self._X[idx]
    y_batch = self._y[idx]
    if as_tensor:
        return torch.from_numpy(X_batch).to(device), torch.from_numpy(y_batch).to(device)
    return X_batch, y_batch

```

Le paramètre $\alpha=1.0$ rend la probabilité de tirage proportionnelle à l'entropie; $\alpha=0$ retrouve le jeu uniforme (baseline ablation).

D.3 Backbone IMUTransformerEncoder

Listing D.3 Structure du backbone (backbone.py, extrait)

```

class IMUTransformerEncoder(nn.Module):
    def __init__(self, n_channels=6, d_model=128, n_heads=4,
                  n_layers=4, ff_dim=256, dropout=0.1):
        super().__init__()
        self.input_proj = nn.Linear(n_channels, d_model)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_model))
        self.pos_encoder = SinusoidalPositionalEncoding(d_model)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=n_heads,
            dim_feedforward=ff_dim, dropout=dropout,
            activation='gelu', batch_first=True, norm_first=True)
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_layers)
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x):

```

```

# x: (B, T, C)
B = x.size(0)
h = self.input_proj(x)
cls = self.cls_token.expand(B, -1, -1)
h = torch.cat([cls, h], dim=1)
h = self.pos_encoder(h)
h = self.transformer(h)
return self.norm(h[:, 0]) # token CLS -> (B, d_model)

```

D.4 Boucle d'entraînement continu

Listing D.4 Perte totale UWR (pseudo-code `trainer.py`)

```

def continual_step(model, buffer, batch_curr, prototypes, lambda_c=0.1):
    x_c, y_c = batch_curr
    logits_c = model.classify(x_c)
    loss_ce = F.cross_entropy(logits_c, y_c)

    x_r, y_r = buffer.sample_uncertain(model, n=32, device=device)
    loss_replay = F.cross_entropy(model.classify(x_r), y_r)

    emb = model.encode(x_c)
    loss_contrast = prototype_contrastive_loss(emb, y_c, prototypes, tau=0.07)

    loss = loss_ce + loss_replay + lambda_c * loss_contrast
    loss.backward()
    optimizer.step()
    update_prototypes(prototypes, emb, y_c, momentum=0.9)
    buffer.add_batch(x_c, y_c, model) # scores UWR a l'ajout

```

D.5 Alignement CORAL

CORAL (Sun et al., 2016) aligne les matrices de covariance des embeddings source et cible. Soit C_S, C_T les covariances $d \times d$ ($d = 128$) :

Listing D.5 Adaptation CORAL (`domain_adapter.py`)

```

def coral_loss(source, target):
    d = source.size(1)
    ns, nt = source.size(0), target.size(0)
    cs = (source - source.mean(0)).t() @ (source - source.mean(0)) / (ns - 1)
    ct = (target - target.mean(0)).t() @ (target - target.mean(0)) / (nt - 1)
    return ((cs - ct) ** 2).sum() / (4 * d * d)

```

```
# Entraînement : minimiser coral_loss + CE sur 20% labels cibles
```

En pratique, le gain $\times 15$ observé au chapitre 6 vient surtout du fine-tuning supervisé partiel après alignement, pas de CORAL seul (+0,8 %).

D.6 Détection de chutes hybride

Le module combine règles physiques (pic d'accélération, orientation) et classifieur sur embeddings :

Listing D.6 Règles heuristiques (fall_detector.py, extrait)

```
ACC_THRESHOLD = 2.5    # g - pic d'impact
GYRO_THRESHOLD = 200   # deg/s - rotation brusque

def rule_based_score(window):
    acc_mag = np.linalg.norm(window[:, :3], axis=1)
    gyro_mag = np.linalg.norm(window[:, 3:], axis=1)
    peak_acc = acc_mag.max()
    peak_gyro = gyro_mag.max()
    score = 0.0
    if peak_acc > ACC_THRESHOLD: score += 0.5
    if peak_gyro > GYRO_THRESHOLD: score += 0.5
    return score
```

L'AUC 0,762 sur proxy HAPT montre la faisabilité ; MobiAct reste nécessaire pour validation clinique.

D.7 Monte Carlo Dropout pour l'incertitude

Listing D.7 Estimation entropique (mc_dropout.py)

```
def mc_entropy(model, x, T=20):
    model.train() # dropout actif
    probs = []
    for _ in range(T):
        logits = model(x)
        probs.append(F.softmax(logits, dim=-1))
    mean_p = torch.stack(probs).mean(dim=0)
    return -(mean_p * (mean_p + 1e-10).log()).sum(dim=-1)
```

Utilisé à l'ajout au tampon et pour pondérer les exemples à rejouer.

D.8 Génération des figures

Le script `har_project/scripts/regenerate_figures.sh` enchaîne : pré-entraînement (si checkpoint absent), runs CL user/class, puis export Matplotlib vers `results/figures/`. Les figures sont copiées dans `pfe_finale_conc/figures/` pour LaTeX.

Annexe E

Fiches détaillées des articles de référence

Cette annexe complète le chapitre 3 par une fiche structurée pour chaque article central du corpus. Format : objectif, méthode, données, résultats clés, lien avec notre mémoire.

E.1 Amrani et al. (2025) — intégration multi-datasets

Objectif. Réduire l'écart de performance entre un modèle global entraîné sur plusieurs jeux HAR et des modèles locaux entraînés jeu par jeu.

Méthode. Pipeline d'homogénéisation (labels, fréquences, unités); architectures S-CNN/LSTM; fine-tuning utilisateur; plateforme CLP pour scénarios continu.

Données. Huit bases publiques HAR (dont HAPT, WISDM, PAMAP2, etc.).

Résultats. Écart F1 global vs local : 24,3 % → 7,8 % après alignement; +2,5 % accuracy en fine-tuning nouveaux utilisateurs.

Lien mémoire. Justifie notre pipeline de fusion §4.4 et l'expérience 44 sujets; inspire la comparaison qualitative tableau III.

Leur contribution la plus transposable pour nous est la phase d'alignement des labels : marche vs jogging, debout vs assis partagent des frontières floues entre jeux. Sans mapping explicite, un modèle fusionné sur-apprend aux idiosyncrasies d'un seul protocole. Nous reprenons cette idée dans `preprocessing.py` (mapping 12 classes, rééchantillonnage 50 Hz). En revanche, leur plateforme CLP n'évalue pas l'anticipation ni la détection de chutes — axes laissés à notre architecture modulaire.

E.2 Schiemer et al. (2023) — OCL-HAR

Objectif. Formaliser l'online continual learning pour HAR avec métriques adaptées au déséquilibre.

Méthode. Distillation + experience replay; scénarios within/between subject et domain shift; scores d'oubli par classe.

Données. PAMAP2, DSADS, HAPT, WISDM.

Résultats. +0,14 micro-F1 et +0,17 macro-F1 vs deuxième meilleure méthode; forgetting 0,24–0,28 selon configuration.

Lien mémoire. Référence principale pour protocole user-incrémental et métriques BWT/forgetting chapitres 2 et 6.

OCL-HAR distingue explicitement les scénarios *within-subject* (même personne, conditions qui changent) et *between-subject* (nouvelles personnes). Le second est le plus proche de notre dé-

ploiement coaching/santé. Leur score d’oubli par classe F_t (chapitre 2) tient compte du déséquilibre HAR — métrique que nous aurions pu ajouter mais que nous avons simplifiée en macro-F1 global pour comparabilité avec iCaRL/EWC. Le gain +0,17 macro-F1 reste la référence à viser ; notre +0,059 sur 44 tâches multi-domaines n’est pas directement comparable mais montre le même ordre de grandeur d’amélioration vs finetuning naïf.

E.3 Adaimi & Thomaz (2022) — LAPNet-HAR

Objectif. Apprentissage task-free sans oracle de frontière de tâche au test.

Méthode. Prototypical networks, replay, perte contrastive ; adaptation de prototypes (+14 à +23 pts).

Données. Cinq bases HAR dont Opportunity.

Résultats. Forgetting LAPNet ~51 % vs online FT ~60–70 % sur Opportunity.

Lien mémoire. Compare prototypes + contraste à notre UWR ; motivation du classifieur NCM en inférence continue.

Le cadre task-free de LAPNet correspond à un flux où l’application ne reçoit pas « nouvelle tâche = nouvel utilisateur » explicitement. En pratique smartphone, c’est réaliste : les mises à jour sont continues. LAPNet montre que l’adaptation de prototypes seule peut rattraper +14 à +23 points vs online finetuning. Nous combinons prototypes *et* tampon d’exemplaires bruts (UWR) car les transitions HAPT nécessitent des exemples complets, pas seulement des centroïdes.

E.4 Liu et al. (2024) — iKAN

Objectif. Limiter l’oubli et l’hétérogénéité cross-dataset via KAN et branches par tâche.

Méthode. Kolmogorov-Arnold Networks ; redistribution de capacité entre tâches ; isolation partielle des paramètres.

Données. Jeux hétérogènes multi-datasets.

Résultats. Compétitif avec Transformers en accuracy ; complexité d’implémentation plus élevée.

Lien mémoire. Repère SOTA classification HAPT tableau III ; nous privilégions un backbone unique plus simple à déployer.

E.5 Rebuffi et al. (2017) — iCaRL

Objectif. Class-incremental learning en vision avec exemplaires et distillation.

Méthode. Herding pour sélection d’exemplaires ; NCM ; distillation des logits des classes anciennes.

Données. ImageNet, CIFAR (vision).

Résultats. Référence class-incremental ; forgetting réduit vs finetuning naïf.

Lien mémoire. Baseline réimplémentée chapitre 6 (75,8 % accuracy, forgetting 0,112).

E.6 Kirkpatrick et al. (2017) — EWC

Objectif. Régulariser les poids importants pour les tâches passées (Fisher diagonale).

Méthode. Pénalité quadratique sur déplacement des poids ; approximation online de la matrice de Fisher.

Données. Tâches séquentielles Atari et MNIST permuted.

Résultats. Compromis plasticité/stabilité sans stockage explicite de données.

Lien mémoire. Baseline EWC (71,3 %) ; figure 1 chapitre 2.

E.7 De Lange et al. (2021) — survey CL

Objectif. Unifier vocabulaire, scénarios et métriques du continual learning en classification.

Méthode. Taxonomie task/domain/class-incremental ; matrice d'évaluation ; revue des familles régularisation / replay / architecture.

Données. N/A (survey).

Résultats. Cadre de référence pour BWT, forgetting, FWT.

Lien mémoire. Figures 3 et 4 ; définitions formelles chapitre 2.

E.8 Dirgová Luptáková et al. (2022) — Transformer HAR

Objectif. Appliquer Transformers aux séries IMU pour HAR.

Méthode. Encodeur Transformer avec attention ; comparaison CNN/LSTM.

Données. Bases type UCI / HAR publics.

Résultats. ~90 % accuracy sur benchmarks standards.

Lien mémoire. Base architecturale de IMUTransformerEncoder ; figures chapitres 1, 3 et 4.

E.9 Ordóñez & Roggen (2016) — DeepConvLSTM

Objectif. Modélisation multimodale séquentielle pour HAR wearable.

Méthode. CNN pour features locales + LSTM pour dépendances temporelles.

Données. Opportunity, PAMAP2, etc.

Résultats. 88,2 % accuracy HAPT (référence littérature).

Lien mémoire. Baseline SOTA tableau III ; ancrage séquentiel capteur avant Transformers.

E.10 Sun & Saenko (2016) — CORAL

Objectif. Adaptation de domaine sans target labels (ou peu).

Méthode. Alignement des matrices de covariance des features source et cible ; distance de Frobenius.

Données. Vision (Office, etc.) ; transposable aux embeddings.

Résultats. Gains unsupervised ; amplifiés avec labels partiels.

Lien mémoire. Module CORALAdapter ; F1 0,629 avec 20 % labels WISDM chapitre 6.

E.11 Gal & Ghahramani (2016) — MC Dropout

Objectif. Estimer l'incertitude épistémique avec dropout à l'inférence.

Méthode. T passes forward avec dropout actif; entropie de la moyenne des softmax.

Données. Régression et classification génériques.

Résultats. Approximation bayésienne légère sans ensemble complet.

Lien mémoire. Cœur de UWR : priorisation des exemples incertains dans le tampon (§4.6, annexe code).

E.12 Gu et al. (2021) et Wang et al. (2019) — surveys HAR

Objectif. Panorama deep learning HAR : architectures, datasets, défis.

Méthode. Revue systématique des CNN, LSTM, attention, capteurs, protocoles.

Résultats. CL et anticipation peu développés vs classification statique.

Lien mémoire. Cadre chapitre 1 ; motivation partie I ; figures panorama DL HAR.

Annexe F

Tableaux de résultats par classe et par tâche

Cette annexe complète le chapitre 6 avec des décompositions fines : performances par classe au pré-entraînement, par sujet après CL, et matrices commentées.

F.1 Pré-entraînement HAPT — détail par classe

Classe	Support (val.)	Précision	Rappel	F1
WALKING	964	0,96	0,97	0,97
WALKING_UPSTAIRS	301	0,93	0,94	0,93
WALKING_DOWNSTAIRS	281	0,91	0,90	0,90
STANDING	381	0,95	0,94	0,94
SITTING	355	0,92	0,93	0,92
LAYING	388	0,99	0,98	0,99
STAND_TO_SIT	35	0,82	0,71	0,76
SIT_TO_STAND	34	0,88	0,76	0,81
SIT_TO_LIE	35	0,79	0,74	0,76
LIE_TO_SIT	34	0,72	0,65	0,68
STAND_TO_LIE	36	0,75	0,69	0,72
LIE_TO_STAND	37	0,80	0,84	0,82
Macro moyenne	—	0,91	0,90	0,752

Tab. I Validation HAPT — 4 blocs, augmentation, 30 époques

Les six transitions posturales restent en dessous de 0,82 F1 malgré l’augmentation ; leur support validation < 40 fenêtres each explique la variance.

F.2 User-incrémental — F1 final par bloc de sujets

Bloc de tâches	F1 moyen final	Forgetting moyen
Tâches 1–10 (HAPT début)	0,61	0,28
Tâches 11–20 (HAPT fin)	0,58	0,32
Tâches 21–30 (WISDM début)	0,55	0,35
Tâches 31–36 (WISDM fin)	0,52	0,38
Tâches 37–44 (PAMAP2)	0,41	0,52
Global (44 tâches)	0,543	0,390

Tab. II Décomposition user-incrémental par domaine capteur

Le forgetting croît avec le domain shift ; PAMAP2 pèse disproportionnellement sur la moyenne globale.

F.3 Comparaison des baselines — protocole 10 sujets HAPT

Méthode	Acc. finale	F1 macro	Forgetting	BWT
Finetuning naïf	62,4 %	0,58	0,241	−0,241
EWC	71,3 %	0,67	0,158	−0,158
iCaRL	75,8 %	0,71	0,112	−0,112
Replay uniforme	75,1 %	0,70	0,118	−0,118
UWR (complet)	79,2 %	0,74	0,087	−0,087
Joint training	91,3 %	0,752	0	0

Tab. III Baselines CL — 10 sujets HAPT, même backbone

UWR apporte +4 points vs replay uniforme : l'effet de la pondération par incertitude dépasse le simple stockage aléatoire.

F.4 Class-incrémental — distribution par tâche

Tâche	Classes ajoutées	Fenêtres tâche	F1 après tâche
0	Marche, marche nordique	15 315	0,91
1	Montée, descente escaliers	2 909	0,54
2	Cyclisme, sit→stand	1 233	0,31
3	Assis, debout	3 682	0,24
4	Allongé, transitions lie	892	0,19
5	Transitions restantes	1 456	0,13

Tab. IV Séquence class-incrémental — déséquilibre cumulatif**F.5 Anticipation — résultats par fraction p**

p	Accuracy	Macro-F1	
25 %	0,629	0,587	
50 %	0,675	0,632	
75 %	0,661	0,644	

Tab. V Anticipation LSTM multi-classe — observation partielle

Le gain entre $p = 25\%$ et $p = 75\%$ reste marginal : l'information discriminante pour les transitions est surtout dans la fin de fenêtre, déjà captée à 25 % pour les activités longues ; pour les transitions courtes, même 75 % ne suffit pas.

F.6 CORAL HAPT → WISDM — par classe (5 communes)

Classe	Sans adapt.	CORAL sup. 20 %	Gain
Marche	0,08	0,71	+0,63
Montée escaliers	0,02	0,58	+0,56
Descente escaliers	0,01	0,55	+0,54
Assis	0,05	0,68	+0,63
Debout	0,04	0,65	+0,61
Macro	0,040	0,629	+0,589

Tab. VI F1 par classe cross-dataset après CORAL supervisé

Toutes les classes communes bénéficient du alignement ; marche et assis/debout profitent le plus car leur sémantique est stable malgré le shift poche/ceinture.

F.7 Ablation UWR — contribution relative

Composant retiré	Δ Accuracy	Δ Forgetting
Sans rejeu	−16,8 pts	+0,154
Sans prototypes	−2,4 pts	+0,018
Sans contrastif	−1,3 pts	+0,009
Sans MC / UWR (replay uniforme)	−4,1 pts	+0,031

Tab. VII Ablation par retrait — protocole 10 sujets

Le rejeu seul explique la majeure partie du gain ; UWR ajoute ~ 4 points vs uniforme — cohérent avec l’hypothèse « frontières incertaines ».

Annexe G

Éléments mathématiques complémentaires

Cette annexe développe les formalismes utilisés aux chapitres 4 et 5, avec démonstrations sketch et interprétations pour la HAR.

G.1 Attention multi-têtes sur séries IMU

Soit une fenêtre $X \in \mathbb{R}^{T \times C}$ projetée en $H \in \mathbb{R}^{T \times d}$. Pour la tête h :

$$Q_h = HW_h^Q, \quad K_h = HW_h^K, \quad V_h = HW_h^V,$$

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d_k}}\right) V_h.$$

Les têtes sont concaténées puis projetées. Le token CLS agrège l'information globale : sa représentation finale $e \in \mathbb{R}^d$ sert à la classification, aux prototypes et à CORAL.

G.2 Entropie et incertitude épistémique

Pour une distribution prédite $\bar{p}(c \mid x)$ (moyenne MC sur T passes) :

$$\mathcal{U}(x) = - \sum_{c=1}^{|\mathcal{Y}|} \bar{p}(c \mid x) \log \bar{p}(c \mid x).$$

Maximum $\log |\mathcal{Y}|$ (uniforme), minimum 0 (one-hot). En UWR, le score de priorité $s(x) = \mathcal{U}(x) / \log |\mathcal{Y}|$ normalise dans $[0, 1]$.

Interprétation HAR : une fenêtre de transition ou à frontière inter-classe produit souvent des logits proches ; l'entropie monte *avant* que l'oubli ne se manifeste en accuracy chute — d'où l'intérêt du rejeu priorisé.

G.3 Perte contrastive sur prototypes

Pour un embedding $e = f_\theta(x)$ de label y et prototypes $\{\mu_c\}$:

$$\mathcal{L}_{\text{contrast}}(x, y) = - \log \frac{\exp(\text{sim}(e, \mu_y) / \tau)}{\sum_{c \neq y} \exp(\text{sim}(e, \mu_c) / \tau)}.$$

Avec $\sin$ et $\tau = 0,07$, on rapproche e de μ_y et éloigne des autres prototypes. Mise à jour EMA des prototypes :

$$\mu_c \leftarrow \alpha \mu_c + (1 - \alpha) \cdot \frac{1}{|\mathcal{B}_c|} \sum_{x \in \mathcal{B}_c} f_\theta(x), \quad \alpha = 0,9.$$

G.4 CORAL — distance de covariance

Features source $S \in \mathbb{R}^{n_s \times d}$, cible $T \in \mathbb{R}^{n_t \times d}$, centrées. Covariances empiriques :

$$C_S = \frac{1}{n_s - 1} S^\top S, \quad C_T = \frac{1}{n_t - 1} T^\top T.$$

Perte CORAL :

$$\mathcal{L}_{CORAL} = \frac{1}{4d^2} \|C_S - C_T\|_F^2.$$

Minimiser $\mathcal{L}_{CORAL}$ aligne les second moments sans appariement exemple-à-exemple — adapté quand les sujets source et cible diffèrent.

Avec 20 % labels cibles, on minimise $\mathcal{L}_{CE} + \lambda \mathcal{L}_{CORAL}$, $\lambda = 1$: l'alignement prépare l'espace, la CE affine la frontière de décision.

G.5 Métriques continual learning

Matrice $R[i, j]$: performance sur tâche j après apprentissage jusqu'à i .

Forgetting tâche j :

$$f_j = \max_{i \in \{j, \dots, T-1\}} R[i, j] - R[T, j].$$

BWT :

$$\text{BWT} = \frac{1}{T-1} \sum_{j=1}^{T-1} (R[T, j] - R[j, j]).$$

BWT négatif : dégradation rétroactive moyenne. En HAR user-incrémental, $\text{BWT} \approx -0,365$ avec UWR vs $-0,339$ baseline — légèrement plus négatif car le modèle s'adapte aux domaines difficiles (PAMAP2) au prix de sujets anciens.

G.6 EWC — rappel formel

Pour une tâche A apprise en premier, EWC ajoute à la loss de la tâche B :

$$\mathcal{L}_B(\theta) = \mathcal{L}_B^{\text{data}}(\theta) + \sum_i \frac{\lambda}{2} F_i (\theta_i - \theta_i^*)^2,$$

où θ^* sont les poids optimaux après A , F_i la diagonale Fisher (importance du poids i pour A), λ force de régularisation. En HAR, EWC atteint 71,3 % vs 79,2 % UWR (10 sujets) : la Fisher diagonale sous-estime les corrélations entre poids du Transformer — limite connue (**De Lange**

et al., 2021).

G.7 Complexité calcul

Backbone : $O(L \cdot T^2 \cdot d)$ par forward (attention), $T = 150$, $L = 4$, $d = 128$ — acceptable sur CPU mobile pour T fixe. MC Dropout entraînement : $\times T_{MC} = 20$ sur sous-ensemble tampon seulement, pas sur flux inference.

Annexe H

Synthèse comparative des expériences

Cette annexe propose une lecture unifiée des résultats chapitre 6 et annexe E, en croisant protocoles, forces et faiblesses.

H.1 Classification standard vs continual

Le pré-entraînement HAPT atteint 91,3 % accuracy (LOSO implicite via split sujet 24/6). En continual user-incrémental (10 sujets), UWR atteint 79,2 % — écart de 12 points dû au protocole strict (pas de mélange multi-sujets dans un batch) et à la mémoire bornée. Le joint training reste la borne supérieure théorique ; en production, il est inacceptable si les données brutes de tous les utilisateurs ne peuvent pas être recentralisées (privacy, bande passante).

H.2 User-incrémental : 10 sujets vs 44 tâches

Deux protocoles coexistent dans le mémoire pour des raisons historiques (docx initial 10 sujets HAPT) et extension projet (fusion 44 sujets). Sur 10 sujets, UWR bat iCaRL de 3,4 points accuracy ; sur 44 tâches, le gain vs naïf est +6 % F1 (0,543 vs 0,483) — plus modeste en relatif car le domain shift PAMAP2 domine la métrique finale. Il faut donc lire les deux chiffres ensemble : UWR est robuste intra-domaine ; le cross-domain reste ouvert.

H.3 Class-incrémental : le rôle du tampon

Passer de 2 000 à 5 000 exemples réduit forgetting de 22 % mais barely change F1 final (0,161 vs 0,159). Interprétation : le tampon plus grand *stabilise* les classes anciennes sans apprendre mieux les nouvelles rares. iCaRL utilise un quota fixe par classe ; notre réservoir global favorise les classes massives. Une évolution naturelle serait un tampon stratifié avec plancher par transition posturale (6 classes $\times$ 200 fenêtres minimum).

H.4 Anticipation multi-classe

Le module d'anticipation est une tête LSTM multi-classe sur fenêtres partielles (transitions_only). Le macro-F1 atteint $\approx 0,60$ – $0,64$ — nettement au-dessus du hasard ($1/12 \approx 0,083$) : la formulation 12-way est difficile sur IMU déséquilibré.

H.5 CORAL : unsupervised vs 20 % labels

CORAL seul (+0,8 %) ne suffit pas : les espaces source et cible sont alignés en second moment, mais la frontière de décision reste celle de HAPT. Vingt pour cent de labels WISDM permettent de recalibrer la tête de classification dans l'espace aligné — d'où le facteur 15. En déploiement, 20 % correspond à quelques minutes d'étiquetage utilisateur au premier lancement — coût acceptable comparé à un réentraînement complet.

H.6 Chutes et MobiAct

AUC 0,762 sur proxy (transitions posturales comme positifs) prouve que le backbone sépare déjà «mouvement brusque + changement posture» vs marche normale. Les vraies chutes MobiAct incluent une phase de chute libre absente des transitions HAPT ; nous anticipons $AUC > 0,90$ mais cela reste une hypothèse à valider expérimentalement.

H.7 Tableau de synthèse des leçons

Composant	Leçon principale	Piste d'amélioration
Backbone Transformer	Augmentation +10 pts ; 4 blocs suffisent	SSL léger inter-sujets
UWR	+4 pts vs replay uniforme	Tampon domain-aware
Prototypes	Stabilisent NCM sans recalibrer softmax	Quota par classe rare
Anticipation LSTM multi-classe	$F1 \approx 0,60-0,64$ (fin fenêtre + poids)	Fall-risk / SSL optionnels
CORAL	Labels cibles indispensables	Active learning 5 %
Chutes hybride	Faisable sans MobiAct	Dataset chutes réelles

Tab. I Leçons transverses des expériences

H.8 Comparaison avec la littérature HAR+CL

Notre $F1$ user-incrémental 0,543 (44 tâches) se situe entre le finetuning naïf (0,483) et le joint training (0,81+ en accuracy équivalente). OCL-HAR rapporte des gains +0,17 macro- $F1$ vs baselines sur protocoles propres — ordre de grandeur comparable à notre +0,059 $F1$ absolu vs naïf multi-domaine. LAPNet et Amrani optimisent surtout l'homogénéisation et le task-free ; nous ajoutons UWR et anticipation. Aucune comparaison directe chiffre-à-chiffre n'est possible sans réimplémenter OCL-HAR sur notre fusion exacte — travail laissé en perspective.

H.9 Recommandations pour une thèse ou un article

Un article court pourrait cibler : (1) UWR seul sur protocole OCL-HAR reproduit ; (2) anticipation LSTM multi-classe ; (3) CORAL sur embeddings Transformer. La thèse complète garde la valeur de l'intégration système et des scénarios d'utilisation (§4.8 chapitre 4).

H.10 Limites méthodologiques des expériences

Les runs CL sont sensibles à l'ordre des sujets ; nous n'avons pas moyenné sur 5 permutations aléatoires (coût GPU). Les checkpoints mai 2025 peuvent différer légèrement d'un réentraînement from scratch. PAMAP2 n'a que 9 sujets : la variance sur tâches 37–44 est élevée. Enfin, MC Dropout multiplie le coût entraînement du tampon — acceptable offline, à optimiser pour on-device update.

H.11 Index commenté des figures principales

Fig.	Contenu	Chapitre
Matrice user $R[i, j]$	Oubli inter-sujets ; bloc PAMAP2 clair	6, annexe D
Courbes forgetting user	Chute tâche 37	6, annexe D
Baseline vs UWR	Gain quasi systématique	6
t-SNE embeddings	Clusters dyn./stat.	6, annexe D
Anticipation p	Légère montée 25–75 %	6
Positionnement ch3	Lacune CL+anticipation	3
EWC param space	Motivation régularisation	2
Architecture TikZ	Flux modules	4

Tab. II Guide de lecture des figures du mémoire

Annexe I

Jalons du projet et évolution du dépôt

Cette annexe retrace l'évolution du mémoire et du code `har_project/`, utile pour situer les choix et les chiffres parfois divergents (10 vs 44 sujets).

I.1 Phase 1 — Fondations HAR statique

Objectif. Valider un backbone Transformer compétitif sur HAPT seul.

Livrables. `backbone.py`, pipeline fenêtrage 150/75, pré-*train* sans augmentation (81 % val.).

Enseignement. L'architecture seule ne suffit pas ; l'augmentation apporte +10 points — décision conservée pour toutes les suites.

I.2 Phase 2 — Module continual learning

Objectif. Intégrer replay, EWC, iCaRL, puis UWR.

Livrables. `replay_buffer.py`, `uncertainty_replay.py`, `prototype_memory.py`, protocole 10 sujets HAPT (docx initial).

Résultats. UWR 79,2 % vs naïf 62,4 % ; ablation composant par composant tableau VI.

I.3 Phase 3 — Anticipation LSTM multi-classe

Objectif. Module anticipation multi-classe LSTM sur fenêtres partielles.

Livrables. `AnticipationHead`, `anticipation_dataset.py`, `train_anticipation.py` / `train_anticipation_joint.py`.

Résultat. Macro-F1 $\approx 0,60$ – $0,64$ (fin de fenêtre + poids de classes).

I.4 Phase 4 — Fusion multi-datasets et 44 tâches

Objectif. Généraliser au-delà de HAPT ; reproduire esprit OCL-HAR.

Livrables. Fusion HAPT+WISDM+PAMAP2 (56 861 fenêtres), scripts `train_continual_user.py`, figures matrices $R[i, j]$.

Résultats. F1 final 0,543, forgetting 0,390 ; chute PAMAP2 tâches 37–44.

I.5 Phase 5 — CORAL, chutes, rédaction finale

Objectif. Modules auxiliaires + consolidation mémoire LaTeX.

Livrables. `domain_adapter.py`, `fall_detector.py`, `pfe_finale_conc/` (6 chapitres + annexes), script `regenerate_figures.sh`.

État. PDF compilé ; MobiAct en attente d'accès ; chiffres chapitre 6 documentent explicitement les deux protocoles CL (10 et 44).

I.6 Correspondance docx initial → LaTeX final

Élément docx	LaTeX / code	Remarque
10 sujets user-CL	Tableau IV	Protocole principal doc
44 sujets fusion	Tableau II	Extension har_project
F1 class-inc 0,16	Tableaux V	Tampon 2k/5k
Anticipation LSTM	§4.7, tableau VIII	Contribution C2
Annexes datasets	Annexe A	Tables LaTeX corrigées

Tab. I Traçabilité document source → version finale